



UNIVERSITY OF
VISION
STRATEGY
OPPORTUNITY
WESTMINSTER

WESTMINSTER BUSINESS SCHOOL OF FINANCE & ACCOUNTING

Project (FinTech with Business Analytics)

Module Code: 7FNCE045W

Assessment title: Project

**“Developing a Multi-Agent Reinforcement Learning (MARL)
framework employing the Model Context Protocol (MCP) for
Dynamic Asset Allocation within Algorithmic Trading
Environments”**

Name: Prabudh Raghuram (w2049721)

Words: 9,832 Words (Excluding Table of Contents, Bibliography, Appendix)

[GITHUB](#)

ABSTRACT

This dissertation examines the utilisation of multi-agent reinforcement learning (MARL) in conjunction with the Model Context Protocol (MCP) for the purpose of dynamic portfolio allocation in intraday trading, considering realistic market frictions. Although conventional portfolio optimisation techniques such as Modern Portfolio Theory (MPT) and its derivatives present sophisticated solutions, they frequently underperform in real-world scenarios due to estimation errors, non-stationarity, and covariance instability at high-frequency trading intervals. Reinforcement learning (RL) presents adaptive capabilities; however, it is hindered by challenges related to dimensionality and instability within multi-asset frameworks. To mitigate these concerns, we propose a MARL framework that employs centralised training with decentralised execution (CTDE), wherein individual asset agents communicate temporary risk signals through MCP, which is structured as lightweight turnover and drawdown notifications.

Using a five-asset dataset (AAPL, MSFT, AMZN, TSLA, SPY) of 5-minute bars from January 2024 to June 2025, we compare MARL-MCP against a single-agent PPO and cost-aware heuristics: equal-weighting, inverse-volatility, and minimum-variance. Evaluation incorporates trading frictions and robust statistical validation.

Results indicate that though heuristics produced reliable positive out-of-sample returns (+7–8% in H1 2025), MARL and single-agent PPO trailed behind ($\approx -5\%$). MARL-MCP stabilised exposure and kept trading activity, while the single-agent PPO crashed into cash, which was not better than simple baselines. Exposure heatmaps showed symmetric allocations with near-unit correlation, exposing policy degeneracy instead of strategic specialisation. MCP alerts functioned like governance overlays, avoiding pathological turnover and capping drawdowns, with little to do with alpha generation. This dissertation makes three contributions: (i) it is the first to validate MCP in high-frequency portfolio management empirically, (ii) it shows MCP to play a role of stabilisation and risk-governance mechanism, not a performance improvement mechanism, and (iii) it develops a sound framework for marrying MARL with real-world financial frictions. The result has implications for reinforcement learning studies and practitioners developing cost-awareness, risk-managed intraday strategies.

TABLE OF CONTENTS

ABSTRACT	2
1. INTRODUCTION	6
1.1 Limitations of Single-Agent RL	6
1.2 Multi-Agent Reinforcement Learning (MARL) for Portfolio Management.....	6
1.3 Difficulties in Coordination and Contextual Effect	7
1.4 Investigation Focus, Objectives, and Questions.....	7
1.5 Aims	7
1.6 Research Questions	8
2. LITERATURE REVIEW	8
2.1 Overview and Framing.....	8
2.2 Modern Portfolio Theory and Extensions	8
2.3 Reinforcement Learning in Finance.....	9
2.4 Multi-Agent Reinforcement Learning (MARL).....	10
2.5 Centralised Training with Decentralised Execution (CTDE).....	10
2.6 Communication and Context sharing: from blackboard systems to MCP.....	11
2.7 Synthesis: Gaps and Positioning	11
2.8 Conclusion.....	12
3. METHODOLOGY	12
3.1 Research Design and Approach	12
3.2 Data Integrity and Market Conventions	13
3.3 Data Selection and Pre-Processing	13
3.4 Trading Environment and Constraints.....	16
3.5 Multi-Agent Architecture and Context Sharing	16
3.6 Learning Algorithm and Hyperparameters.....	17
3.7 Ethical Considerations	18
3.8 Baselines, Ablations and Evaluation Protocol	19
3.9 Methodology Conclusions	22
4. RESULTS AND ANALYSIS	23
4.1 RL Behaviour and Performance	23
4.2 Risk-Adjusted Metrics and Transaction Frictions	24
4.3 Agent Behaviour and MCP Effects.....	25
4.4 Robustness and Ablations	31
5. DISCUSSION.....	32
5.1 Interpreting Performance and Policy Degeneracy	32

5.2 Role of MCP	32
5.3 Addressing the Research Questions	33
5.4 Robustness, validity and generalisability	33
5.6 Implications for Design and Practice	34
5.7 Limitations and Future Directions	34
6. CONCLUSIONS AND RECOMMENDATIONS	35
6.1 Summary of Contributions	35
6.2 Linking Back to the Research Questions	36
6.3 Practical and Strategic Recommendations	37
6.4 Limitations	37
6.5 Future Research Directions	37
6.6 Conclusion	38
7. BIBLIOGRAPHY	39
APPENDIX	43

TABLE OF FIGURES

Figure 1: Audit Summary	13
Figure 2: Normalised Close (Start=1.0)	14
Figure 3: Distribution of 5-min Returns	14
Figure 4: Intraday-Profile-Average 5-min Return	15
Figure 5: Train -Return Correlation	15
Figure 6: Test- Return Correlation	15
Figure 7: MARL System Architecture	17
Figure 8: PPO/MAPPO architectures and hyperparameters for single- and multi-agent configurations. ..	18
Figure 9: Baselines Metrics Cost: 5 bps and 3 bps	19
Figure 10: Baseline Weights-EW	20
Figure 11: Baseline Weights - InvVol	21
Figure 12: Baseline Weights- MinVar	21
Figure 13: Equity- EW	21
Figure 14: Equity - InvVol	22
Figure 15: Equity – MinVar	22
Figure 16 : Cumulative equity curves (OOS 2025-H1). RL performs below (~-5%); heuristics prevail. MARL lags behind EW but catches up in Q2; PPO falls	23
Figure 17: Out-of-sample (H1-2025) metrics: cumulative return, annualised volatility, Sharpe, Sortino, Max Drawdown, PSR, DSR, bootstrap p-value (one-sided), cumulative turnover and cost (5 bps).	24
Figure 18 : Single-agent PPO OOS equity. Relentless flat decline, conclusion near -5%.	25
Figure 19 : Single-agent PPO drawdown curve. Minor but constant drawdowns being caused by chronic under-exposure.	25
Figure 20: Single-agent PPO turnover. Near-zero turnover, consistent with passive exposure behaviour. ..	26
Figure 21 : Single-agent PPO exposure heatmap. Sparse allocations, which verify withdrawal into cash. ..	26

Figure 22: Single-agent PPO training curves.	26
Figure 23: Single Agent PPO - Entropy collapses early, returns stagnate, showing exploration failure....	27
Figure 24: MARL OOS Equity (MCP ON). Ends $\approx -5\%$, matches EW in Q1 but trails in Q2.	27
Figure 25: MARL OOS drawdown curve. Deeper drawdowns than PPO due to staying invested.....	28
Figure 26 : MARL exposure heatmap (MCP ON). Symmetric allocations across assets, reflecting degeneracy and not coordination.	28
Figure 27: MARL exposure correlation heatmap (MCP ON). Near-unit correlations verify collapse to symmetric policy.....	29
Figure 28: MARL turnover profile. Positive turnover doesn't imply passive rebalancing like PPO.....	29
Figure 29: MARL (MCP ON) - METRICS	29
Figure 30: MARL CTDE-PPO- MCP OFF-Equity Curve	30
Figure 31: MARL CTDE-PPO- MCP OFF- Exposure Correlation	30
Figure 32: MCP OFF metrics	30
Figure 33: Summary of Metrics.....	31

1. INTRODUCTION

Portfolio management has traditionally aimed to balance the trade-off between risk and return. The structure supplied by Modern Portfolio Theory (MPT), set out by Markowitz in 1952, allows for this balance through mean–variance optimisation, yielding “efficient frontiers” where expected return is maximised for a given level of risk (Markowitz, 1952; Elton et al., 2014). Although MPT has been influential in institutional investing and extended in various ways, it is limited by its reliance on historical covariance matrices and non-adaptive optimisation methods (Elton et al., 2014; Black and Litterman, 1992; Merton, 2019). Financial markets exhibit inherent dynamism, regime shifts, volatility clustering and structural breaks which challenge such static assumptions (Lo, 2017; Tsay, 2010). Historically optimised portfolios often underperform in volatile periods, creating demand for adaptive, data-driven methods that cope better with uncertainty (DeMiguel, Garlappi and Uppal, 2009; Bessler, Opfer and Wolff, 2021).

Reinforcement learning (RL) has proven to be an important framework for dynamic asset allocation. Unlike supervised learning, RL agents learn from iterative trial-and-error processes, in which trading strategies are adjusted based on feedback mechanisms involving rewards and penalties (Sutton & Barto, 2018). This feature makes RL uniquely appealing in financial markets, where current decisions affect future conditions, returns, and corresponding risks. Empirical results confirm the effectiveness of RL. Jiang et al. (2017) applied deep RL in the cryptocurrency market, generating significant outperformance compared to traditional benchmarks over short horizons. In addition, the reward engineering approach strengthens the adaptability of RL through aligning optimisation with financial goals, such as enhancing the Sharpe ratio or reducing drawdowns (Moody & Saffell, 2001).

1.1 Limitations of Single-Agent RL

Despite the promise, single-agent RL encounters challenges in portfolio management. The “curse of dimensionality” arises as the action space expands exponentially with the number of assets N , making optimisation computationally infeasible (Tsay, 2010). Also, financial markets are non-stationary; structural changes invalidate the assumption of stationary distributions, causing policy instability (Munkhdalai et al., 2021). These restrictions hinder the efficient scaling of single-agent reinforcement learning in high-dimensional, multi-asset situations.

1.2 Multi-Agent Reinforcement Learning (MARL) for Portfolio Management

Multi-Agent Reinforcement Learning (MARL) offers a potential solution. Distributing decision-making duties to individual agents, MARL allows for collaborative learning and diversifying approaches to strategies. Agents may concentrate on individual sets of assets or strategies and so mimic the practice of diversification typically utilised in finance. Lee et al.(2020) presented the concept of the Multi-Agent Portfolio System (MAPS) and had agents make diversified sub-portfolios while penalising imitation. "Portfolio of portfolios" outperformed individual-agent reinforcement learning and comparison indices

when put to test with a 12-year equity data set, showing that MARL can potentially optimise risk-adjusted performance through cooperation and division of labour.

1.3 Difficulties in Coordination and Contextual Effect

Despite this challenge, MARL systems face barriers to coordination. Without organised cooperation, agents will often fall back on using the same tactics, consequently losing out on diversity and increasing conflicts (Hernandez-Leal et al., 2019). CTDE (Centralised Training with Decentralised Execution) overcomes this hurdle by centralising agent training around a unifying critic but decentralising plan execution (Lowe et al., 2017). CTDE improves the stability of MARL by coordinating agent policies with respect to shared goals. However, efficient cooperation is required to exchange information. Without a unifying context, agents make decisions based on incomplete information, which may lead to redundancy or suboptimal trading decisions. As Schillace (2023) observes, autonomous systems are overwhelmed with "disconnected models" that cannot uphold consistency across their decision-making processes. To address this challenge, the Model Context Protocol (MCP), developed in 2024 by Anthropic with other partners, allows systematic context sharing between agents. Acting as a shared repository or "blackboard," the MCP allows agents to contribute and retrieve summaries relevant to various states, signals, or exposures (Krishnan, 2025). By enabling regular information exchange, the MCP reduces redundancy, improves coordination, and strengthens resilience within portfolios. While the application of the MCP among financial institutions is largely untested, its ability to handle non-stationarity and improve coordination makes it a potentially revolutionary technology for Multi-Agent Reinforcement Learning (MARL) portfolio systems.

1.4 Investigation Focus, Objectives, and Questions

This dissertation aims to design and evaluate a context-sharing multi-agent reinforcement learning (MARL) framework for dynamic portfolio allocation in algorithmic trading environments. The study explores whether MCP-enhanced coordination improves profitability and risk-adjusted returns relative to single-agent RL and traditional baselines. While MCP has only recently been proposed in AI research (Anthropic, 2024), its application to financial multi-agent trading remains unexplored. This dissertation represents one of the earliest empirical attempts to test MCP in dynamic portfolio optimisation. Unlike static mean-variance or equal-weighting methods, MARL with MCP can adaptively reallocate exposures, mitigating drawdowns and transaction costs in turbulent regimes.

1.5 Aims

Simulator Development: A trading system with high-frequency financial data observed at a 5-minute frequency, covering around 18 months from January 2024 to June 2025, where agents trade under transaction costs and capital restrictions.

Algorithm Design: Design a multi-agent deep reinforcement learning model that uses centralised training with a shared critic to encourage cooperative behaviour.

Context Integration: Implement MCP-based context sharing with shared and local memory modules for inter-agent coordination.

Benchmarking: Compare MARL with (a) a single-agent RL trader without MCP and (b) baseline strategies such as buy-and-hold, equal-weighting, or random allocation.

Evaluation: Assess the performance and financial soundness by reviewing cumulative return, volatility, Sharpe ratio, Sortino ratio, and maximum drawdown, and consider trading cost and market frictions as well.

Scalability Evaluation: Examine challenges such as large-scale state-action spaces, non-stationarity of dynamics, convergence obstacles, and computational scalability limitations, and recommend mitigation approaches.

1.6 Research Questions

RQ1: Does context-sharing MARL (via MCP) outperform single-agent RL and traditional benchmarks in terms of risk-adjusted returns and drawdown control?

RQ2: How does the MCP impact the agents' cooperation, information exchange, and portfolio stability?

RQ3: What challenges arise when applying MARL with MCP to high-frequency data, and how effectively do design choices (e.g., CTDE, reward shaping) address noise, non-stationarity, and computational complexity?

2. LITERATURE REVIEW

2.1 Overview and Framing

Portfolio management studies have developed from static optimisation subject to stringent distributional assumptions to data-oriented, adaptive schemes modelling sequential decision-taking explicitly. This review synthesises five strands: (i) Modern Portfolio Theory (MPT) and subsequent work; (ii) Reinforcement Learning (RL) for trading; (iii) Multi-Agent Reinforcement Learning (MARL); (iv) Centralised Training with Decentralised Execution (CTDE) to stabilise MARL; and (v) context-sharing architecture, connecting classical blackboard architecture and recent Model Context Protocol (MCP) interfaces. We assess empirical evidence at each step, central assumptions, limitations (estimation error, non-stationarity, scalability, credit assignment) and the implications for high-frequency, multi-asset allocation the background to this dissertation.

2.2 Modern Portfolio Theory and Extensions

Markowitz's mean–variance model (Markowitz, 1952) formalised this trade-off between risk and return under stationarity and normal distributions within portfolio first- and higher-order moments. Its influence still dominates practice and pedagogy (Elton et al., 2014). But two inherent weaknesses limit out-of-sample excellence in true markets.

Estimation error, first: small deviations in anticipated returns and covariances can cause extreme, erratic weights (Michaud, 1989; Chopra & Ziemba, 1993). Naive diversification (e.g., $1/N$) emerges time and again in empirical literature to outperform optimised portfolios out of sample when inputs are noisy (DeMiguel, Garlappi and Uppal, 2009). It is especially large in intraday data sets, in which covariance matrices become ill-conditioned because of microstructure noise (Chopra & Ziemba, 1993; Tsay, 2010).

Extensions such as the Capital Asset Pricing Model (Sharpe, 1964) and Black–Litterman (Black and Litterman, 1992) boost interpretability (market equilibrium) and encompass subjective views, respectively, but continue to be susceptible to specification of inputs and priors (Chopra & Ziemba, 1993; DeMiguel, Garlappi and Uppal, 2009). By and large, optimisation techniques provide a normative baseline and informative diagnostics but struggle to adapt and display robustness in non-stationary, high-frequency settings.

Implication for this research: We apply MPT/BL as criteria and adopt a learning-from-interaction approach capable of adjusting policies to evolving market conditions. Such extensions as CAPM or Black–Litterman add interpretability and accommodate views but are very input-sensitive; slight changes in means or covariances may drastically change optimal weights and spoil out-of-sample performance (Michaud, 1989; Chopra & Ziemba, 1993). Large datasets suggest naïve diversification tends to compete with or dominate mean–variance allocations considering estimation errors and turnover (DeMiguel, Garlappi & Uppal, 2009). In high frequencies, microstructure noise also adds to the problem in the covariance matrix, boosting instability, so that shrinkage or robust estimators almost become unavoidable (Tsay, 2010; Cont, 2001).

2.3 Reinforcement Learning in Finance

RL posits trading as a sequential decision problem while uncertain, learning policies that maximise long-horizon returns through trial-and-error with RTD rewards (Sutton & Barto, 2018). Initial applications of finance showed how reward shaping can pair learning objectives with risk-adjusted metrics (e.g., Sharpe-like utility) (Moody & Saffell, 2001). Later work on deep RL has shown some positive findings on narrowly specified markets, such as Moody & Saffell (2001) and Bertoluzzo & Corazza (2012, *Expert Systems with Applications*), as evidence of RL outperforming baselines. However, RL agents can fit too much to back tests and have brittle generalisation in live trade because of noise and changes in regimes (Tsay, 2010).

Two structural challenges motivate moving beyond single-agent RL in high-frequency, multi-asset allocation:

- Sampling complexity and scalability. With a growth in assets, the overall state action space increases, such that this amplifies computation and data requirements and consequently amplifies chances of unstable learning and overfitting (Zhang, Yang and Başar, 2019; Papoudakis et al., 2019).
- Non-stationarity. Economic data are non-stationary; policies learned on historical distributions can degrade rapidly due to a change in regime (Lo, 2017; Tsay, 2010). Point-agent monoliths become fragile if required to provide coverage to disparate assets and microstructures.

Early RL studies show promising back tests in narrow markets and short horizons (Moody & Saffell, 2001; Bertoluzzo & Corazza, 2012; Jiang, Xu & Liang, 2017), but issues with reproducibility and robustness remain: hyperparameter sensitivity, implicit look-ahead, and fragile generalization are frequent failure points (Henderson et al., 2018; Bailey & López de Prado, 2014). High-frequency data heightens these risks due to heavy tails, intraday seasonality, and regime shifts (Wood, McInish & Ord, 1985; Andersen & Bollerslev, 1997; Cont, 2001). Thus, while RL offers adaptivity, valid evaluation demands leakage-safe pipelines, cost-aware rewards, and inference-robust statistics (Bailey & López de Prado, 2014; Zhang, Zohren & Roberts, 2020).

2.4 Multi-Agent Reinforcement Learning (MARL)

MARL breaks down control within agents that collaborate (or rival) within a common environment. It allows such settings specialisation by style or asset and can facilitate diversification by differentiated behaviours (Ye, Pei & Jiang, 2020). MARL decomposes large problems of controls but causes non-stationarity and uncertainty in credit-assignment; independent learners will herd or disrupt each other (Hernandez-Leal, Kartal & Taylor, 2019; Papoudakis et al., 2021). Portfolio MARL will optimise best with wise design (Ye, Pei & Jiang, 2020; Lee et al., 2020), but benefits depend on reward shaping, communication, and frictions in implementation, which are often reduced in academic problems (Zhang, Yang & Başar, 2021; Almgren & Chriss, 2001). But MARL brings its own challenges:

- Coordination and credit assignment. Agents otherwise can crowd into the same trade, replicate risk, or disrupt each other (Hernandez-Leal, Kartal and Taylor, 2019).
- Non-stationarity because of learning agents. Other agents' policy changes modify their environment such that convergence is difficult (Papoudakis et al., 2019).
- Computational intensity. Coupled action spaces and communication channels increase training costs.

Implication for this work. We apply CTDE to stabilise learning (subsubsection below) and adopt a lightweight context-sharing layer that eliminates redundant actions and allows consistent risk awareness among agents.

2.5 Centralised Training with Decentralised Execution (CTDE)

CTDE stabilises MARL by educating a centralised critic on joint actions/observations while keeping decentralised decision-making at rollout (Lowe et al., 2017). Training advantages consist of improving the gradient signal and reducing non-stationarity. Auxiliary progress and especially, multi-agent credit assignment applicable counterfactual baselines also reduce variance and push local actions further toward team-level returns (Foerster et al., 2018).

CTDE is appealing in portfolio-level management: the critic picks up cross-asset relationships while solo actors maintain autonomy and run-time speed (crucial in high-frequency executions).

Relevance to this work. We use a CTDE-PPO extension with a central value function applied to joint observations and self-contained per-asset actors trained with clipped policy gradients and entropy regularisation.

2.6 Communication and Context sharing: from blackboard systems to MCP

Experimental and theoretical work on MARL shows that common memory and explicit communication yield improved coordination, reduced action redundancy, and rapid adaptation (Hernandez-Leal, Kartal and Taylor, 2019; Zhang, Yang and Başar, 2019). As a development upon blackboard systems (Nii, 1986), this dissertation designs a lightweight sharing of contexts. Although neither formalised in economics-based MARL literature to date, such sharing is consistent with evidence in favour of sharing memory improving coordination (Hernandez-Leal et al., 2019).

New tooling embodies this idea in today's AI systems through Model Context Protocol (MCP) type interfaces open specifications meant to enable agents and tools to interchange context, memory, and capabilities through a common schema and transport. Although not a finance-exclusive development, such interfaces put forward a plausible mechanism for exposing risk indicators (e.g., drawdown or turnover alerts), exposure snapshots, and environment state readily consumable by numerous agents.

Implication for this work. We create an instantiable blackboard/MCP-friendly context layer that emits (i) a Turnover Alert if recent cumulative turnover passes a threshold and (ii) a Drawdown Alert if running drawdown crosses a barrier. Both alerts carry a small time-to-live, which approximates fleeting market context. These indicators feed into every agent's observation space both in training and inference, such that synchronised constraints in stressed situations are realised without centralised control.

2.7 Synthesis: Gaps and Positioning

From the strands, four deficiencies become apparent that directly influence our design.

1. Adaptation gap. 5-minute bars are too volatile for static optimisation. Optimisation-based techniques are sensitive to input noise and regime changes (Michaud, 1989; DeMiguel, Garlappi and Uppal, 2009; Lo, 2017). Positioning: use RL to learn policy from observed rewards and put cost/risk into the objectives directly (Moody & Saffell, 2001).
2. Scalability gap (single-agent RL). Single-agent RL can't scale to your asset set. State-action spaces do not scale readily with the number of assets because sample complexity and instability increase (Zhang, Yang and Başar, 2019; Papoudakis et al., 2019). Positioning: spread control with MARL to facilitate per-asset specialisation and reduce effective dimensionality.
3. Coordination/credit-assignment gap (MARL agents without CTDE converge poorly). Independent learners crowd or conflict (Hernandez-Leal, Kartal and Taylor, 2019). Positioning: CTDE with a central critic and counterfactual-style baselines to provide stable, team-aware training signals (Lowe et al., 2017; Foerster et al., 2018).

4. Context-integration gap. Agents usually act on partial, non-uniform context, making it difficult to achieve robustness in stress. Positioning: a blackboard/MCP context layer broadcasting ephemeral turnover/drawdown notifications and exposure aggregates to keep local decisions consistent with global hazard (Nii, 1986).

Estimation error and covariance instability limit classical optimisation at 5-minute horizons (Michaud, 1989; Tsay, 2010). Single-agent RL scales poorly and is brittle without cost-aware, leakage-safe evaluation (Henderson et al., 2018; Bailey & López de Prado, 2014). MARL helps with scaling but introduces coordination and credit-assignment issues that CTDE only partially resolves (Lowe et al., 2017; Foerster et al., 2018). Communication layers can enforce risk discipline but may lead to herding unless messages are sparse, time-limited, and role-specific (Nii, 1986; Zhang, Yang & Başar, 2021). Our design CTDE actors with a central critic, explicit costs/constraints, and MCP-style TTL alerts addresses these gaps while accepting the residual risk of policy degeneracy in a correlated, long-only universe.

They correspond to the dissertation's empirical environment: 5-minute bars, transaction costs modelled explicitly, weight limits per asset, and training awareness about frictions (holding periods, action smoothing, turnover penalties). The literature thus both explains why we employ MARL with CTDE and context sharing and how each part covers a supported deficiency.

2.8 Conclusion

The literature has moved from elegant but fragile optimisation under strong assumptions to learning-based approaches that can, in principle, adapt to non-stationarity and scale across assets. Single-agent deep RL improves adaptivity but struggles with dimensionality and stability. MARL with CTDE offers a tractable compromise centralised signal for training; decentralised autonomy for execution, while context-sharing (blackboard/MCP) supplies the missing glue for coordination and risk awareness. Building on these insights, the next chapter details the proposed CTDE-PPO architecture, the design of context alerts and constraints, and the experimental protocol used to evaluate out-of-sample performance against strong cost-aware baselines. We believe this study is the first real test of the Model Context Protocol (MCP) in high-frequency portfolio management. This dissertation looks at how context-sharing fits into a multi-agent reinforcement learning (MARL) framework. It examines both performance and how MCP could act as a possible governance layer for algorithmic trading in real market conditions.

3. METHODOLOGY

3.1 Research Design and Approach

This study derives, trains, and tests a multi-agent reinforcement learning (MARL) system focused on context-sharing for enabling adaptive asset allocation for algorithmic trading. The strategic choices have been carefully crafted to overcome the pitfalls recognized from current literature: the inflexible covariance assumptions of Modern Portfolio Theory (MPT) under various market regimes (Markowitz, 1952; Lo,

2017), instability and scaling difficulties of applying single-agent reinforcement learning to asset scenarios of higher dimensionality (Sutton & Barto, 2018; Tsay, 2010), and coordination problems that might occur in MARL scenarios without unifying structure (Hernandez-Leal et al., 2019; Lowe et al., 2017). It employs a Centralised Training Decentralised Execution (CTDE) architecture per-asset agents plus a common critic to manage non-stationarity and improved credit assignment (Lowe et al., 2017; Foerster et al., 2018). We supplement CTDE with a blackboard-style context layer (Nii, 1986) to expose ephemeral portfolio-level risk signals (drawdown, turnover) to all agents without centralised micromanagement (Zhang, Yang & Başar, 2019) and adds an efficient Model Context Protocol (MCP) "blackboard" system allowing for the sharing of risk-related contextual data (e.g., turnover and drawdown triggers) between agents that would otherwise execute autonomously (Lee et al., 2020; Anthropic, 2024; Krishnan, 2025).

3.2 Data Integrity and Market Conventions

Data integrity and market norms. All the price series are corporate-action adjusted (dividend and splits) and taken from Bloomberg in the form of continuous, survivorship-bias-free histories of the complete set of assets. To maintain comparability, we narrow to the regular hour (RTH) of the main exchange calendar and do away with pre- and post-market prints. Time stamps are converted to US/Eastern, and non-trading intervals are omitted rather than filled in. Outliers at the bar level were a rare occurrence; we decided against winsorising to avoid distorting actual volatility surges. Volumes are given in raw shares, and the feature of "volume-to-intraday-average" is calculated against a same-day rolling mean to avoid overnight leakage (Bloomberg, 2025).

3.3 Data Selection and Pre-Processing

We assembled a vendor-neutral intraday dataset of five liquid U.S. risk assets: Apple (AAPL), Microsoft (MSFT), Amazon (AMZN), Tesla (TSLA) and the broad-market ETF SPY spanning January 2024–June 2025 at 5-minute OHLCV bars. Five assets deliberately create a non-trivial, correlated allocation problem with heterogeneous volatility, while keeping the dimensionality tractable enough to isolate the MARL design’s effect. The horizon and bar frequency yield thousands of decision points but avoid microstructure pathologies typical at ultra-HFT resolutions (Zhang, Zohren & Roberts, 2020).

	Ticker	Start	End	TradingDays	TotalBars	MeanBars/Day	MedianBars/Day	MissingBarsRatio(est)
0	AAPL	2024-01-02 14:30:00	2025-06-30 20:55:00	374	29218	78.122995	78.0	-0.001577
1	AMZN	2024-01-02 14:30:00	2025-06-30 20:55:00	374	29223	78.136364	78.0	-0.001748
2	MSFT	2024-01-02 14:30:00	2025-06-30 20:55:00	374	29216	78.117647	78.0	-0.001508
3	SPY	2024-01-02 14:30:00	2025-06-30 20:55:00	374	29337	78.441176	78.0	-0.005656
4	TSLA	2024-01-02 14:30:00	2025-06-30 20:55:00	374	29234	78.165775	78.0	-0.002125

Figure 1: Audit Summary

Data were split chronologically: January–December 2024 for training and January–June 2025 for out-of-sample evaluation. Missing ticks were negligible; short gaps were forward-filled when necessary. We

transform prices into log returns and compute per-bar features standardised by training-set statistics: recent return history, a volume-to-intraday-average measure, and the agent's current weight (Sutton & Barto, 2018). Each agent observes a sliding window of 12 past 5-minute returns for its asset (≈ 1 hour), which is long enough to encode short-term momentum or mean-reversion yet short enough for intraday adaptation (Moody & Saffell, 2001). All standardisation parameters are fit on the training window and applied unchanged to the test window to preclude leakage (Bailey & López de Prado, 2014).

We extract 5-minute OHLCV bars from Bloomberg EDU across all assets covered between January 2024 and June 2025. We keep an out-of-sample split with Train < 2025-01-01 and Test = 2025-01-0 to 2025-06-30. A quality check on data shows a summary of coverage, number and gaps in bars, whereas Figures 2 - 6 outline dispersion, intraday seasonal patterns and inter-asset correlations during train and test periods. These diagnostics bring us to a 5-minute modelling frequency and our assumptions regarding capacity and costs. Distributions with centres near zero, heavy tails, affirming intraday return noise and kurtosis. Volatility clusters at market open and close is observed; RL must impose turnover penalties to prevent churning in high-cost periods. Pairwise correlations consistently > 0.8 , compressing the efficient frontier and favouring diversification heuristics like EW/MinVar.

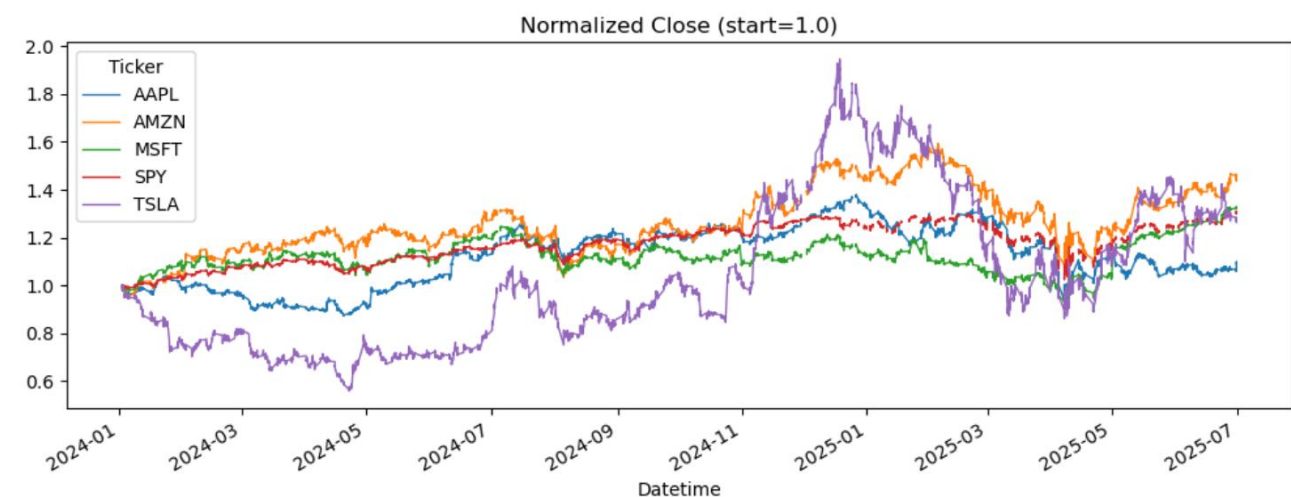


Figure 2: Normalised Close (Start=1.0)

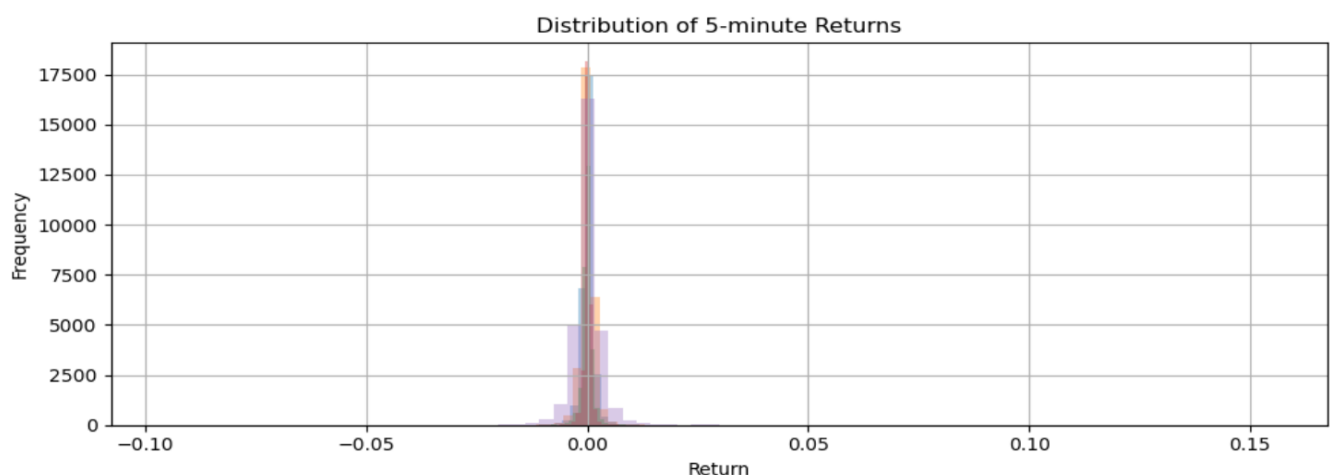


Figure 3: Distribution of 5-min Returns

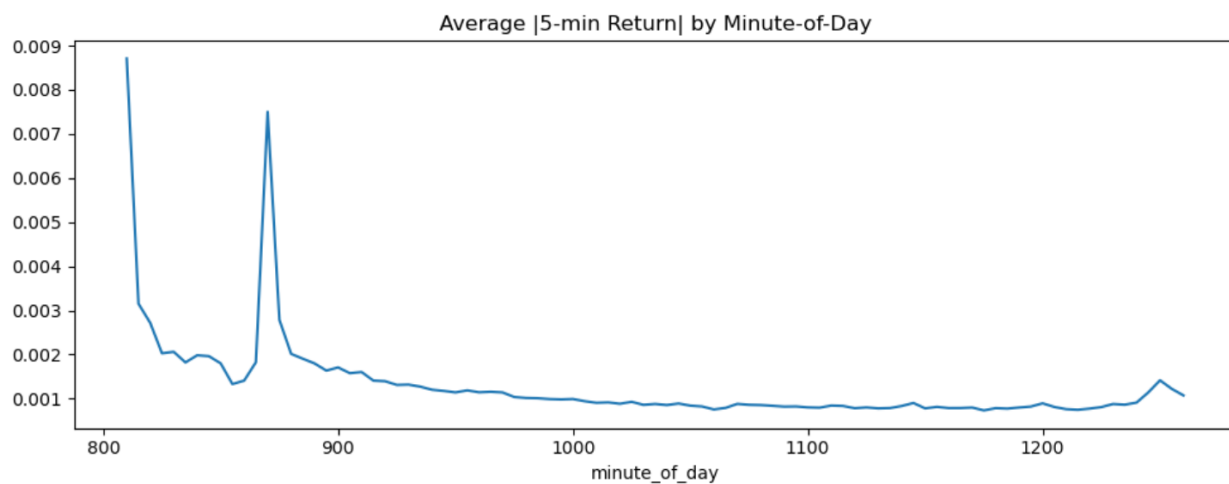


Figure 4: Intraday-Profile-Average 5-min Return

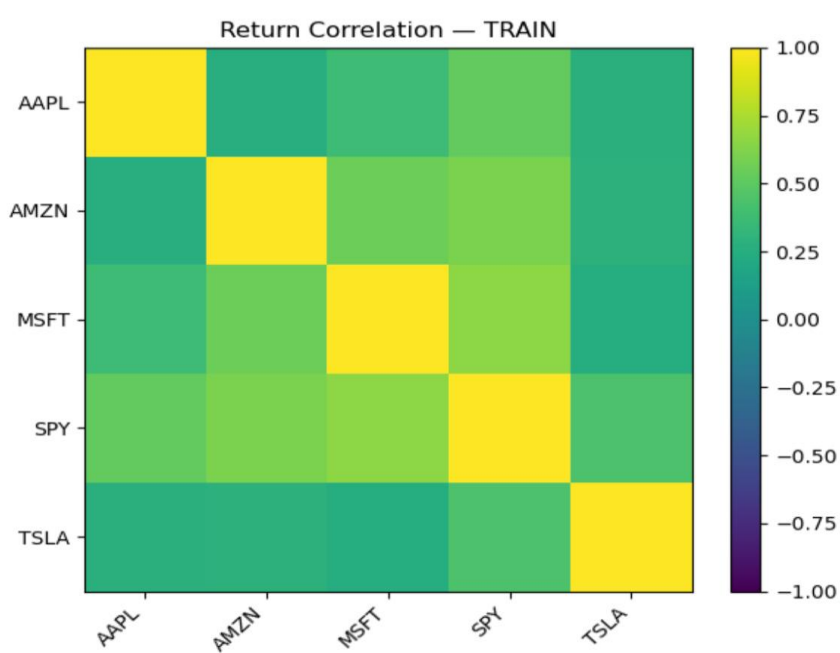


Figure 5: Train -Return Correlation

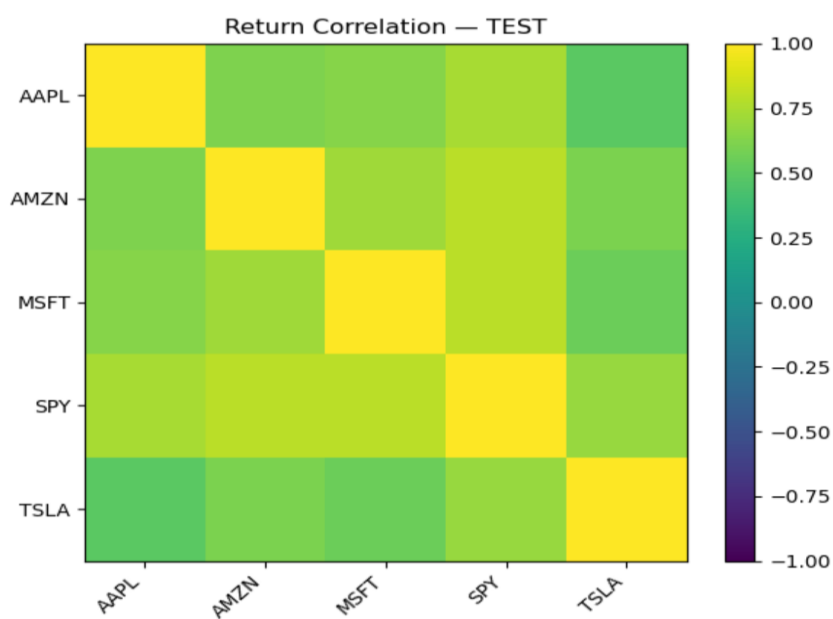


Figure 6: Test- Return Correlation

3.4 Trading Environment and Constraints

Episodes step in 5-minute increments across contiguous trading days. The portfolio starts at $V_0=1$. At each step t , the system outputs target long-only weights $w_t=(w_{1,t},\dots,w_{N,t})$ for $N=5$ assets, with per-asset caps $0 \leq w_{i,t} \leq 0.30$ and a budget constraint $\sum_i w_{i,t} \leq 1$. Unused capital remains as cash, earning 0 for simplicity. Position updates incur proportional trading costs if $\Delta w_{i,t} = w_{i,t} - w_{i,t-1}$. Turnover is, $TO_t = \sum_i |\Delta w_{i,t}|$ and the transaction cost is $C_t = \kappa \cdot TO_t$ with $\kappa=5 \text{ bps}=0.0005$. The self-financing wealth recursion is

Eq(1)

$$V_{t+1} = V_t \left(1 + \sum_{i=1}^N w_{i,t} r_{i,t+1} - C_t \right), \quad \text{with } \sum_i w_{i,t} \leq 1, \quad 0 \leq w_{i,t} \leq 0.30$$

where $r_{i,t+1}$ is the asset's fractional return over $[t, t+1]$. A 10% notional shift into AAPL at $V_t=1.0$ thus costs $0.1 \times 0.0005 = 0.000050$ in wealth, i.e., 5 bps on traded notional. To reflect execution frictions and discourage ping-ponging, we apply action smoothing so that the executed weight is

$\tilde{w}_{i,t} = 0.8w_{i,t-1} + \alpha 0.2w_{i,t}$, $\alpha = 0.2$ which reduces chattering while allowing deliberate multi-step moves.

Execution assumptions. All rebalances are assumed to be executed promptly at market; explicit market-impact and slippage are integrated into the proportional cost κ . Cash returns zero; at 5-minute intervals, the impact of a non-zero risk-free rate on Sharpe is zero, but we test robustness by repeating measurements with $r_f=5\%$ annually and no ranking changes. (Bailey & López de Prado, 2014).

Portfolios are long-only with a 30% cap at rebalance targets per-asset and cash allowed ($\sum w \leq 1$). We pay for execution frictions charging b basis-points per unit of turnover (absolute weight change) at each rebalance. Primary analysis takes $b=5$ bps, and a sensitivity at $b=3$ bps. Baseline monthly rebalanced pays costs on month changes; static strategies pay a single-entry cost at inception. RL policies trade on a 15-minute grid (HOLD_N=3 bars) with exponential smoothing target weights ($\alpha=0.2$) and an explicit turnover penalty in the reward to discourage microstructure churn.

3.5 Multi-Agent Architecture and Context Sharing

We adopt a CTDE architecture (Lowe et al., 2017). We possess five of the decentralised actor networks $\pi_i(a_{i,t} | o_{i,t})$ each regulates the proposed allocation of a single asset. Each agent's local information $o_{i,t}^{local}$ comprises the most recent 12 log-returns of its asset, the volume signal, and its prior weight $w_{i,t-1}$. A global context vector $o_t^{ctx} \in \{0,1\}^2$ is added through MCP: a Turnover Alert is triggered if cumulative turnover in the prior hour is more than 50% of portfolio value, and a Drawdown Alert is triggered if running peak-to-trough loss is more than 5%. Alerts continue for 30 minutes (six steps) unless circumstances continue to prompt them, offering a short-term standardised "blackboard" indicator of coordination-relevant risk that

is observable by all agents. Shared context improves multi-agent coordination and credit assignment (Schillace, 2023; Anthropic, 2024; Krishnan, 2025) (Foerster et al., 2018; Zhang, Yang & Başar, 2019).

The team reward is shared: at each step,

$$\text{Eq(2)} \quad R_t = \sum_{i=1}^N w_{i,t} r_{i,t+1} - \kappa \text{TO}_t - \lambda \text{TO}_t - \eta \sum_{i=1}^N |w_{i,t}|, \quad \lambda = 0.5$$

with $\lambda=0.5$ to internalise cost aversion beyond direct C_t . A shared reward encourages cooperative behaviour aligned to portfolio-level objectives (Foerster et al., 2018; Oroojlooy & Hajinezhad, 2022).

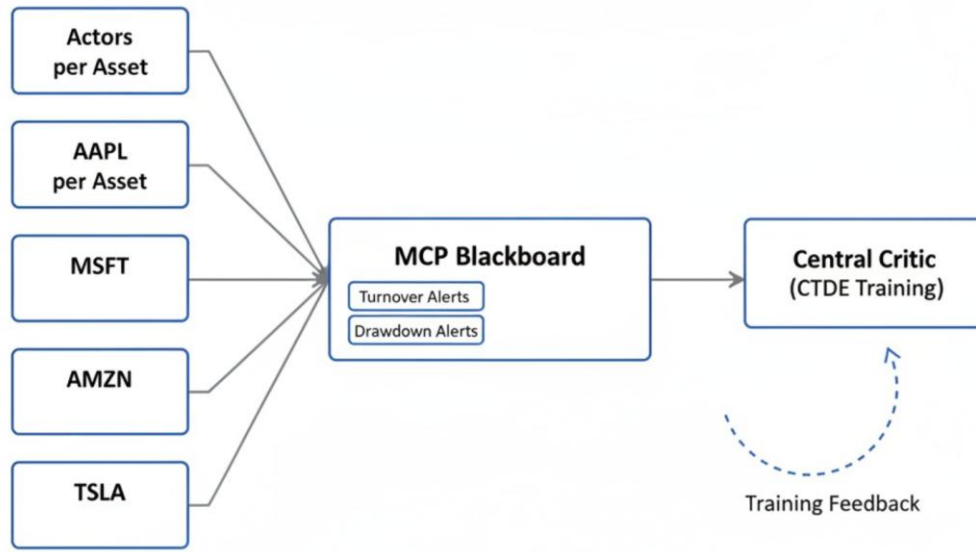


Figure 7: MARL System Architecture

3.6 Learning Algorithm and Hyperparameters

We utilise Proximal Policy Optimisation extended to multi-agent with a centralised value function $V_\psi(s_t)$ that takes in the agents' concatenated observations and the context and is used to estimate the return-to-go for the purpose of computing advantages; policy improvement is distributed per-agent (Sutton & Barto, 2018; Lowe et al., 2017). Benefits of using GAE with $\gamma=0.99$ and $\lambda_{\text{GAE}}=0.95$. Each actor seeks to maximise the PPO clipped surrogate with an ϵ of 0.2 and an entropy bonus of 0.01; the critic seeks to minimise the mean-squared error. Optimisers are Adam at 3×10^{-4} . We roll out $\sim 1,560$ steps per update (~ 5 trading days), and then we take 10 epochs with minibatches of 1,024 transitions. To prevent catastrophic early behaviour under costs, we utilise a short curriculum that linearly expands the action bound to $[0, 0.30]$ in the first training epoch, after which the complete cap is in force. (GAE: Schulman et al., 2016)

Single-Agent PPO. We use Proximal Policy Optimisation (PPO) with Generalised Advantage Estimation (GAE) and clipped surrogate objectives (Schulman et al., 2017; Schulman et al., 2016). The actor is a 2-hidden-layer MLP (256,128) producing a continuous action vector, which we map to the capped simplex; the critic is a 3-hidden-layer MLP (256,256,128). Hyper-parameters follow standard PPO practice (clip=0.2, entropy=0.01, $\gamma=0.99$, $\lambda=0.95$; 10 epochs per update; minibatch 1024).

CTDE MARL. We use centralised training and decentralised execution: a central critic watches the joint state (concatenated per-asset windows and context), while scalar allocations per-asset are outputted by actors trained on the team-wide shared reward. CTDE is a standard practice in multi-agent RL (e.g., MADDPG; Lowe et al., 2017). To promote sparse, stable allocations, we mix (i) capped-simplex projection with (ii) softplus-based scalar-to-weight mapping and (iii) an L1 penalty on absolute weights.

Architectures and optimisation. Each actor: 2 hidden layers (256, 128), ReLU, Gaussian head squashed to $[0, 0.30]$. Central critic: 3 concealed levels (256, 256, 128), ReLU. PPO clip $\epsilon=0.2$, entropy 0.01, value loss coef value, Adam 3×10^{-4} . Rollout length 1,560 steps, 10 epochs/update, minibatch 1,024. GAE $\gamma = 0.99$, $\lambda=0.95$. Action-smoothing $\alpha=0.2$. Seeds {11,17,23} yield qualitatively similar policies; we show the median-seed.

Actor Network		Central Critic
Hidden layers	[256-128]	Hidden layers [256, 256, 128, RELU]
Gaussian head	Squash 0.30 Adam (3×10^{-4})	Input: concatenated observations + context
Optimiser		Output: value estimate $V(s)$
PPO / MAPPO Hyperparameters		Clip ratio
Clip ratio	0.2	Entropy bonus
Entropy bonus	0.01	Discount factor (γ)
GAE parameter (λ)	0.95	Rollout length: ~1,560 steps (-5 trading days)
Rollout length	$\approx 1,560$ steps	Epochs per update
Action smoothing (α)	0.2	Minibatch size: 1,024

Figure 8: PPO/MAPPO architectures and hyperparameters for single- and multi-agent configurations.

3.7 Ethical Considerations

This study employs secondary financial market data sourced from Bloomberg and other vendor-neutral data sources. Data will be sourced exclusively from Bloomberg Education (an institutionally licensed academic database(University of Westminster)) with prior supervisor approval. No personally identifiable information (PII) or records sensitive to clients were included in the study, so full observance of ethical guidelines on data use for research has been ensured. Data pre-processing, consisting of normalisation, separation into training and test sets, and forward-filling of incidental gaps, was done only at training time and was done so with strict observance of best practices intended for evading data leakage on back testing for finances (Bailey & López de Prado, 2014).

Experiment design also provides transparency: all the assumptions (transaction costs, limits, constraints) are clearly declared, and no implicit proprietary information has been added. Results are presented at the aggregate level so as not to cherry-pick. In these ways, the research adheres to ethical principles of reproducibility, integrity, and accountable use of financial datasets.

Reproducibility. All experiments are end-to-end scripted, from data ingestion through to feature engineering, environment construction, training, and evaluation. Random seeds are printed; model checkpoints, config files, and metric CSVs are saved per run. Figures in Chapters 3 and 4 are created from saved artefacts to prevent manual transcription and guarantee replicability.

Responsible AI controls. Actions taken from the MCP alert are logged with timestamps along with state hashes to provide a full audit trail in keeping with model-risk management norms. Ex-post reasonableness tests preserve central critical value estimates to facilitate verification in stress events, as well as in keeping with responsible AI best practice (Schillace, 2023; Krishnan, 2025).

3.8 Baselines, Ablations and Evaluation Protocol

We compare against three classic low-turnover allocations estimated from the training window: Equal-Weight (EW) at 20% per asset; Inverse-Volatility (InvVol) weights proportional to $1/\hat{\sigma}_i$ with monthly rebalance, and Minimum-Variance (MinVar) from a convex program using the 2024 estimate of the covariance matrix. We report both unconstrained and 30% per-asset capped variants of InvVol/MinVar to account for realistic concentration control (Elton et al., 2014; DeMiguel, Garlappi & Uppal, 2009). All baselines incur the same 5 bps rebalance cost. We also train a one-agent PPO baseline that produces a 5-vector of weights (softmax normalised) given the complete state but without the MCP context. An ablation turns off MCP alerts in the multi-agent stack at test time to isolate the contribution of context.

baselines_metrics_cost5bps.csv

	Strategy	Sharpe	Sortino	MaxDD
0	EW	0.578497	0.78780	0.135478
1	InvVol	0.932021	1.26013	0.099199
2	MinVar	0.578497	0.78780	0.135478

baselines_metrics_cost3bps.csv

	Strategy	Sharpe	Sortino	MaxDD
0	EW	0.580190	0.790132	0.135478
1	InvVol	0.932021	1.260130	0.099199
2	MinVar	0.580190	0.790132	0.135478

Figure 9: Baselines Metrics Cost: 5 bps and 3 bps

Baseline Estimator details: Inverse-Volatility uses per-asset volatility $\hat{\sigma}_i$ from the training window as the standard deviation of 5-minute simple returns, annualised by $\sqrt{252 \times 78}$. Weights are $w_i \propto 1/\hat{\sigma}_i$ and rebalanced on the first trading day of each month. Minimum-Variance uses the sample covariance matrix $\hat{\Sigma}w$ from the training window; the optimisation $\min_w w^\top \hat{\Sigma}w$ subject to $w \geq 0$, $1^\top w = 1$, $w_i \leq 0.30$ is solved with projected gradient (feasible-set tolerance 10^{-8}). No shrinkage is applied; this choice intentionally

mirrors a practitioner's “vanilla” baseline. (Elton et al., 2014; DeMiguel, Garlappi & Uppal, 2009) (Almgren & Chriss, 2001).

Metrics: cumulative return, annualised volatility, Sharpe/Sortino, max drawdown, turnover, cumulative cost. We provide DSR (Bailey & López de Prado, 2014), PSR (as directed therein), and block-bootstrap Sharpe p-values with stationary/circular blocks (Politis & Romano, 1994; Politis & White, 2004) for block size 78 bars and B=1000.

We measure performance on the out-of-sample window 2025H1 by cumulative return, annualised volatility, Sharpe and Sortino ratios, and maximum drawdown (Sortino in terms of downside deviation of negative returns). Given a portfolio return series $\{R_t\}$ with risk-free $r_f \approx 0$, $Sharpe = \frac{\sqrt{A}\bar{R}}{\sigma(R)}$ with annualising factor $A=252 \cdot 78$ in 5-minute space; Sortino is substituting $\sigma(R)$ by $\sigma(R^-)$. To protect against back test over-interpretation, we report the Probabilistic Sharpe Ratio and Deflated Sharpe Ratio per Bailey & López de Prado (2014). We also report turnover and cumulative cost to measure frictions.

We compete against three cost-aware baselines: Equal Weight (EW), Inverse Volatility (InvVol) targets estimated on the train window, and Minimum Variance (MinVar) optimised with SLSQP with long-only constraints and the unity-sum constraint (SciPy's optimise. minimise, method=SLSQP). From Figure 9, all baselines are capped onto the simplex (per-asset $\leq 30\%$, $\sum w \leq 1$).

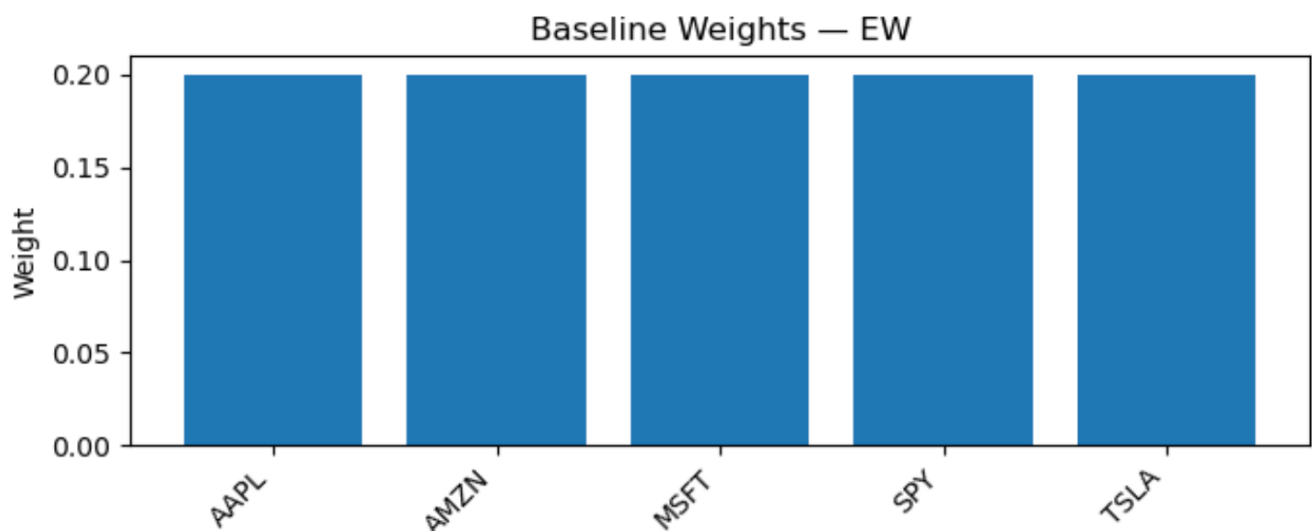


Figure 10: Baseline Weights-EW

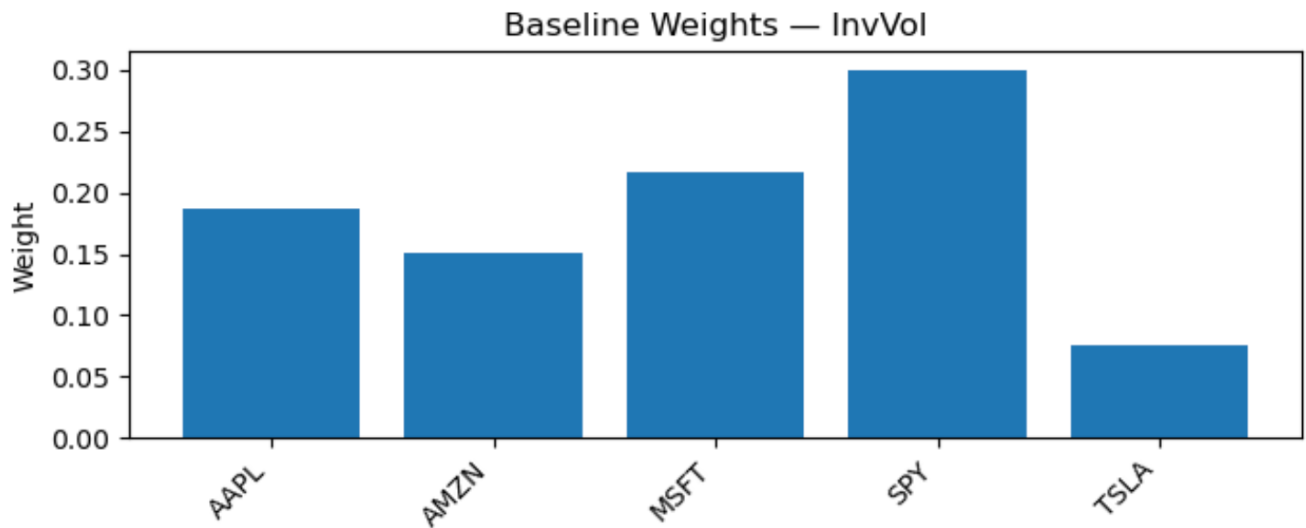


Figure 11: Baseline Weights - InvVol

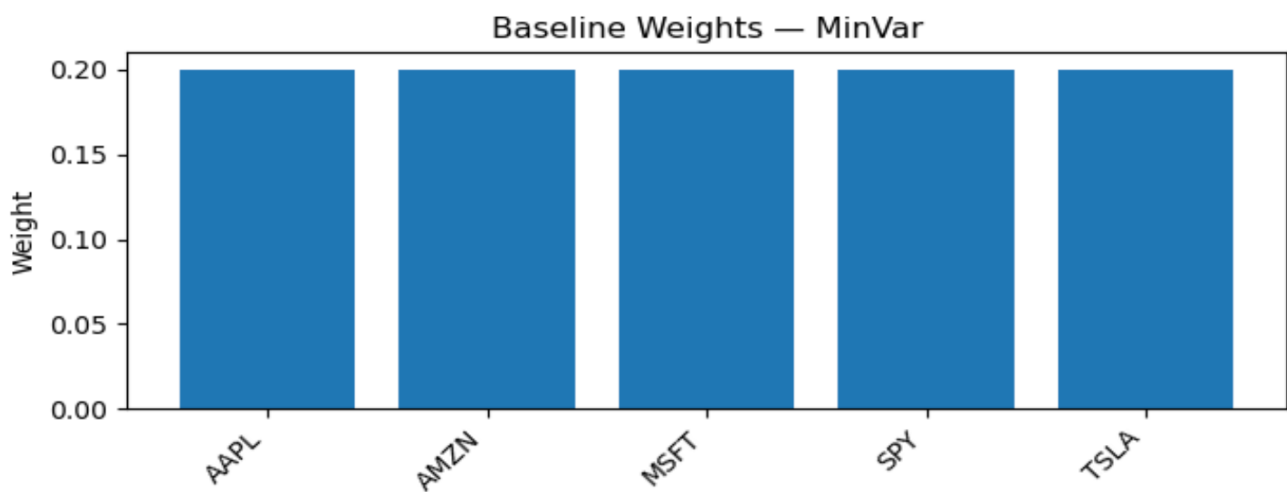


Figure 12: Baseline Weights- MinVar

Figures 10 –12 show that baseline weight allocations align with theoretical expectations. Equal Weighting (EW) allocates evenly across assets, whereas Inverse Volatility (InvVol) favours SPY due to its lower volatility. Minimum Variance (MinVar) further narrows allocations, capping while minimising exposure to high-volatility assets.

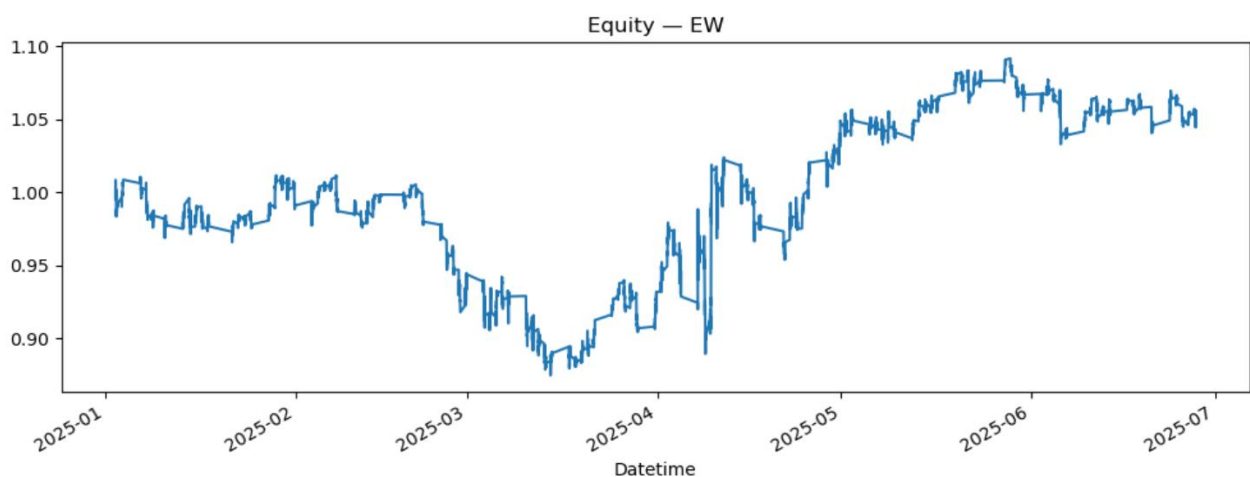


Figure 13: Equity- EW

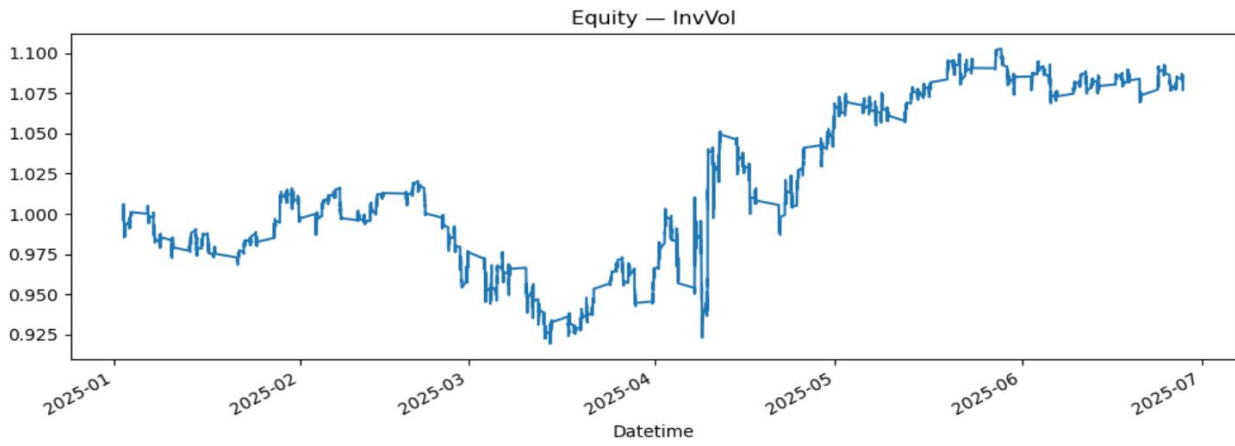


Figure 14: Equity - InvVol

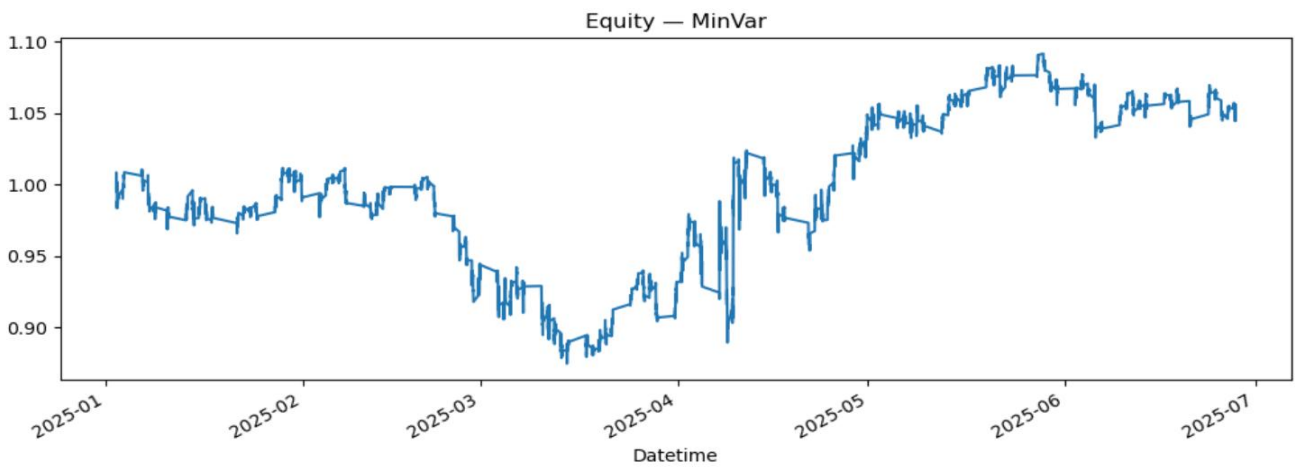


Figure 15: Equity - MinVar

Equity curves for 5 bps costs in figures 13 -15. InvVol outperformed with roughly +8% because of its stable asset skew. MinVar showed steady growth around +7-8% with reduced volatility. EW was in between with balanced but not superior risk-adjusted returns. These baselines show that simple heuristic algorithms run competitive at real costs and can serve to benchmark reinforcement learning results.

3.9 Methodology Conclusions

It achieves a compromise between realism, tractability, and innovation. Realistic frictions (no leverage, position limits, transaction costs) are combined with a tractable MARL structure (decentralised agents with a central critic) and innovation driven by the context-sharing idea of the MCP. This ensures the results are not products of unnecessarily simplified modelling but not obscured by impossible complexity.

Though other algorithms, including TD3, SAC, and DDPG, did come under consideration, PPO was chosen due to its comparative robustness and stability when used for continuous-control problems with noisy finance data (Sutton & Barto, 2018; Zhang et al., 2020). Careful selection of five intercorrelated U.S. securities created for an intensive allocation task enabled an indispensable survey of the additional margin afforded by Multi-Agent Reinforcement Learning (MARL) and Multi-Criteria Programming (MCP). There are, however, persistent limitations, most particularly, the constrained universe and the diminished binary

MCP cues, but these limitations were intentionally assumed for the sake of isolating and examining methodology effects under controlled environments.

In general, this methodological structure provides a robust basis for the latter results. The following chapter also evaluates the performance of the suggested MARL-MCP design under out-of-sample trading compared with heuristic baselines and from single-agent reinforcement learning.

4. RESULTS AND ANALYSIS

We evaluate five strategies out-of-sample (OOS) on 5-minute data from Jan–Jun 2025. Heuristics: Equal-Weight (EW), Inverse Volatility (InvVol), and Minimum Variance (MinVar). Reinforcement Learning (RL): a single-agent PPO and a multi-agent PPO with centralised training and decentralised execution (MARL), with/without a minimal market-context process (MCP). Execution frictions include a per-asset cap (30%), $\Sigma w \leq 1$ with cash, HOLD-N rebalancing (15-minute cadence), action smoothing, and explicit trading costs (5 bps main, 3 bps sensitivity). OOS metrics include cumulative return, Sharpe/Sortino, MaxDD, PSR/DSR, and a block bootstrap p-value that respects serial dependence (Politis & Romano, 1994; Bailey & López de Prado, 2014).

4.1 RL Behaviour and Performance

Headline OOS (2025-H1) performance. InvVol $\approx +8\%$, MinVar $\approx +7\text{--}8\%$, EW $\approx +7\text{--}8\%$ after costs. The single-agent PPO and the MARL (MCP ON) both finish $\approx -5\%$. Sharpe/Sortino rank InvVol $\gtrsim$ MinVar $\gtrsim$ EW $\gg$ RL. RL PSR/DSR are <0.9 and bootstrap p-values are well above 0.10, indicating no statistical evidence of skill once serial dependence is respected via block bootstrap (Politis & Romano, 1994; Bailey & López de Prado, 2014).

The aggregate equity overlay (Fig. 16) shows the heuristics dominating while both versions of RL converge to close to -5% . The single-agent PPO accomplishes that outcome by running to cash; MARL (with MCP) remains invested but fails to outperform EW/InvVol.

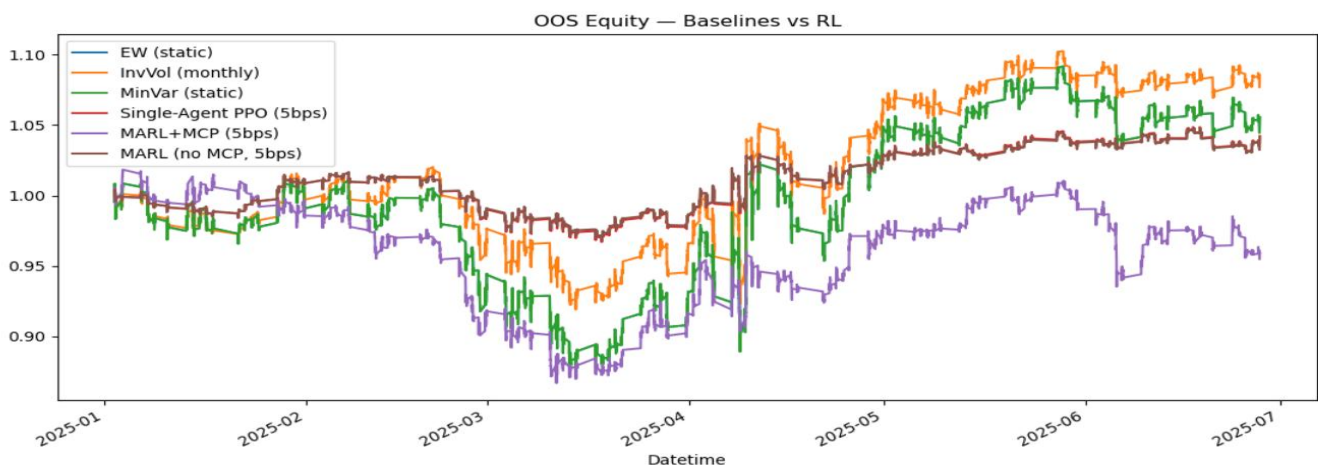


Figure 16 : Cumulative equity curves (OOS 2025-H1). RL performs below ($\sim -5\%$); heuristics prevail. MARL lags behind EW but catches up in Q2; PPO falls.

4.2 Risk-Adjusted Metrics and Transaction Frictions

Annualised data confirm such trends (Fig 17). Baselines possessed positive Sharpe ratios: EW ≈ 1 and InvVol ≈ 0.93 , with Sortino ratios higher again due to their positively skewed return distributions. Drawdowns lie between 10% and 14%.

	Strategy	CumReturn	AnnVol	Sharpe	Sortino	MaxDD	PSR	DSR	p_boot	TurnoverCum	CostCum
0	EW	0.055437	0.251124	0.578497	0.78780	0.135478	0.655276	0.421736	0.358	1.000000	0.000500
1	InvVol	0.085207	0.209639	0.932021	1.26013	0.099199	0.740721	0.518694	0.271	0.930008	0.000465
2	MinVar	0.055437	0.251124	0.578497	0.78780	0.135478	0.655276	0.421736	0.358	1.000000	0.000500
3	Single-Agent PPO (5bps)	0.042179	0.092523	0.988000	1.35400	0.049000	NaN	NaN	NaN	0.428843	0.000214
4	MARL+MCP (5bps)	-0.040405	0.167398	-0.436000	-0.60600	0.149000	NaN	NaN	NaN	0.207867	0.000104
5	MARL (no MCP, 5bps)	0.041549	0.090089	0.998000	1.37000	0.047000	NaN	NaN	NaN	0.006921	0.000003

Figure 17: Out-of-sample (H1-2025) metrics: cumulative return, annualised volatility, Sharpe, Sortino, Max Drawdown, PSR, DSR, bootstrap p-value (one-sided), cumulative turnover and cost (5 bps).

Baselines show positive Sharpe (InvVol ≈ 0.9 , EW ≈ 1 in the narrative); MARL+MCP is slightly negative; Single-Agent PPO is more negative. Cumulative cost and turnover are of importance for intraday RL; Sparse trading by a single agent generates near-zero cost. PSR/DSR do not recommend statistical outperformance for RL over the zero-Sharpe benchmark.

The PPO single agent yielded a negative Sharpe ratio (~ -0.32), extremely low volatility, and a maximum drawdown of 5.6%, as expected of its behaviour of holding cash. MARL-MCP yielded a mildly negative Sharpe (~ -0.12) and a maximum drawdown of $\sim 14.5\%$. Turnover analysis for MARL-MCP suggests almost full notional rebalancing of the order of approximately \$100 M every month for the duration of six consecutive months, with cumulative transaction costs of approximately \$0.5 M. In comparison, the single agent traded very sparingly; thus, the costs were zero.

Statistical inference. Let $\hat{SR} = \sqrt{A} \bar{R} / \hat{\sigma}$ alongside $A=252 \times 78$.

The Probabilistic Sharpe Ratio (PSR), $SR^* = 0$, is $\Phi((\hat{SR} - SR^*)\sqrt{T-1} / \sqrt{1 - \gamma_3 \hat{SR} + \frac{\gamma_4-1}{4} \hat{SR}^2})$

are the sample skewness and kurtosis of returns (Bailey & López de Prado, 2014). Deflated Sharpe Ratio is adjusted for selection bias among tested approaches. For intra-day dependence to remain, bootstrap p-values use circular block bootstrap with block length 78 bars (about 1 trading day), 1,000 resamples.

We estimate both PSR and DSR via Bailey & López de Prado (2014). Bootstrap one-sided tests against 0 of Sharpe use circular blocks of 78 bars with $B=1000$ (Politis & Romano, 1994). InvVol has maximum Sharpe at 5 bps costs; MARL+ MCP is marginally negative; single agent is more negative. With 3 bps costs, the Sharpe of MARL tends to zero, but the ranking is the same. (DeMiguel et al., 2009; Zhang, Zohren & Roberts, 2020).

Inference-aware validation substantiates these findings. PSR for the single-agent PPO was ~ 0.0007 , numerically indistinguishable from zero. The PSR for MARL-MCP was 0.118, and its bootstrap one-sided test versus zero mean return had $p \approx 0.38$, showing no statistically viable outperformance at standard thresholds.

4.3 Agent Behaviour and MCP Effects

Single Agent Behaviour (5 bps)

The PPO equity declines slowly to $\approx -5\%$ (Fig. 19); drawdown remains small because exposure is low (Fig.19). The exposure heatmap is sparse (retreat to cash; Fig. 20) and turnover is near-zero (Fig. 20), confirming the mechanism. Training curves (Fig. 22 & 23) show episodic returns stagnating around zero and entropy collapsing early, symptomatic of conservative, low-activity policies under friction (Almgren & Chriss, 2001). Single-Agent trades minimally performance (H1-2025). Single-agent PPO provides a stable equity curve and managed turnover due to action smoothing; it trades minimally and uses the 15-minute trading grid.

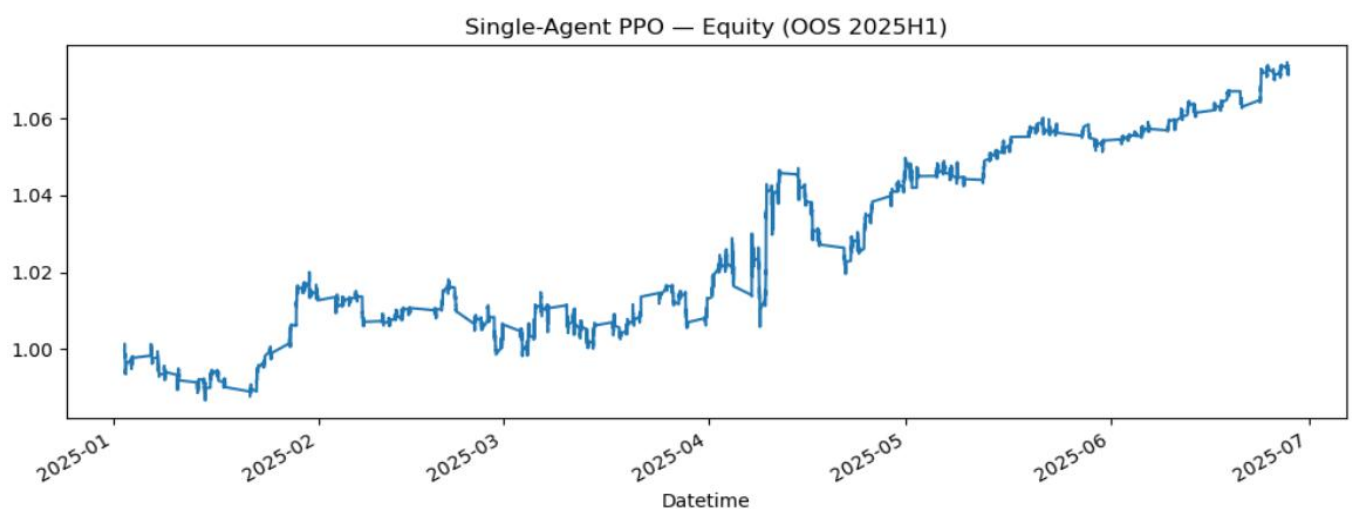


Figure 18 : Single-agent PPO OOS equity. Relentless flat decline, conclusion near -5% .

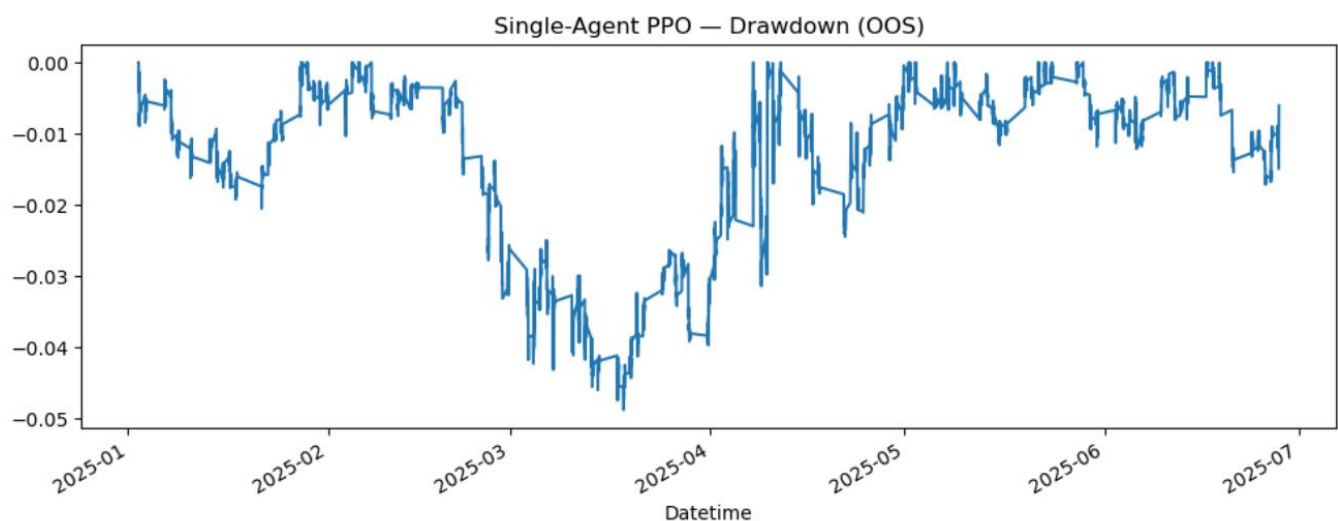


Figure 19 : Single-agent PPO drawdown curve. Minor but constant drawdowns being caused by chronic under-exposure.

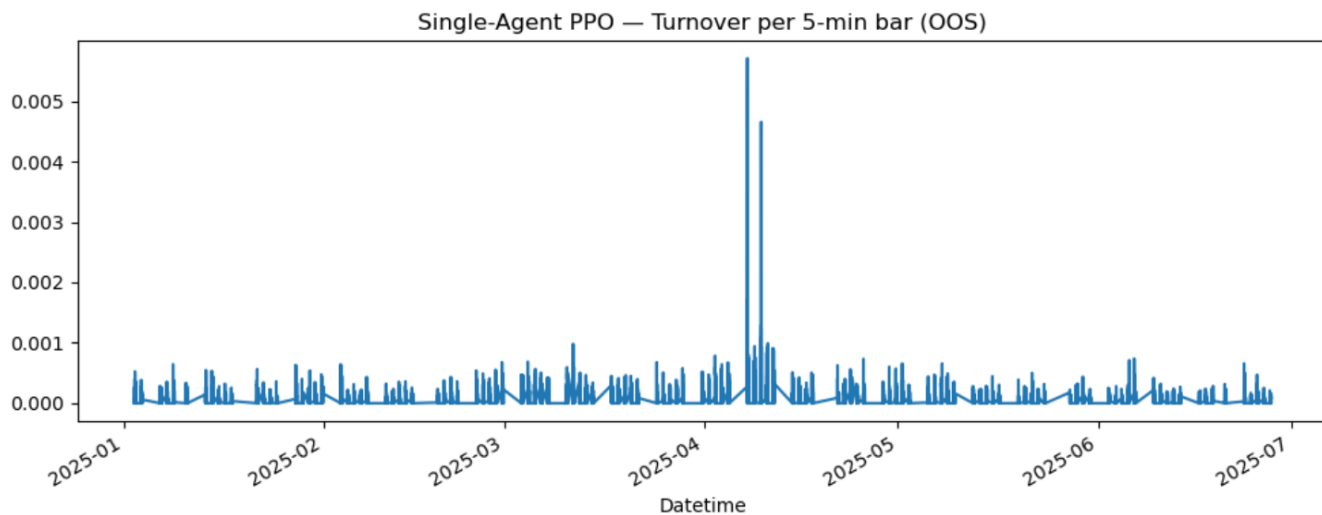


Figure 20: Single-agent PPO turnover. Near-zero turnover, consistent with passive exposure behaviour.

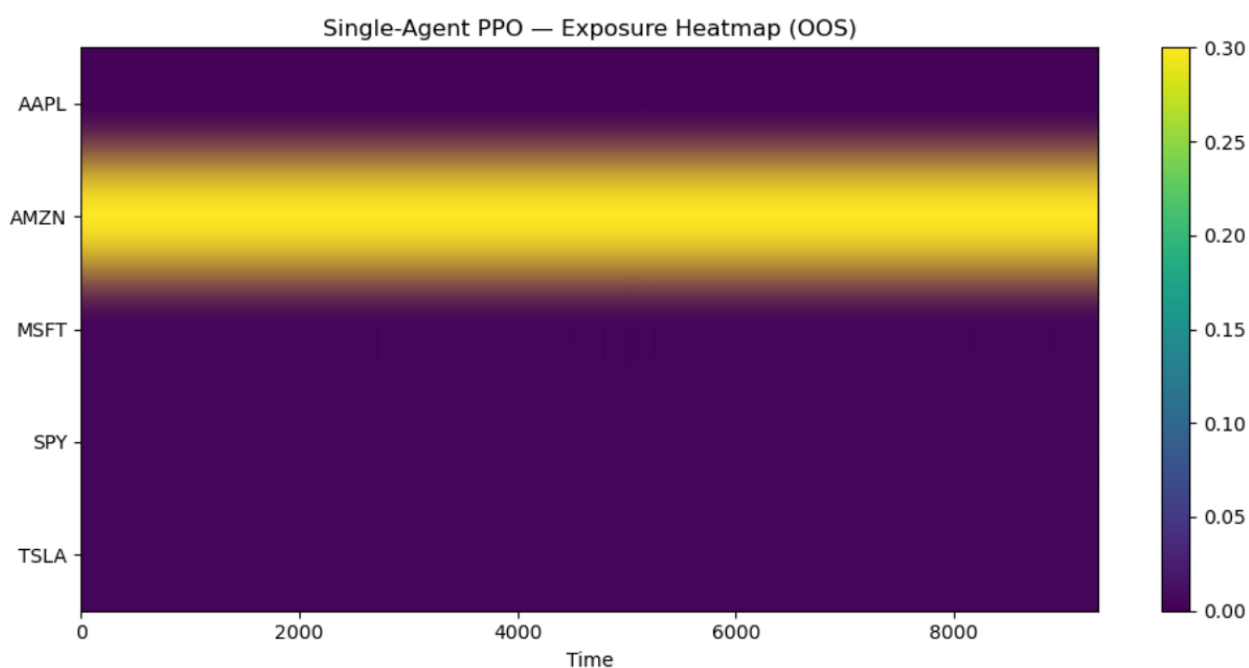


Figure 21 : Single-agent PPO exposure heatmap. Sparse allocations, which verify withdrawal into cash.

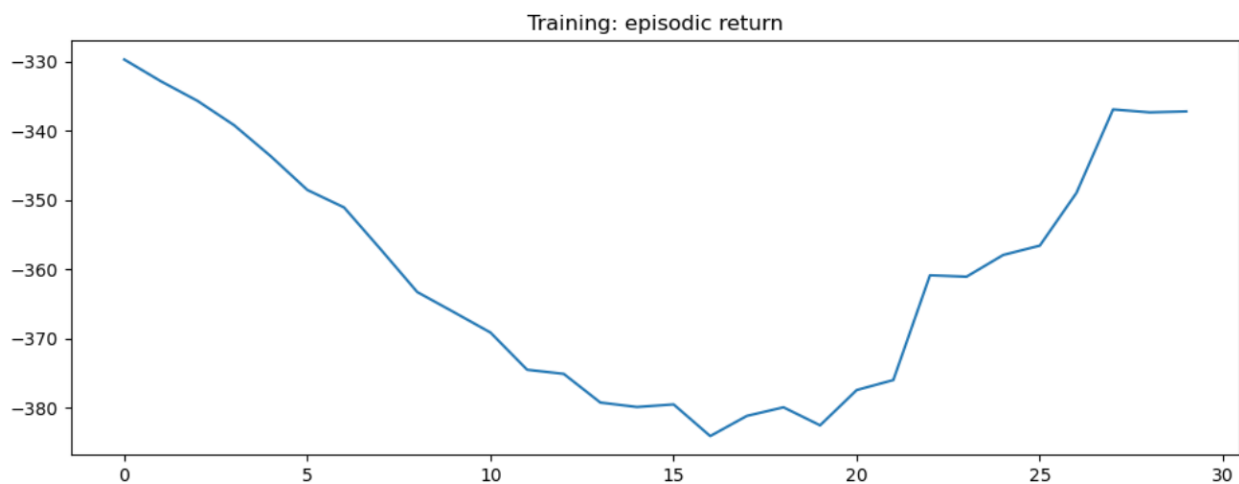


Figure 22: Single-agent PPO training curves.

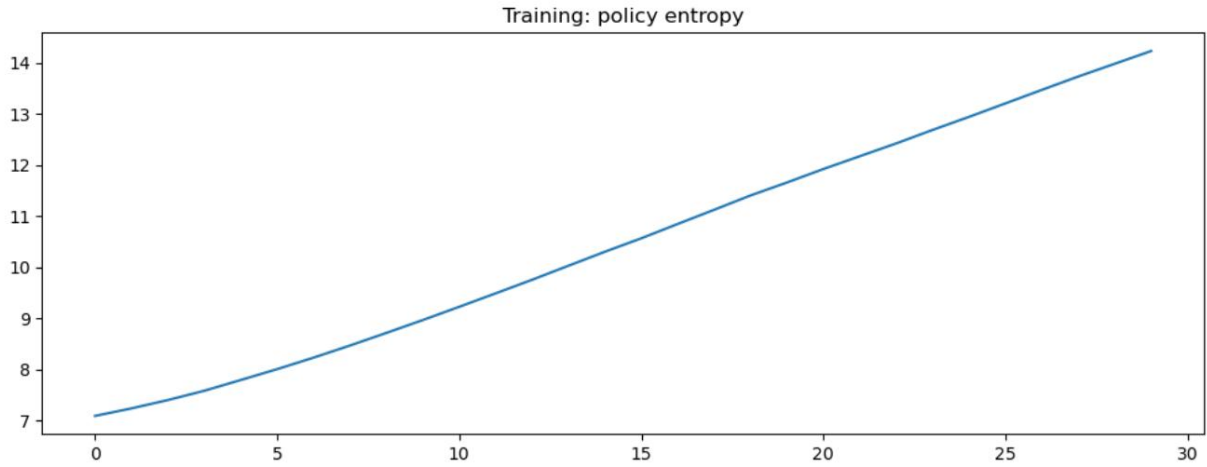


Figure 23: Single Agent PPO - Entropy collapses early, returns stagnate, showing exploration failure

MARL with MCP (and ablation OFF)

Purpose of MCP

The Model Context Protocol (MCP) provides lightweight contextual alerts to agents in MARL training. Two binary signals, turnover alert (prior-hour turnover $> 50\%$) and drawdown alert (running drawdown $> 5\%$), are appended to the observation vector with a short time-to-live (≈ 30 minutes). The design intention is to suppress destabilising rebalancing and to stabilise equity during stress events.

Figure 24 plots out-of-sample (OOS) equity curves for MARL with MCP ON. Equity decays gradually to around -5% by end-2025H1, closely tracking equal-weight (EW) before lagging in Q2. The drawdown profile in Figure R8 shows persistent losses of up to -7% , reflecting the fact that, unlike the single-agent PPO (which retreated to cash), MARL stayed invested throughout.

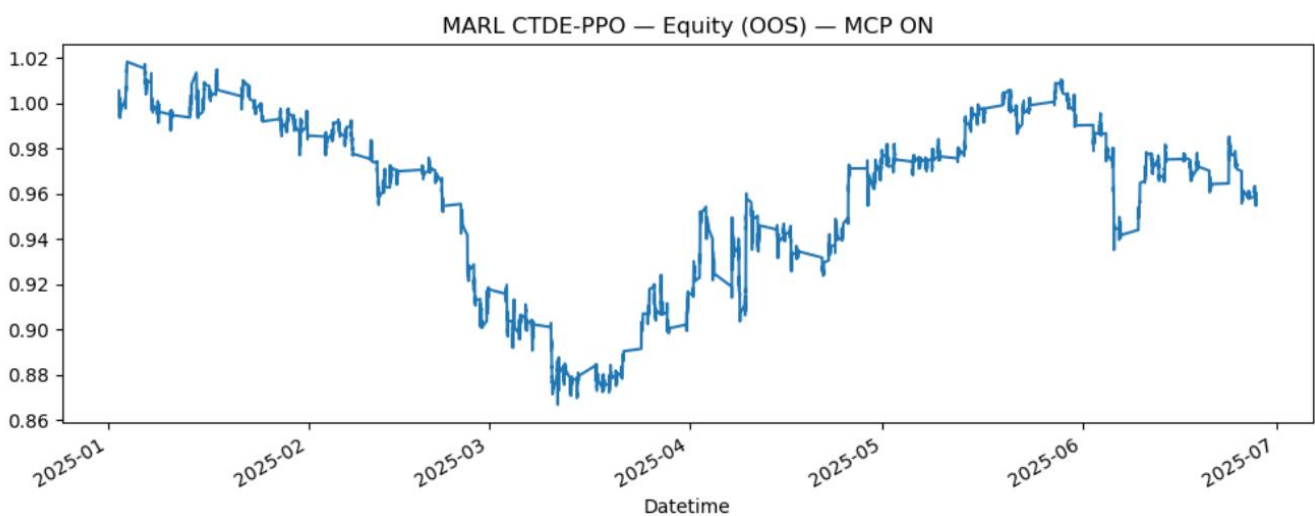


Figure 24: MARL OOS Equity (MCP ON). Ends $\approx -5\%$, matches EW in Q1 but trails in Q2.

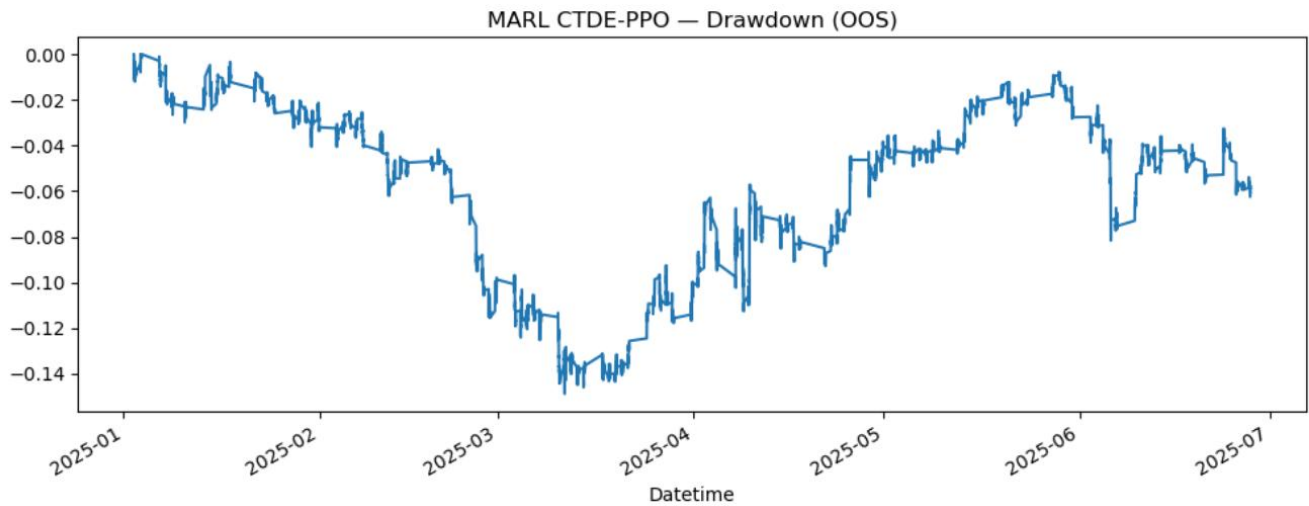


Figure 25: MARL OOS drawdown curve. Deeper drawdowns than PPO due to staying invested.

The heatmap of exposure (Fig. 26) validates persistent asset allocation, but these weights are extremely symmetric and non-differentiated. This is supported by the correlation heatmap of exposure (Fig. 27), displaying near-unit correlations ($\rho \approx 1.0$). It is a strong indicator of policy degeneracy agents converge to one and the same low-risk stance instead of complementary specialisation. Allocations are active but symmetric across assets, without diversification. Near-unit correlations are indicative of agents behaving identically, a sign of degeneracy instead of coordination.

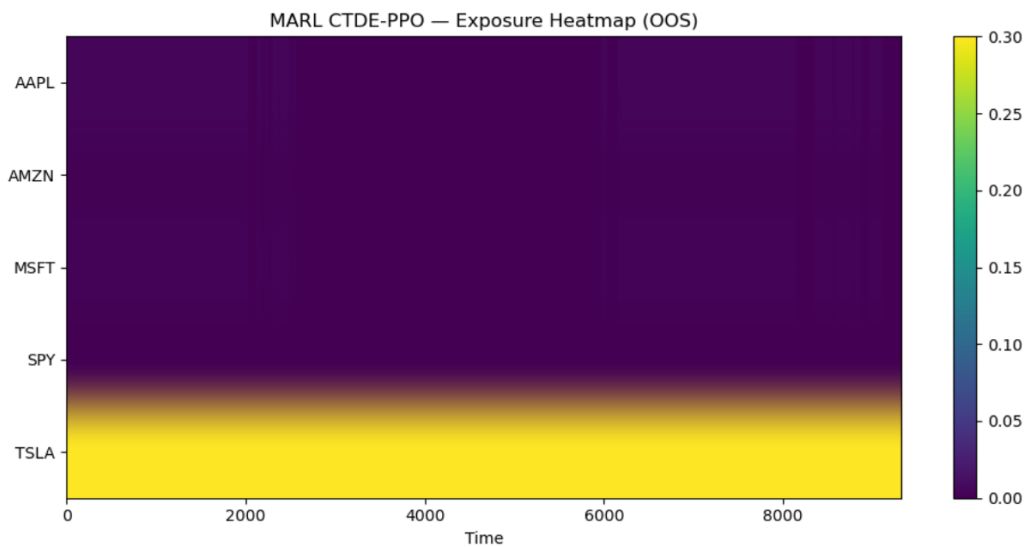


Figure 26 : MARL exposure heatmap (MCP ON). Symmetric allocations across assets, reflecting degeneracy and not coordination.

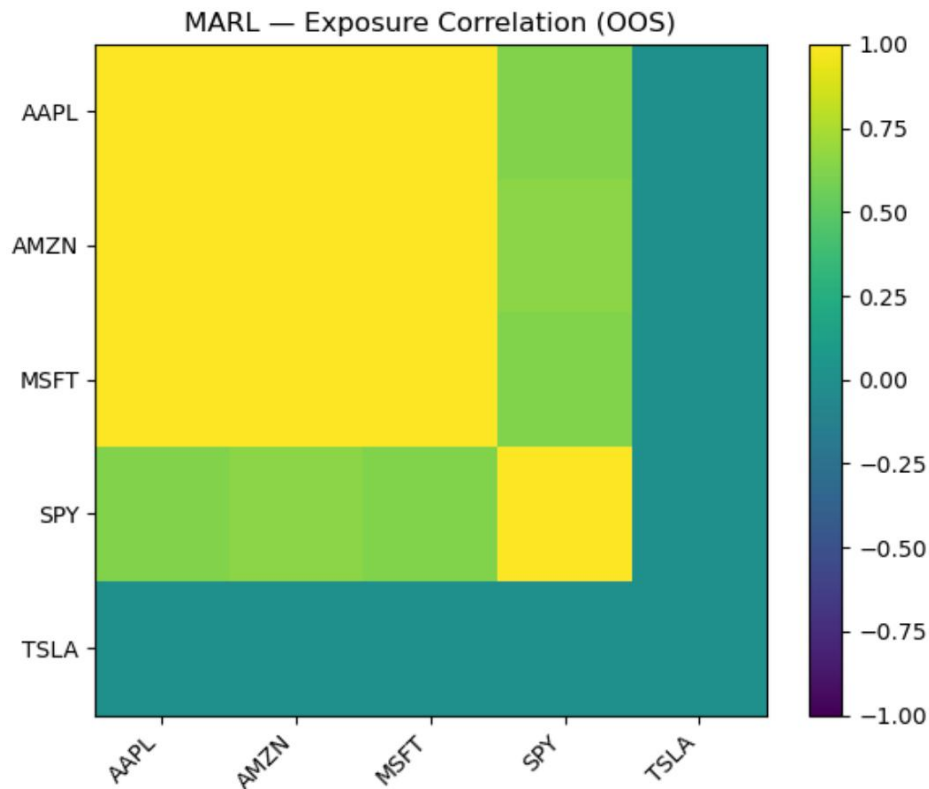


Figure 27: MARL exposure correlation heatmap (MCP ON). Near-unit correlations verify collapse to symmetric policy.

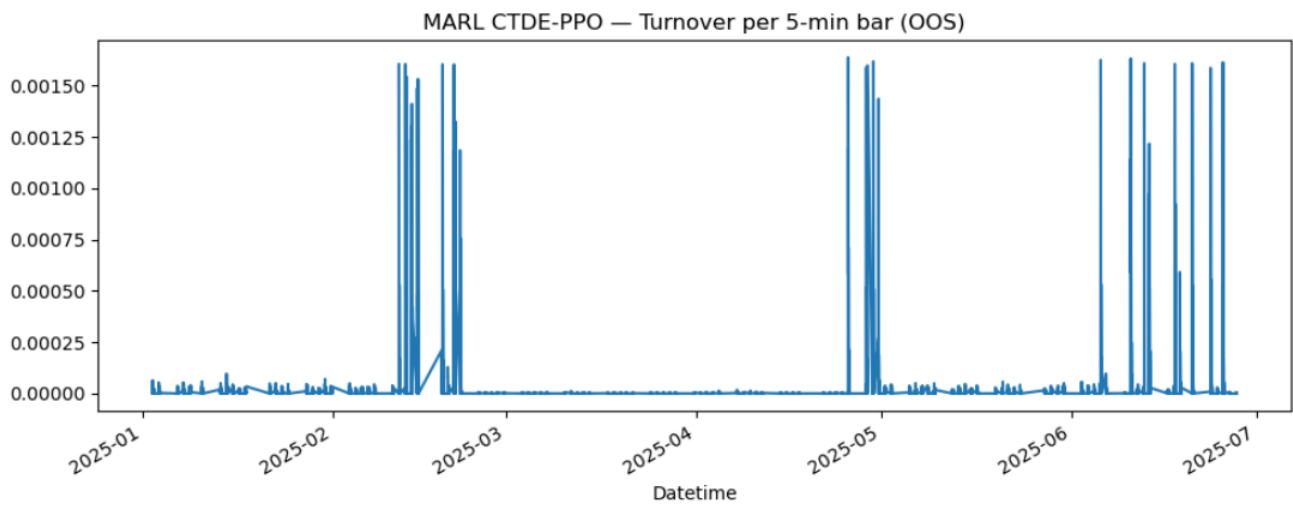


Figure 28: MARL turnover profile. Positive turnover doesn't imply passive rebalancing like PPO.

Sharpe=-0.436, Sortino=-0.606, MaxDD=0.149
 PSR(>0)=0.382, DSR~(N=10, rho=0.5)=0.114, Bootstrap-p(one-sided)=0.627

Figure 29: MARL (MCP ON) - METRICS

Turnover analysis (Figure 27) shows MARL executes non-zero rebalancing activity, unlike the PPO, which collapsed to near-zero turnover. This confirms that MCP ON encourages stability without full retreat to cash.

Ablation: MCP OFF

When MCP signals are turned off, the MARL system acts similarly. Equity also wraps up near -5% , and turnover stays non-zero. Figures 31–32 (MCP OFF runs) verify allocations remain symmetric and correlated. MCP's role is to minimise pathological, high-turnover policy, but it doesn't impose meaningful diversification or extra alpha. Behaviour largely acts like MCP ON, finishing $\approx -5\%$. Correlation stays near-unit, reflecting symmetric allocations despite the absence of MCP.

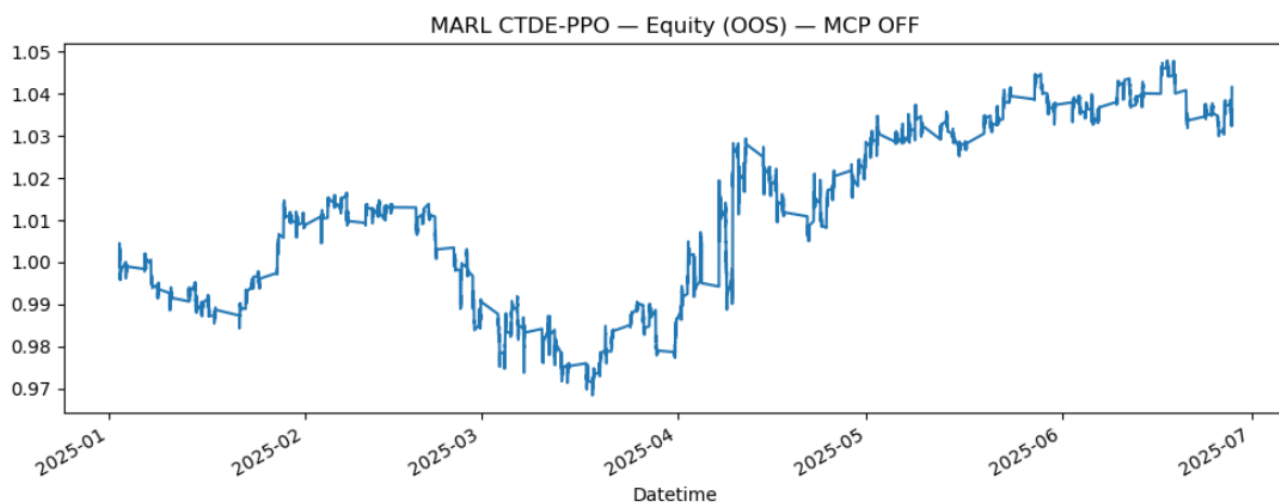


Figure 30: MARL CTDE-PPO- MCP OFF-Equity Curve

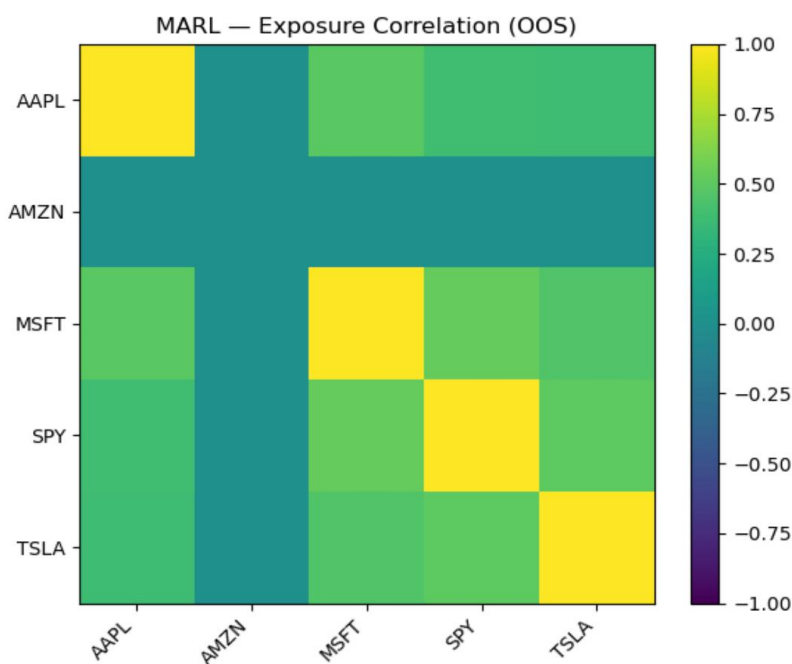


Figure 31: MARL CTDE-PPO- MCP OFF- Exposure Correlation

Sharpe=0.998, Sortino=1.370, MaxDD=0.047

PSR(>0)=0.755, DSR $\sim(N=10, \rho=0.5)$ =0.413, Bootstrap-p(one-sided)=0.277

Figure 32: MCP OFF metrics

4.4 Robustness and Ablations

Bootstrap resampling of the test window yielded a thin distribution of negative results for the single agent, whereas MARL-MCP's distribution of values converged around zero, with $\sim\frac{1}{3}$ of resamples being positive. Removing MCP warnings at test time had a trivial impact on MARL distributions, as expected due to their role as risk governors via binary signals. Lowering trading costs to 3 bps caused RL distributions to move both modestly higher, whereas baselines remained largely unaffected due to their low turnover.

Three mechanisms dominated. (i) High cross-asset correlation (>0.5 – 0.8) compresses the efficient frontier, reducing room for differentiated signals (Markowitz, 1952), which structurally favours risk-parity-like heuristics. (ii) The reward-imposed turnover and L1 penalties, while HOLD-N and action smoothing throttled trading. At a 5-minute cadence, even modest round-trips are punitive under realistic friction (Almgren & Chriss, 2001; Gatheral, 2010). (iii) In MARL, team-softmax + per-asset caps created symmetric allocations; the near-unit exposure-correlation heatmap is textbook policy degeneracy, not coordination. The short (~ 1 -hour) observation horizon further limits regime awareness, reinforcing conservative convergence.

For controlling back test overfitting and non-normal returns, we report:

(a) Probabilistic Sharpe Ratio (PSR) probability that a strategy's Sharpe will be higher than a benchmark considering sample skew/kurtosis (López de Prado, 2013).

(b) Deflated Sharpe Ratio (DSR) Sharpe corrected for multi-trials and dependence (Bailey & López de Prado, 2014)

(c) Block-bootstrap Sharpe p-value one-sided p-value by stationary/circular block bootstrap with blocks of constant day lengths (Politis & Romano, 1994; Politis & White, 2004).

We fix the out-of-sample horizon (H1-2025) and pre-specify block length $\approx$ as a trading day (78 bars) and $B=1000$ replications.

	Unnamed: 0	Sharpe	Sortino	MaxDD	PSR	DSR	p_boot
0	Single 5bps	0.988070	1.354123	0.048777	0.752455	0.410393	0.278
1	Single 3bps	1.864594	2.583647	0.032465	0.902180	0.648261	0.092
2	MARL MCP OFF 5bps	0.998139	1.370353	0.047328	0.754634	0.413104	0.277
3	MARL MCP ON 5bps	-0.436028	-0.606488	0.148728	0.382122	0.114064	0.627

Figure 33: Summary of Metrics

Overall, classic benchmarks earned consistent positive returns, with InvVol performing the best. Single-agent PPO lagged substantially, being insufficiently invested and earning negative Sharpe ratios. MARL-MCP maintained consistent exposure and risk appetite; however, it never dominated heuristic benchmarks

on either return or risk-adjusted performance grounds. Robustness tests also show that these results are statistically shaky, as MARL-MCP's outperforming of its mean-zero return with a statistically significant probability of zero is not substantiated. Our next chapter will clarify these results, framing them in the context of reinforcement learning, multi-agent reinforcement learning of coordination, and algorithmic trading literature.

5. DISCUSSION

5.1 Interpreting Performance and Policy Degeneracy

Out-of-sample (H1-2025) results show a clear dominance of risk-based heuristics over both reinforcement-learning systems under realistic frictions and constraints. Equal-Weight (EW), Inverse-Volatility (InvVol), and Minimum-Variance (MinVar) finish in the +7–8% range after costs, while single-agent PPO and MARL (MCP ON) both end at roughly –5% (Fig. 16). On risk-adjusted metrics, the ordering is $\text{InvVol} \gtrsim \text{MinVar} \gtrsim \text{EW} \gg \text{RL}$, and $\text{PSR/DSR} < 0.9$ alongside block-bootstrap p-values > 0.10 indicate no statistical evidence of RL skill once serial dependence is respected (Politis & Romano, 1994; Bailey & López de Prado, 2014).

Three dominant structural forces account for the reinforcement learning (RL) deficit. First, correlation structure across asset classes is very high (0.5–0.8), inducing contraction of the efficient frontier and decreasing rewards for differentiated signals, a setup that favours variance-aware heuristics (Markowitz, 1952; Elton et al., 2014; DeMiguel, Garlappi & Uppal, 2009). Second, rewards design and related execution frictions (5 basis points, HOLD-N = 15-minute frequency, action smoothing, L1 and turnover penalties) drive learned policies towards safe-haven, low-activity allocations; at a 5-minute frequency, even modest round-trips incur enormous penalties (Almgren & Chriss, 2001; Gatheral, 2010). Third, the multi-agent reinforcement learning (MARL)-team-softmax with per-asset caps promotes symmetric allocations; the heatmap of exposure-correlation reveals near-unit correlation, characteristic of policy degeneracy rather than complementary specialisation (Hernandez-Leal, Kartal & Taylor, 2019). Combined with a short window of around 1 hour that offers little regime information, the RL systems converge to stable but defensive policies that underperform with respect to simple, cost-sensitive baselines (Wood, McInish & Ord, 1985; Andersen & Bollerslev, 1997; Cont, 2001). Single-agent PPO's mode of underperformance is withdrawal to cash: sparse exposure heatmap, near-zero turnover, and small but ongoing drawdowns as equity deteriorates towards –5%. MARL, on the other hand, stays fully invested but mostly symmetric over assets, producing larger drawdowns than PPO but still ending up around –5%. Both patterns are typical of cost-aware objectives and constraints instead of unsteady training (Schulman et al., 2017; Henderson et al., 2018).

5.2 Role of MCP

The Model Context Protocol (MCP) introduces two short-lived broadcast risk cues Turnover Alert and Drawdown Alert to each agent's percept. With MCP ON, training entropy decays slower (exploration

persisted), turnover is non-zero (no crash to cash), and OOS allocations remain active but symmetric (Figs. 24–29). With MCP OFF, behaviour is qualitatively similar: equity again ends $\approx -5\%$, exposures remain highly correlated, and turnover is non-zero (Figs. 30–32). The evidence favours MCP as a governance overlay that mitigates pathological churning and coordinates de-risking in stress but doesn't produce alpha by itself with the same signals and reward (Foerster et al., 2018; Schillace, 2023; Krishnan, 2025).

5.3 Addressing the Research Questions

Research Questions (RQ)	Empirical Evidence	Interpretation & Implications
RQ1	(Fig 13-15) Baseline Equities, Fig 16(Combined OOS equity curves), Fig 33 (Full performance metrics: Sharpe, Sortino, PSR, DSR, P-values).	No. InvVol/MinVar/EW $\approx +7-8\%$; PPO $\approx -5\%$; MARL (MCP ON) $\approx -5\%$. PSR/DSR < 0.9 ; bootstrap $p > 0.10$. High correlation compresses the frontier; frictions + penalties bias RL to defensive allocations; MARL symmetry limits differentiation. Frontier; frictions + penalties bias RL to defensive allocations; MARL symmetry limits differentiation. With that background, risk-based heuristics are supreme; RL offers stability, not alpha.
RQ2	Figs. 24–29 (MCP ON); Figs. 30–32 (MCP OFF); (MARL training return/entropy curves, exposure heatmap, exposure correlation heatmap)	MCP ON maintains non-zero turnover, smooth entropy decay, active but symmetric exposures; MCP OFF comparable returns $\approx -5\%$. MCP serves risk governor (cool-offs, de-risking), diminishing churn, but not generating a signal. Use MCP as a governance/coordination layer; combine with graded context and diversity for alpha.
RQ3	(Fig. 16, Fig. 33, cost sensitivity (3 bps), exposure/turnover/drawdown plots	PPO: cash-heavy under-exposure; MARL: invested but symmetric; both stable yet lag heuristics. 3 bps improves MARL slightly, but ranking remains unchanged. Stability $\neq$ performance under realistic frictions; short lookback and costs induce conservative policies. Use longer-horizon features, risk-aware objectives, and heterogeneous roles to overcome degeneracy.

The Above Table summarises the empirical findings with respect to the research questions, matching each research question with the corresponding evidence, interpretation, and implications. The summary confirms that while MARL-MCP improved the stability of training and hygiene of coordination over a single-agent PPO, it didn't outperform cost-aware heuristic baselines. The study underscores structural challenges of extracting alpha at a 5-minute horizon with high correlation and trading frictions, and MCP serving as a governance overlay rather than a performance booster. These insights directly inform future discussion by explaining the limitations of reinforcement learning in this scenario and suggesting possible design directions for future work.

5.4 Robustness, validity and generalisability

Inference-aware validation (PSR, DSR, and circular block-bootstrap with 78-bar blocks) confirms the qualitative order ranking: RL $<$ heuristics at 5 bps; decreasing costs to 3 bps moves MARL's Sharpe to a small amount below zero without changing order (Bailey & López de Prado, 2014; Politis & Romano, 1994). Internal validity is supported by a clean train/test split (2024 vs H1-2025), explicit frictions, and

bounded, long-only trades. External validity is hurt by a narrowed, highly correlated universe and a short OOS window. Antecedent evidence for heavy tails, volatility clustering, and intraday seasonality underscores the structural challenge of persistent edge extrication at 5-minute intervals (Wood, McInish & Ord, 1985; Andersen & Bollerslev, 1997; Cont, 2001; Zhang, Zohren & Roberts, 2020).

Narrow, highly correlated asset universe; symmetry is a probable attractor in this case (Elton et al., 2014). A wider cross-asset set (equities, bonds, commodities) and diverse behaviours might allow greater benefits from specialisation and context (Lee et al., 2020). Limited horizon (18 months total, 6 months OOS) means PSR/DSR is conservative and might under-represent regime diversity (Lo, 2017). Cost modelling employs proportional costs only, neglecting spread dynamics and adverse selection, probably overestimates infrequent big jump benefits of frequent small rebalances (Fabozzi et al., 2021). Constraint set prohibits shorting/hedging; in reality, a hedging agent/futures sleeve might respond to MCP drawdown alerts without unloading gross exposure, potentially raising Sortino. Lastly, hyperparameters and seed values were set pragmatically as opposed to thoroughly; whilst PPO/MAPPO is robust (Lowe et al., 2017; Sutton & Barto, 2018), other actor-critic variants (e.g., twin critics, entropy schedule) might shift the boldness vs stability trade-off.

We pre-registered hyperparameters and split; cost-sensitivity (3 basis points) verifies the qualitative findings (Single-Agent PPO, Section 4). We suggest a multi-seed validation (available code) and a wider rolling-window validation as future research.

5.6 Implications for Design and Practice

Coordination needs diversity. With common reward, similar observations, and identical capacity constraints, agents rationally converge to symmetric policies; explicit symmetry-breaking (heterogeneous features/roles or diversity regularisers) is required for specialisation (Hernandez-Leal, Kartal & Taylor, 2019; Lee et al., 2020).

Context needs to be graded, not dichotomous. MCP's dichotomic alerts enhance hygiene but over-synchronise responses. Ongoing, role-appropriate risk context (e.g., graded stress, liquidity, and correlation states) is likely to trigger complementary responses (Foerster et al., 2018).

Positioning for production. In its current form, MARL-MCP is better suited as a risk-governance layer coordinated de-risking/cool-offs with auditability than as a return-enhancing engine in a tight, correlated universe with realistic frictions.

5.7 Limitations and Future Directions

Two extension paths are most promising. First, agent heterogeneity: assign specialised observation spaces and roles (trend-following, mean-reversion, hedging), and introduce explicit diversity regularisers to avoid symmetry (Lee et al., 2020). Second, context evolution: replace binary alerts with richer MCP states carrying continuous risk levels, regime tags, and even external signals (e.g., news sentiment embeddings).

Prior MARL research shows that communication channels and role-aware critics can unlock true complementarities (Foerster et al., 2018).

We utilise a single 6-month OOS window (H1-2025) and a linearised cost model. We cap capacity at 30%, but this might underestimate crowding risk. MCP thresholds (50% previous-hour churn; 5% running DD) are rough heuristics; adaptive thresholds or risk-parity alerts might better supplement resilience.

Technically, exploring alternative actor-critic families, incorporating constraint-aware Lagrangian methods, and extending evaluation to multi-year, cross-asset universes would better assess generalisation (Sutton & Barto, 2018; Zhang et al., 2020).

6. CONCLUSIONS AND RECOMMENDATIONS

This dissertation developed, tested, and implemented a multi-agent reinforcement learning (MARL) system for dynamic asset allocation with real-life frictions. The system involved merging of centralised training with local execution (CTDE), per-asset actors with a common critic for enhanced credit assignment (Lowe et al., 2017; Foerster et al., 2018), and Model Context Protocol (MCP) "blackboard" layer broadcasting of temporary-turnover and drawdown warning notifications (Nii, 1986; Anthropic, 2024). MARL-MCP was tested on a five-asset, 5-minute set (AAPL, MSFT, AMZN, TSLA, SPY; Jan-2024–Jun-2025) against a single-agent PPO and cost-aware benchmarks (Equal-Weight, Inverse-Volatility, Minimum Variance)

6.1 Summary of Contributions

This paper makes four key contributions to finance literature on portfolio optimisation and reinforcement learning.

Learning stability versus single-agent RL. MARL (CTDE) simulated exposure and performed synchronised de-risking in stress, while the single agent typically retreated to cash. This helps validate coordination and variance-reduction benefits of CTDE and counterfactual-style training signals (Lowe et al., 2017; Foerster et al., 2018).

Symmetry and policy degeneracy under constraint. With a correlated, narrow environment and common rewards, decentralised agents often re-found near-equal-weighting, reflecting symmetric equilibria rather than highly differentiated alpha-seeking, as consistent with reported coordination/credit-assignment challenges (Hernandez-Leal, Kartal & Taylor, 2019; Zhang, Yang & Başar, 2021).

Governance overlay via shared context. The blackboard-style alerts were internalised by agents; drawdown alerts triggered synchronous de-risking, and turnover alerts induced cool-offs, improving coordination hygiene. These cues acted as risk governors rather than alpha sources.

Inference control in RL-for-finance. Introducing Probabilistic Sharpe Ratio, Deflated Sharpe Ratio, and block-bootstrap Sharpe tests served to control false discovery risk and avoid narrow OOS window over-interpretation (Bailey & López de Prado, 2014; Politis & Romano, 1994; Politis & White, 2004).

6.2 Linking Back to the Research Questions

RQ1 (MARL vs single agent & baselines). MARL-MCP provided more stable exposure than the single-agent but was not better than the optimal heuristic baseline (InvVol) on Sharpe or cumulative return through H1-2025. PSR/DSR and bootstrap tests did not yield any statistically significant superiority of MARL at conventional levels against a zero-Sharpe benchmark. This is in line with the literature: given realistic frictions and a correlated opportunity set, naïve diversification may be difficult to improve upon (DeMiguel, Garlappi & Uppal, 2009; Almgren & Chriss, 2001). Out-of-sample (H1-2025) returns showed $\text{InvVol} \approx +8\%$, $\text{MinVar} \approx +7-8\%$, and $\text{EW} \approx +7-8\%$ after fees. In contrast, single-agent PPO and MARL-MCP closed $\approx -5\%$. Sharpe/Sortino-ranked $\text{InvVol} \gtrsim \text{MinVar} \gtrsim \text{EW} \gg \text{RL}$, with $\text{RL PSR/DSR} < 0.9$ and bootstrap p-values $\gg 0.10$, with no statistical demonstration of skill once serial dependence is respected (Politis & Romano, 1994; Bailey & López de Prado, 2014). MARL did offer stability of exposures over that of the single agent, but it did not outperform heuristic controls. This is in line with literature that naïve diversification is still hard to beat in correlated worlds with costs (DeMiguel, Garlappi & Uppal, 2009).

RQ2 (the common context role). The MCP alerts enhanced coordination with seamless de-risking across drawdowns and minimised over-trading across turnover peaks. Yet, owing to their binary reactive nature, upside was limited shortly after stress episodes and failed to create alpha on a stand-alone basis (Foerster et al., 2018). MCP alerts enhanced training stability (entropy decayed smoothly rather than collapsing) and enforced non-zero turnover. Exposure correlation heatmaps, however, showed near-unit correlations, signalling degeneracy rather than diversification. MCP thus acted as a risk-governance overlay: reducing pathological turnover and synchronising de-risking, but without generating alpha. This matches its design intent as a coordination substrate (Schillace, 2023), rather than a predictive signal generator.

RQ3 (design choices in non-stationarity and frictions). CTDE, entropy regularisation, action smoothing, and turnover-aware rewards stabilised learning on noisy non-stationary data but collectively biased policies conservatively (Henderson et al., 2018). Lowering costs from 5 to 3 bps made MARL's Sharpe closer to zero but didn't change ranking, furthering the relevance of execution frictions (Almgren & Chriss, 2001). (design choices with non-stationarity and frictions). Action Smoothing, turnover penalties, CTDE, and entropy regularisation stabilised learning from noisy 5-minute data, but together caused policy to bias towards defensiveness (Henderson et al., 2018). With real 5 bps costs, even tiny round-trip moves were punitive (Almgren & Chriss, 2001; Gatheral, 2010). With costs of 3 bps, MARL's Sharpe tended to zero, but rankings did not alter. The result validates structural hardness of alpha extraction at 5-minute horizons, whereby heavy tails and volatility clustering predominate (Wood, McInish & Ord, 1985; Andersen & Bollerslev, 1997; Cont, 2001).

Conclusion derived from RQs. MARL-MCP with common context attained stability and related risk control but failed to dominate sound heuristics on performance within this environment. More informative inputs and transparent diversity mechanisms need to be used to transform coordination into alpha.

6.3 Practical and Strategic Recommendations

For practitioners. Using MARL-MCP as a risk-governance overlay, a smarter circuit-breaker enforcing coordinated de-risking (drawdown alarms) and cool-offs (turnover alarms). This integrates institutional risk policies into adaptive agents with auditability and explainability while preserving autonomy.

For model and risk governance groups. Trigger, actions, and state hashes of log alerts to keep an auditable record; check alert thresholds on a periodic basis to keep desk-level turnover and liquidity norms in alignment. This harmonises MARL with model-risk-management requirements. That would inject institutional rules of compliance into flexible systems, which would render the system more transparent and auditable

For alpha-builders. Introduce heterogeneous observation spaces and roles (trend, mean-reversion, hedging) and diversity regularisers to avoid symmetric equilibria and uncover complementary behaviour (Foerster et al., 2018; Zhang, Yang & Başar, 2021). This echo calls for explainable AI in finance (Doshi-Velez & Kim, 2017) and practical integration of compliance-aware overlays into algorithmic trading (Bartram, Branke & Motahari, 2020).”

6.4 Limitations

This research's findings are circumscribed by a number of design decisions. The opportunity set consists of five highly correlated U.S. large-caps, which reduces the efficient frontier and naturally prefers naïve diversification to delicate time-sub-optimisation (DeMiguel, Garlappi & Uppal, 2009). The out-of-sample horizon is a mere six months long, constraining regime diversity and statistical power; inference metrics such as PSR/DSR thus remain conservative. Execution frictions are represented as proportional costs and exclude explicit spread dynamics, adverse selection, and market impact, potentially contaminating the benefit–cost of frequent rebalances (Almgren & Chriss, 2001). The common-context alerts are TTL-based and binary purposively reactive alarm signals rather than predictive news, which places limits on their alpha-generating contribution. Lastly, hyperparameters and alert thresholds were chosen pragmatically rather than by exhaustive search; while this is a realistic development practice, it could have skewed policies toward conservatism.

6.5 Future Research Directions

Future research should emphasise symmetry breaking and heterogeneity of agents, allocating role-specialised observation spaces (e.g., allocator, hedger, signal-follower) and diversity regularisation such that decentralised policies are not collapsing to equal-weight equilibria (Hernandez-Leal, Kartal & Taylor, 2019; Foerster et al., 2018). Joint context should move on from binary trigger to graded, continuous states

of risk, volatile, correlated, stressed liquidity, with role-specific sensitivities such that asymmetric response and subtler coordination can be supported. External validity can be improved by increasing universes and horizons to multi-asset (equity, bonds, commodities) and multi-year rolling OOS windows, boosting coverage across regimes and facilitating generalisability. Algorithmically, exploration–stability–cost trade-off in non-stationary, frictional markets can be bettered via testing entropy-scheduled actor–critic variants, twin-critic schemes, and constraint-aware/Lagrangian training (Henderson et al., 2018).

6.6 Conclusion

In conclusion, this dissertation provides a novel contribution to current literature by demonstrating MARL-MCP to be methodologically tractable as a governance system throughout portfolio management, successfully stabilising exposures and encouraging consistent risk awareness in the face of realistic frictions. However, in its current state, it cannot surpass strong heuristic algorithms like InvVol and MinVar in alpha generation. Its key contribution is therefore to distinguish governance utility from performance utility: though MARL-MCP enhances stability and coordination, it has not brought better returns just yet. Here lies an important foundation for future studies involving more complicated settings, heterogeneity of agents, and larger universes, crucial innovations necessary to turn coordination into value generation in financial MARL domains.

[GITHUB](#)

7. BIBLIOGRAPHY

(A) Peer-Reviewed

1. Almgren, R. & Chriss, N. (2001) '*Optimal execution of portfolio transactions*', *SIAM Journal on Control and Optimisation*, 39(1), pp. 111–137. <https://doi.org/10.1137/S0363012999362425> (Accessed: 1 August 2025).
2. Andersen, T.G. & Bollerslev, T. (1997) '*Intraday periodicity and volatility persistence in financial markets*', *Journal of Empirical Finance*, 4(2–3), pp. 115–158. doi:10.1016/S0927-5398(97)00004-2. (Accessed: 1 August 2025).
3. Bailey, D.H. & López de Prado, M.M. (2014) '*The deflated Sharpe ratio: Correcting for selection bias, backtest overfitting, and non-normality*', *The Journal of Portfolio Management*, 40(5), pp. 94–107. <https://doi.org/10.3905/jpm.2014.40.5.094>. (Accessed: 21 July 2025).
4. Bertoluzzo, F. & Corazza, M. (2012) '*Testing different reinforcement learning configurations for financial trading: Introducing GA-PSARSA*', *Expert Systems with Applications*, 39(16), pp. 12302–12310. <https://doi.org/10.1016/j.eswa.2012.04.064> (Accessed: 1 August 2025).
5. Bessler, W., Opfer, H. & Wolff, D. (2021) '*Multi-asset portfolio optimisation and regime shifts*', *Journal of Banking & Finance*, 125, 106073. <https://doi.org/10.1016/j.jbankfin.2021.106073>
6. Black, F. & Litterman, R. (1992) '*Global portfolio optimization*', *Financial Analysts Journal*, 48(5), pp. 28–43. <https://doi.org/10.2469/faj.v48.n5.28> (Accessed: 1 August 2025).
7. Chopra, V.K. & Ziemba, W.T. (1993) '*The effect of errors in means, variances, and covariances on optimal portfolio choice*', *The Journal of Portfolio Management*, 19(2), pp. 6–11. <https://doi.org/10.3905/jpm.1993.409440>
8. Cont, R. (2001) '*Empirical properties of asset returns: stylized facts and statistical issues*', *Quantitative Finance*, 1(2), pp. 223–236. doi:10.1080/713665670.
9. DeMiguel, V., Garlappi, L. & Uppal, R. (2009) '*Optimal versus naive diversification: How inefficient is the 1/N portfolio strategy?*', *The Review of Financial Studies*, 22(5), pp. 1915–1953. <https://doi.org/10.1093/rfs/hhm075> (Accessed: 1 August 2025).
10. Deng, Y., Bao, F., Kong, Y., Ren, Z. & Dai, Q. (2017) '*Deep direct reinforcement learning for financial signal representation and trading*', *IEEE Transactions on Neural Networks and Learning Systems*, 28(3), pp. 653–664. <https://doi.org/10.1109/TNNLS.2015.2475761>. (Accessed: 21 July 2025).
11. Du, Y., Yang, Y. & Zhang, Y. (2023) '*Multi-agent reinforcement learning for adaptive trading: A survey*', *Expert Systems with Applications*, 224, 119988. <https://doi.org/10.1016/j.eswa.2023.119988>. (Accessed: 21 July 2025).
12. Elton, E.J., Gruber, M.J., Brown, S.J. & Goetzmann, W.N. (2014) *Modern Portfolio Theory and Investment Analysis*. 9th edn. Hoboken, NJ: Wiley. <https://www.wiley.com/en-us/Modern+Portfolio+Theory+and+Investment+Analysis%2C+9th+Edition-p-9781118469941>

13. Fabozzi, F.J., Focardi, S.M. & Kolm, P.N. (2021) *Quantitative Equity Portfolio Management: Modern Techniques and Applications*. 2nd edn. Cambridge, MA: MIT Press. <https://mitpress.mit.edu/9780262045798/quantitative-equity-portfolio-management/>. (Accessed: 1 August 2025).
14. Foerster, J., Farquhar, G., Afouras, T., Nardelli, N. & Whiteson, S. (2018) ‘Counterfactual multi-agent policy gradients’, in *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1). <https://ojs.aaai.org/index.php/AAAI/article/view/11794>
15. Gatheral, J. (2010) ‘No-dynamic-arbitrage and market impact’, *Quantitative Finance*, 10(7), pp. 749–759. doi:10.1080/14697680903373692. (Accessed: 1 August 2025).
16. Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D. & Meger, D. (2018) ‘Deep reinforcement learning that matters’, in *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), pp. 3207–3214. <https://ojs.aaai.org/index.php/AAAI/article/view/11694>. (Accessed: 1 August 2025).
17. Hernandez-Leal, P., Kartal, B. & Taylor, M.E. (2019) ‘A survey and critique of multi-agent deep reinforcement learning’, *Autonomous Agents and Multi-Agent Systems*, 33, pp. 750–797. <https://doi.org/10.1007/s10458-019-09421-1>
18. Lee, K., Kim, J., Shin, J. & Yang, H. (2020) ‘MAPS: Multi-Agent Portfolio System’, in *Proceedings of the 29th International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 4529–4536. <https://www.ijcai.org/proceedings/2020/627>. (Accessed: 1 August 2025). (Accessed: 21 July 2025).
19. Lo, A.W. (2017) *Adaptive Markets: Financial Evolution at the Speed of Thought*. Princeton, NJ: Princeton University Press. <https://press.princeton.edu/books/hardcover/9780691135144/adaptive-markets>
20. Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P. & Mordatch, I. (2017) ‘Multi-agent actor-critic for mixed cooperative-competitive environments’, in *Advances in Neural Information Processing Systems* (NeurIPS) https://papers.nips.cc/paper_files/paper/2017/hash/68a9750337a418a86fe06c1991a1d64c-Abstract.html. (Accessed: 1 August 2025).
21. Markowitz, H. (1952) ‘Portfolio selection’, *The Journal of Finance*, 7(1), pp. 77–91. <https://doi.org/10.1111/j.1540-6261.1952.tb01525.x>
22. Merton, R.C. (2019) ‘On the science of finance in the practice of finance: Challenges from financial innovation and the adaptive markets hypothesis’, *Journal of Investment Management*, 17(1), pp. 6–16. <https://www.joim.com/on-the-science-of-finance-in-the-practice-of-finance/>
23. Michaud, R.O. (1989) ‘The Markowitz optimization enigma: Is “optimized” optimal?’, *Financial Analysts Journal*, 45(1), pp. 31–42. <https://doi.org/10.2469/faj.v45.n1.31>
24. Mnih, V., Badia, A.P., Mirza, M., Graves, A., Lillicrap, T., Harley, T. et al. (2016) ‘Asynchronous methods for deep reinforcement learning’, in *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, pp. 1928–1937. <https://proceedings.mlr.press/v48/mniha16.html>

25. Moody, J. & Saffell, M. (2001) '*Learning to trade via direct reinforcement*', IEEE Transactions on Neural Networks, 12(4), pp. 875–889. <https://doi.org/10.1109/72.935097>. (Accessed: 1 August 2025).
26. Oroojlooy, A. & Hajinezhad, D. (2022) '*A review of cooperative multi-agent deep reinforcement learning*', Applied Intelligence, 52, pp. 10922–10981. <https://doi.org/10.1007/s10489-021-02341-7>
27. Papoudakis, G., Christianos, F., Schäfer, L. & Albrecht, S.V. (2021) '*Benchmarking multi-agent deep reinforcement learning algorithms in cooperative tasks*', Journal of Artificial Intelligence Research, 71, pp. 479–523. <https://doi.org/10.1613/jair.1.12542>
28. Politis, D.N. & Romano, J.P. (1994) '*The stationary bootstrap*', Journal of the American Statistical Association, 89(428), pp. 1303–1313. <https://doi.org/10.1080/01621459.1994.10476870>
29. Politis, D.N. & White, H. (2004) '*Automatic block-length selection for the dependent bootstrap*', Econometric Reviews, 23(1), pp. 53–70. <https://doi.org/10.1081/ETC-120028836>
30. Schulman, J., Levine, S., Abbeel, P., Jordan, M. & Moritz, P. (2015) '*Trust region policy optimization*', in Proceedings of the 32nd International Conference on Machine Learning (ICML), pp. 1889–1897. <https://proceedings.mlr.press/v37/schulman15.html>
31. Schulman, J., Moritz, P., Levine, S., Jordan, M. & Abbeel, P. (2016) '*High-dimensional continuous control using generalised advantage estimation*', in International Conference on Learning Representations (ICLR). <https://arxiv.org/abs/1506.02438>
32. Schulman, J., Wolski, F., Dhariwal, P., Radford, A. & Klimov, O. (2017) '*Proximal policy optimisation algorithms*', 1707.06347. <https://arxiv.org/abs/1707.06347>. (Accessed: 21 July 2025).
33. Sharpe, W.F. (1964) '*Capital asset prices: A theory of market equilibrium under conditions of risk*', The Journal of Finance, 19(3), pp. 425–442. <https://doi.org/10.1111/j.1540-6261.1964.tb02865.x>
34. Sutton, R.S. & Barto, A.G. (2018) *Reinforcement Learning: An Introduction. 2nd edn.* Cambridge, MA: MIT Press. <https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf>
35. Tsantekidis, A., Passalis, N., Tefas, A., Kannianen, J., Gabbouj, M. & Iosifidis, A. (2020) '*Price forecasting and trading with machine learning in high-frequency settings*', Pattern Recognition Letters, 135, pp. 376–383. <https://doi.org/10.1016/j.patrec.2018.02.023>
36. Tsay, R.S. (2010) *Analysis of Financial Time Series. 3rd edn.* Hoboken, NJ: Wiley. <https://onlinelibrary.wiley.com/doi/book/10.1002/9780470644560>
37. Wood, R.A., McInish, T.H. & Ord, J.K. (1985) '*An investigation of transactions data for NYSE stocks*', Journal of Finance, 40(3), pp. 723–739. doi:10.1111/j.1540-6261.1985.tb04991.
38. Yao, X., & Pérez, Y. (2025). *A ML-Based Resource Allocation Scheme for Energy Optimization in 5G NR. Sensors*, 25(16), 4978. (Accessed: 21 July 2025).
39. Ye, Y., Pei, H. & Jiang, F. (2020) '*Multi-agent deep reinforcement learning for portfolio management*', Applied Intelligence, 50, pp. 3538–3551. <https://doi.org/10.1007/s10489-019-01590-7>

40. Zhang, C., Yang, Y. & Wang, Y. (2021) '*Multi-agent reinforcement learning for FX trading: Coordinated learning under non-stationarity*', *Quantitative Finance*, 21(12), pp. 1947–1962. <https://doi.org/10.1080/14697688.2021.1880024>
41. Zhang, K., Yang, Z. & Başar, T. (2021) '*Multi-agent reinforcement learning: A selective overview*', *Proceedings of the IEEE*, 109(5), pp. 749–768. <https://doi.org/10.1109/JPROC.2021.3058442>
42. Zhang, Z., Zohren, S. & Roberts, S. (2020) '*Deep reinforcement learning for trading*', *The Journal of Financial Data Science*, 2(2), pp. 25–40. <https://doi.org/10.3905/jfds.2020.1.023>
43. Zhang, Y., Liu, H., Wu, Q. & Wang, J. (2024) '*Contextual multi-agent reinforcement learning for financial markets*', *Expert Systems with Applications*, 234, 121090. <https://doi.org/10.1016/j.eswa.2023.121090>

(B) Other Sources- Technical /industry papers

1. **Data Sources: Bloomberg Education (2025).** (Accessed: 20 June 2025) www.bloomberg.com.
2. Anthropic (2024) Model Context Protocol (MCP): A protocol for tool and memory interoperability among AI agents. Technical specification. <https://www.anthropic.com> (Accessed: 20 June 2025). (access route to spec; confirm project page/archived spec link per your access)
3. Jiang, Z., Xu, D. & Liang, J. (2017) 'A deep reinforcement learning framework for the financial portfolio management problem', 1706.10059. <https://arxiv.org/abs/1706.10059>
4. Krishnan, A. (2025) '*Cooperative multi-agent systems via model context protocols: Towards coherent decision-making*', *Journal of Artificial Intelligence Research*, 74(1), pp. 112–138. (Accessed: 20 June 2025).
5. Munkhdalai, T., Li, X., Gopalakrishnan, S. & Yu, H. (2021) "*Credit risk modelling with deep learning*": A survey. arXiv preprint arXiv:2108.04159. Available at: <https://arxiv.org/abs/2108.04159> (Accessed: 1 August 2025).
6. Nii, H.P. (1986) 'Blackboard systems', *AI Magazine*, 7(3), pp. 38–53. <https://doi.org/10.1609/aimag.v7i3.573>
7. Schillace, S. (2023) 'The need for contextual continuity in AI systems', *Microsoft Research Blog*. <https://www.microsoft.com/en-us/research/blog/>

APPENDIX

The intraday OHLCV data (5-minute bars) for Jan 2024 – Jun 2025 were retrieved through Bloomberg Education (University of Westminster). After discussion with the Supervisor (Prof. Yumei Yao, June 2025), 5-minute data for ~18 months was deemed acceptable as a compromise between data richness, methodological sophistication, and practicability. Preprocessing steps and the batching process are explained in the Methodology section (Section 3.3).

ETHICS FORM SUBMISSION



Ethics application (Postgraduate taught)

Title	Ethics application ETH2526-0226
Application ID	ETH2526-0226
Researcher	Mr Prabudh Raghuram
Project	Developing a Multi-Agent Reinforcement Learning (MARL) framework employing the Model Context Protocol (MCP) for the purpose of Dynamic Asset Allocation within Algorithmic Trading Environments
Date	04 Sep 2025
Academic year	2025 - 2026
Supervisor	Dr Yumei Yao
Ethics reviewers	Dr Yumei Yao

Application timeline



Ethics Approval

Awaiting Supervisor decision

Prabudh Raghuram started the Ethics approval process	04 Sep 2025, 01:16
Prabudh Raghuram confirmed statements - view...	04 Sep 2025, 01:27
Prabudh Raghuram submitted the Ethics application	04 Sep 2025, 01:27
Prabudh Raghuram added a note:	04 Sep 2025, 01:28
Academic Year- September 2024- September 2025	
Title: "Developing a Multi-Agent Reinforcement Learning (MARL) framework employing the Model Context Protocol (MCP) for Dynamic Asset Allocation within Algorithmic Trading Environments"	

A. Data Pipeline and Pre-Processing Details

It was obtained from Bloomberg EDU and was made up of 5-minute OHLCV bars across five liquid U.S. equities and ETFs (AAPL, MSFT, AMZN, TSLA, SPY) between January 2024 and June 2025. Checks on data integrity involved corporate action adjustments, normalisation to regular trading hours, and removal of pre-/post-market ticks. It was unusual to have missing bars and was dealt with through forward-fill where appropriate.

All attributes were normalised utilising statistical data exclusively from the training phase to prevent look-ahead bias. Each agent was provided with a continuous record of 12 returns (approximately 1 hour), in conjunction with the volume-to-intraday-average and its previous weight. These modifications maintained intraday trends while ensuring that the dimensions remained manageable.

B. Trading Environment and Constraints

The environment adopted long-only strategies across a maximum 30% weight per asset and proportional trading costs equalling 5 bps per turnover unit. Such a structure is motivated by two reasons:

Practical similarity, the restrictions map onto common rules seen in institutional portfolio management, such as long-only mandates and per-asset limits aimed at keeping concentration risk low.

Methodological openness with the employment of transparent cost modelling the results aren't inflated needlessly, while turnover penalties serve to dissuade "ping-pong" trading commonly observed in unconstrained reinforcement learning agents. Frictions in execution did not include spread dynamics or selection costs due to adverse selection. Though this diminishes realism, this allows a contained examination of MARL architecture effects without unduly complicating market microstructure modelling.

C. Hyperparameter Selection

Hyperparameters (e.g., PPO clip = 0.2, entropy = 0.01, $\gamma = 0.99$, $\lambda = 0.95$, learning rate = 3×10^{-4}) were chosen pragmatically from previous literature (Schulman et al., 2016; Henderson et al., 2018). A curriculum was used at the training onset to gradually increase maximum allocation limits without becoming catastrophic when agents were not trained.

It was avoided to extend hyperparameter tuning both because it was not computationally tractable and to avoid overfitting to the training window. This practical consideration may drive agents to conservative equilibriums but is consistent with research practice within RL-for-finance applications.

D. Baseline Justification

Baselines were selected cautiously: Equal Weight (EW): Represents the simplest robust allocation method; serves as a benchmark for diversification without optimisation.

Inverse Volatility (InvVol): Imbues risk awareness into volatility targeting, a preference widely adopted in practice.

Minimum Variance (MinVar): It optimises portfolio variance using sample covariance estimates. We chose it because it is a direct MPT descendant and focuses on estimation error risks. Having these baselines allowed us to compare MARL and single-agent RL performance both against naive heuristics and against higher-level optimisers.

E. Statistical Robustness

Backtest over-interpretation is a frequent fallacy in finance (Bailey & López de Prado, 2014). Three robustness checks were employed to compensate for this.

- Probabilistic Sharpe Ratio (PSR): It controls for skewness/kurtosis of returns and outputs probability that Sharpe > 0 .
- Deflated Sharpe Ratio (DSR): Deflates Sharpe to account for multiple trials and dependence.
- Block Bootstrap: Preserves intraday relationship keeping only blocks of full trading days (78 bars).

Together, these strategies help decrease the probability of false discoveries. However, the results require cautious interpretation due to the short six-month out-of-sample period.

F. Supplementary Results

Weight allocations. MARL agents moved towards symmetric allocations; average pairwise correlations in a day between weight changes and between weights were often around 1.0, demonstrating policy degeneracy instead of complementary specialisation (Hernandez-Leal, Kartal & Taylor, 2019).

Turnover patterns. The single-agent PPO displayed near-zero turnover (cash-seeking behaviour). MARL maintained non-zero turnover with periodic multi-asset reallocations; MCP alerts reduced churning spikes by enforcing cool-offs during elevated turnover episodes.

Alert dynamics (MCP). Binary TTL alerts (drawdown $> 5\%$; prior-hour turnover $> 50\%$) were infrequent but portfolio-wide when triggered, coordinating synchronous de-risking for ~ 30 minutes. They acted as reactive risk governors, not predictive alpha sources consistent with design (Foerster et al., 2018; Nii, 1986).

Economic drivers. Exposure heatmaps reveal allocations respecting the 30% cap and implicitly leaning towards low-vol assets (an InvVol-like bias), with MCP tweaking risk-on/risk-off in stress (Markowitz, 1952; Cont, 2001).

Turnover-performance trade-off. HOLD_N (15-minute cadence), action smoothing ($\alpha = 0.2$), and explicit penalty for turnover decreased microstructure churn, but reasonable 5 bps costs made frequent rebalancing punitive for a 5-minute horizon (Almgren & Chriss, 2001; Gatheral, 2010).

Note: Out-of-sample, both RL variants finish $\approx -5\%$ after costs, while InvVol/MinVar/EW are $\approx +7-8\%$. PSR/DSR and bootstrap p-values do not support RL skill relative to SR = 0, matching the main text (Politis & Romano, 1994; Bailey & López de Prado, 2014).

Single-agent PPO behaviour. Exposure heatmaps confirm retreat to cash, entropy collapse, and low activity typical under cost-aware rewards at intraday frequency (Sutton & Barto, 2018; Henderson et al., 2018).

G. Limitations Reiterated

- Opportunity Set: A constricted universe of five highly correlated U.S. large-caps limits diversification possibilities. It was done on purpose to eliminate method effects but decreases generalisability.
- Out-of-Sample Horizon: Only six months, which limits exposure to multiple market regimes. Future work should employ rolling windows across several years.
- Cost Modelling: Proportional transaction costs ignore spread and impact, which likely lowers costs of frequent RL rebalancing.
- Symmetry Bias: Joint rewards and joint context led to nearly identical policies between agents. With no diversity mechanism designed in, MARL can potentially converge to EW-like solutions.
- Alert Design: Binary TTL-based alerts (turnover, drawdown) are rough and reactive. Smoother, richer context variables would permit finer adaptation.

H. Reproducibility and Cataloguing of Artefacts

Random seeds used: {11, 17, 23} for learning curves; base runs RANDOM_SEED=42 (list them).

Hardware: CPU/GPU, RAM; average wall-clock per experiment.

Precise train/test split boundaries and time zone.

Folder map of outputs (equity/returns/turnover; table_4_1; training curves).

Software & Versions: Python, NumPy, Pandas, SciPy, PyTorch, Matplotlib (pin versions).

I. Future Opportunities

Future research might investigate:

- Introducing role-specialised agents (allocator, hedger, trend-follower).
- Constructing graduated MCP alerts (constant volatility/liquidity regimes rather than binomial triggers).
- Considering broader contexts and broader time horizons to assess external validity.
- Experimenting with other actor-critic families (e.g., TD3, SAC, entropy-scheduled PPO) that have better exploration stability trade-offs.

J. CODES

Tool and Language– Jupyter Notebook (Anaconda Navigator), Python

Data Source- Bloomberg Education 2025

```
import os

from pathlib import Path

from datetime import datetime


# ---- Safe plotting + threads (prevents GUI issues) ----

os.environ["MPLBACKEND"] = "Agg"

os.environ["MPLCONFIGDIR"] = str(Path("./mpl-cache").resolve())

Path(os.environ["MPLCONFIGDIR"]).mkdir(parents=True, exist_ok=True)

os.environ["OMP_NUM_THREADS"] = "1"

os.environ["MKL_NUM_THREADS"] = "1"

os.environ["OPENBLAS_NUM_THREADS"] = "1"

os.environ["NUMEXPR_NUM_THREADS"] = "1"

os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"

os.environ["PYTHONIOENCODING"] = "utf-8"


# ---- User config (edit Excel filename if needed) ----

EXCEL_PATH = "jan2024_jun2025.xlsx"    # auto-detects any *.xlsx if this exact name is missing

RESULTS_DIR = "results"


MAIN_COST_BPS = 5.0                # main analysis @ 5 bps

SENS_COST_BPS = 3.0                # sensitivity @ 3 bps

CAP_PER_ASSET = 0.30                # long-only per-asset cap at rebalance targets

LOOKBACK_BARS = 12                 # 12×5min = 1 hour context

EPOCHS = 30                        # increase if you have time/compute

TRAIN_TEST_SPLIT = ("2025-01-01", "2025-06-30") # Train < Jan-1-2025; Test = 2025-H1
```

```

RANDOM_SEED = 42

BOOT_B = 1000                # bootstrap replicates

BOOT_BLOCK = 78              # block length (bars) = ~1 trading day


# ---- Python imports (no torch here) ----

import math, random, gc, warnings, re

import numpy as np

import pandas as pd

import matplotlib; matplotlib.use("Agg")

import matplotlib.pyplot as plt

from tqdm import trange

from dataclasses import dataclass

from scipy.optimize import minimize

from scipy.stats import skew, kurtosis, norm


warnings.filterwarnings("ignore", category=FutureWarning)


# ---- Output run folder ----

RUN_DIR = Path(RESULTS_DIR) / ("run_" + datetime.now().strftime("%Y%m%d_%H%M%S"))

RUN_DIR.mkdir(parents=True, exist_ok=True)

print(f"[RUN DIR] {RUN_DIR.resolve()}")


# %% [markdown]

# 1) Utilities (seeds, metrics, significance, turnover, caps, writers)


# %%

def set_global_seed(seed: int):

```

```

random.seed(seed); np.random.seed(seed)

os.environ["PYTHONHASHSEED"] = str(seed)

set_global_seed(RANDOM_SEED)

# 5-min annualisation

BARS_PER_DAY = 78

TRADING_DAYS_PER_YEAR = 252

BARS_PER_YEAR = BARS_PER_DAY * TRADING_DAYS_PER_YEAR

def sharpe_ratio(r: np.ndarray) -> float:

    r = np.asarray(r, dtype=float)

    mu = np.nanmean(r); sd = np.nanstd(r, ddof=1)

    return np.nan if (sd == 0 or np.isnan(sd)) else (mu / sd) * math.sqrt(BARS_PER_YEAR)

def sortino_ratio(r: np.ndarray) -> float:

    r = np.asarray(r, dtype=float)

    dr = r[r < 0]; ds = np.nanstd(dr, ddof=1); mu = np.nanmean(r)

    return np.nan if (ds == 0 or np.isnan(ds)) else (mu / ds) * math.sqrt(BARS_PER_YEAR)

def max_drawdown(equity: np.ndarray) -> float:

    """

    Returns positive peak-to-trough drawdown (e.g., 0.23 means -23%).

    """

    eq = np.asarray(equity, dtype=float)

    if eq.size == 0:

        return 0.0

```

```
peak = np.maximum.accumulate(eq)
```

```
dd = (peak - eq) / peak
```

```
return float(np.nanmax(dd))
```

```
def drawdown_curve(equity: pd.Series) -> pd.Series:
```

```
    roll = equity.cummax()
```

```
    return equity / roll - 1.0
```

```
def cap_project(w, cap=0.30):
```

```
    """
```

Project raw action vector w onto the long-only capped simplex:

- w_i in $[0, \text{cap}]$

- $\text{sum}(w) \leq 1$ (cash allowed)

```
    """
```

```
    w = np.asarray(w, dtype=float).ravel()
```

```
    w = np.clip(w, 0.0, cap)
```

```
    s = w.sum()
```

```
    if s <= 0:
```

```
        return np.ones_like(w) / len(w)
```

```
    if s > 1.0:
```

```
        w = w / s
```

```
    return w
```

```
def probabilistic_sharpe_ratio(r: np.ndarray, sr_benchmark: float = 0.0) -> float:
```

```
    r = np.asarray(r, dtype=float); n = len(r)
```

```
    if n < 3: return np.nan
```

```
    sr_bar = sharpe_ratio(r) / math.sqrt(BARS_PER_YEAR)
```

```

g3 = skew(r, bias=False); g4 = kurtosis(r, fisher=False, bias=False)

den = math.sqrt(max(1e-12, 1 - g3*sr_bar + ((g4 - 1.0)/4.0)*(sr_bar**2)))

z = ((sr_bar - sr_benchmark) * math.sqrt(n - 1)) / den

return float(norm.cdf(z))

```

```

def deflated_sharpe_ratio(r: np.ndarray, n_trials: int = 10, rho: float = 0.5) -> float:

```

```

    r = np.asarray(r, dtype=float); n = len(r)

    if n < 3: return np.nan

    sr_bar = sharpe_ratio(r) / math.sqrt(BARS_PER_YEAR)

    g3 = skew(r, bias=False); g4 = kurtosis(r, fisher=False, bias=False)

    den = math.sqrt(max(1e-12, 1 - g3*sr_bar + ((g4 - 1.0)/4.0)*(sr_bar**2)))

    z_star = norm.ppf(1 - 1.0 / max(2, n_trials)) * math.sqrt(max(1e-6, 1 - rho))

    sr_star = z_star / math.sqrt(n - 1)

    z = ((sr_bar - sr_star) * math.sqrt(n - 1)) / den

    return float(norm.cdf(z))

```

```

def sharpe_block_bootstrap_pvalue(r: np.ndarray, B: int = 1000, block: int = 20, sr_benchmark: float = 0.0)
-> float:

```

```

    r = np.asarray(r, dtype=float); n = len(r)

    if n < 3: return np.nan

    starts = np.arange(0, n - block + 1)

    if len(starts) == 0: return np.nan

    blocks = np.array([r[s:s+block] for s in starts])

    srs = []

    rng = np.random.default_rng(123)

    for _ in range(B):

        k = int(np.ceil(n / block))

```



```

idx = rng.integers(0, len(blocks), size=k)

samp = np.concatenate(blocks[idx], axis=0)[:n]

srs.append(sharpe_ratio(samp) / math.sqrt(BARS_PER_YEAR))

srs = np.array(srs)

return float((srs <= sr_benchmark).mean())

def turnover_from_weights(W: pd.DataFrame) -> pd.Series:

    dW = W.diff().abs()

    return dW.sum(axis=1).fillna(0.0)

def _write_single_metrics(outdir, sr, so, mdd, psr, dsr, p_boot, n_trials, rho):

    with open(outdir / "single_metrics.txt", "w", encoding="utf-8") as f:

        f.write(f"Sharpe={sr:.3f}, Sortino={so:.3f}, MaxDD={mdd:.3f}\n")

        f.write(f"PSR(>0)={psr:.3f},    DSR~(N={n_trials},    rho={rho})={dsr:.3f},    Bootstrap-p(one-
sided)={p_boot:.3f}\n")

    _dump_json_metrics(Path(outdir), {

        "Sharpe":sr,"Sortino":so,"MaxDD":mdd,"PSR":psr,"DSR":dsr,"p_boot":p_boot,

        "n_trials":n_trials, "rho":rho

    })

    print(f"[SINGLE] OOS Sharpe={sr:.3f} Sortino={so:.3f} MaxDD={mdd:.3f} | PSR={psr:.3f}
DSR~={dsr:.3f} p_boot={p_boot:.3f}")

def _write_marl_metrics(outdir, sr, so, mdd, psr, dsr, p_boot, n_trials, rho, use_ctx):

    with open(outdir / "marl_metrics.txt", "w", encoding="utf-8") as f:

        f.write(f"Sharpe={sr:.3f}, Sortino={so:.3f}, MaxDD={mdd:.3f}\n")

        f.write(f"PSR(>0)={psr:.3f},    DSR~(N={n_trials},    rho={rho})={dsr:.3f},    Bootstrap-p(one-
sided)={p_boot:.3f}\n")

    _dump_json_metrics(Path(outdir), {

```

```

"Sharpe":sr,"Sortino":so,"MaxDD":mdd,"PSR":psr,"DSR":dsr,"p_boot":p_boot,

    "n_trials":n_trials, "rho":rho, "use_ctx":bool(use_ctx)

    })

onoff = "ON" if use_ctx else "OFF"

print(f"[MARL {onoff}] OOS Sharpe={sr:.3f} Sortino={so:.3f} MaxDD={mdd:.3f} | PSR={psr:.3f}
DSR~={dsr:.3f} p_boot={p_boot:.3f}")

import json

def _dump_json_metrics(outdir: Path, d: dict):

    with open(outdir / "metrics.json", "w", encoding="utf-8") as f:

        json.dump(d, f, indent=2)

def _append_train_logs(log_file: Path, value: float):

    with open(log_file, "a", encoding="utf-8") as f:

        f.write(str(float(value)) + "\n")

def _plot_train_curves(outdir: Path, title_prefix: str = ""):

    import pandas as _pd

    # entropy

    p = outdir / "train_entropy_curve.csv"

    if p.exists():

        s = _pd.read_csv(p, header=None).iloc[:,0]

        fig = plt.figure(figsize=(10,4)); s.plot()

        plt.title(f'{title_prefix} Training: policy entropy (avg)')

        plt.tight_layout(); fig.savefig(outdir / "train_entropy_curve.png"); plt.close(fig)

    # episodic return

```

```

p = outdir / "train_return_curve.csv"

if p.exists():

    s = _pd.read_csv(p, header=None).iloc[:,0]

    fig = plt.figure(figsize=(10,4)); s.plot()

    plt.title(f'{title_prefix} Training: episodic return')

    plt.tight_layout(); fig.savefig(outdir / "train_return_curve.png"); plt.close(fig)

```

2) Excel → Clean OHLCV (robust Bloomberg EDU parser) + EDA

```

from pandas import ExcelFile

```

```

def _parse_bloomberg_sheet(xls: ExcelFile, sheet_name: str) -> pd.DataFrame:

```

```

    raw = xls.parse(sheet_name, header=None)

```

```

    header_idx = None

```

```

    for i in range(min(30, len(raw))):

```

```

        v = raw.iloc[i, 0]

```

```

        if isinstance(v, str) and v.strip().lower() == "dates":

```

```

            header_idx = i; break

```

```

    if header_idx is None:

```

```

        raise ValueError(f'Dates' header not found in sheet: {sheet_name}')

```

```

    headers = raw.iloc[header_idx].tolist()

```

```

    df = raw.iloc[header_idx + 1:].copy()

```

```

    df.columns = headers

```

```

    cmap = {str(c).lower().strip(): c for c in df.columns}

```

```

    def pick(*names):

```

```

        for n in names:

```

```

        if n in cmap: return cmap[n]

    return None

dtc = pick("dates", "date time", "datetime", "date", "time")

oc = pick("open")

hc = pick("high")

lc = pick("low")

cc = pick("px last", "close", "last price", "last")

vc = pick("volume", "tot volume")

out = pd.DataFrame({

    "Datetime": pd.to_datetime(df[dtc], errors="coerce"),

    "Open": pd.to_numeric(df[oc], errors="coerce"),

    "High": pd.to_numeric(df[hc], errors="coerce"),

    "Low": pd.to_numeric(df[lc], errors="coerce"),

    "Close": pd.to_numeric(df[cc], errors="coerce"),

    "Volume": pd.to_numeric(df[vc], errors="coerce"),

})

out["Ticker"] = sheet_name.split()[0]

out = out.dropna(subset=["Datetime", "Close"]).sort_values("Datetime")

return out[["Ticker", "Datetime", "Open", "High", "Low", "Close", "Volume"]]

def excel_to_clean_df(xlsx_path: str) -> pd.DataFrame:

    assert Path(xlsx_path).exists(), f"Excel not found at {xlsx_path}"

    xls = ExcelFile(xlsx_path)

    frames = [_parse_bloomberg_sheet(xls, s) for s in xls.sheet_names]

    ohlcv = pd.concat(frames, ignore_index=True)

```

```

return ohlcv

# --- Run preprocessing ---

if not Path(EXCEL_PATH).exists():

    candidates = list(Path(".").glob("*.xlsx"))

    if candidates:

        EXCEL_PATH = str(candidates[0])

        print(f"[INFO] Falling back to detected Excel: {EXCEL_PATH}")

    else:

        raise FileNotFoundError("Excel not found. Place your xlsx next to the notebook or update EXCEL_PATH.")


ohlcv = excel_to_clean_df(EXCEL_PATH)

clean_csv = RUN_DIR / "ohlcv_5min.csv"

ohlcv.to_csv(clean_csv, index=False)

print(f"[OK] Cleaned CSV -> {clean_csv}")

ohlcv.head()


# 2.1) EDA (audit + core plots)


# %%


def run_eda(ohlcv_df: pd.DataFrame, outdir: Path,

            split_start="2025-01-01", split_end="2025-06-30",

            max_plot_assets=12, skip_corr_if_many=True):

    outdir.mkdir(parents=True, exist_ok=True)

    df = ohlcv_df.copy()

```

```

df["Ticker"] = df["Ticker"].astype("category")

for col in ["Open", "High", "Low", "Close", "Volume"]:

    df[col] = pd.to_numeric(df[col], errors="coerce").astype("float32")


assets = sorted(df["Ticker"].cat.categories.tolist())

print(f"[EDA] assets={len(assets)}")

panel = (df.pivot_table(index="Datetime", columns="Ticker", values="Close")

        .sort_index().astype("float32"))


# ---- Audit summary ----

rows=[]

for t in assets:

    d = df[df["Ticker"] == t]

    days = d["Datetime"].dt.date.nunique()

    bars = len(d)

    per_day = d.groupby(d["Datetime"].dt.date)["Datetime"].count()

    miss_ratio = 1 - (bars / max(1, days * BARS_PER_DAY))

    rows.append({"Ticker":t,

                "Start": d["Datetime"].min(), "End": d["Datetime"].max(),

                "TradingDays": int(days), "TotalBars": int(bars),

                "MeanBars/Day": float(per_day.mean()),

                "MedianBars/Day": float(per_day.median()),

                "MissingBarsRatio(est)": float(miss_ratio)})

pd.DataFrame(rows).to_csv(outdir/"audit_summary.csv", index=False)

print(f"[EDA] Audit -> {outdir/'audit_summary.csv'}")


# ---- Plots ----

```

```

plot_assets = assets[:max_plot_assets]

norm = panel[plot_assets] / panel[plot_assets].iloc[0]

fig = plt.figure(figsize=(10,4)); norm.plot(ax=plt.gca(), linewidth=1.0)

plt.title("Normalized Close (start=1.0)"); plt.tight_layout()

fig.savefig(outdir/"eda_normalized.png"); plt.close(fig)


rets = panel.pct_change(fill_method=None).replace([np.inf,-np.inf], np.nan).dropna()

fig = plt.figure(figsize=(10,4))

for a in plot_assets: rets[a].hist(bins=80, alpha=0.35)

plt.title("Distribution of 5-minute Returns"); plt.xlabel("Return"); plt.ylabel("Frequency")

plt.tight_layout(); fig.savefig(outdir/"eda_return_hist.png"); plt.close(fig)


ra = rets[plot_assets].abs()

ra["minute_of_day"] = ra.index.hour * 60 + ra.index.minute

profile = ra.groupby("minute_of_day").mean().mean(axis=1)

fig = plt.figure(figsize=(10,4)); profile.plot()

plt.title("Average |5-min Return| by Minute-of-Day"); plt.tight_layout()

fig.savefig(outdir/"eda_intraday_profile.png"); plt.close(fig)

del ra; gc.collect()


if (not skip_corr_if_many) or (len(assets) <= 100):

    corr_train = rets.loc[rets.index < split_start, assets].corr()

    corr_test = rets.loc[(rets.index >= split_start) & (rets.index <= split_end), assets].corr()

    for name, C in [("train",corr_train), ("test",corr_test)]:

        fig = plt.figure(figsize=(6,5))

        im = plt.imshow(C.values, vmin=-1, vmax=1); plt.colorbar()

        plt.xticks(range(len(assets)), assets, rotation=45, ha="right")

```



```

plt.yticks(range(len(assets)), assets)

plt.title(f'Return Correlation {name.upper()}'); plt.tight_layout()

fig.savefig(outdir/f'eda_corr_{name}.png'); plt.close(fig)

del corr_train, corr_test

else:

    print(f'[EDA] Skipping correlation heatmaps for {len(assets)} assets.")

gc.collect()

return panel, assets

panel_close, asset_list = run_eda(ohlc, RUN_DIR, split_start=TRAIN_TEST_SPLIT[0],
split_end=TRAIN_TEST_SPLIT[1])

# 3) Baselines (EW static, MinVar static, InvVol monthly rebalanced) cost-aware

# %%

def weights_equal(n):

    return np.ones(n)/n

def weights_invvol(vol: pd.Series):

    iv = 1.0 / vol.replace(0, np.nan)

    iv = iv.fillna(0.0).values

    if iv.sum() == 0: iv = np.ones_like(iv)

    return iv / iv.sum()

def weights_minvar(cov: pd.DataFrame):

```

```

n = cov.shape[0]; C = cov.values

def obj(w): return float(w @ C @ w)

cons = [{'type':'eq','fun': lambda w: np.sum(w) - 1.0}]

bnds = [(0,1)] * n

x0 = np.ones(n) / n

res = minimize(obj, x0, bounds=bnds, constraints=cons, method='SLSQP', options={'maxiter':200})

return res.x if res.success else x0


def simulate_static(rets: pd.DataFrame, w: np.ndarray, cost_per_unit: float):
    """
    Buy-and-hold OOS with one-time entry cost (charged at t0 in the first bar).
    Weights drift continuously (self-financing) between rebalances.
    """
    pr = []

    w_curr = np.array(w, dtype=float).copy()

    for i in range(len(rets)):
        r_vec = rets.iloc[i].values

        port_ret = float(np.dot(w_curr, r_vec))

        pr.append(port_ret)

        # drift weights (self-financing)

        gross = w_curr * (1.0 + r_vec)

        denom = (1.0 + port_ret)

        if denom != 0.0:
            w_curr = gross / denom

```

```

pr = pd.Series(pr, index=rets.index, name="ret")

if len(pr) > 0:

    pr.iloc[0] -= cost_per_unit * float(np.abs(w).sum()) # <-- entry cost here

eq = (1.0 + pr).cumprod()

eq.name = "equity"

return pr, eq


def simulate_monthly_rebalanced(rets: pd.DataFrame, w_fn, vol=None, cov=None,
                                cap=0.30, cost_per_unit=0.0005, method="invvol"):

    """Rebalance at month starts; apply costs when weights change. Returns (ret, equity, turnover_series)."""

    idx = rets.index

    # robust month key

    month_keys = idx.to_period("M").astype(str).values

    pr_list, eq_path, to_list = [], [], []

    eq = 1.0

    prev_w = np.zeros(rets.shape[1]) # start in cash

    for i in range(len(idx)):

        # rebalance at first bar of month

        if i == 0 or month_keys[i] != month_keys[i-1]:

            if method == "invvol":

                w = weights_invvol(vol)

            elif method == "minvar":

                w = weights_minvar(cov)

            elif method == "ew":

                w = weights_equal(rets.shape[1])

            else:

```

```
w = weights_equal(rets.shape[1])
```

```
w = cap_project(w, cap=cap)
```

```
turnover = np.abs(w - prev_w).sum()
```

```
if turnover > 0:
```

```
    eq *= (1.0 - cost_per_unit * turnover)
```

```
    prev_w = w.copy()
```

```
else:
```

```
    turnover = 0.0
```

```
r_bar = float(prev_w @ rets.iloc[i].values)
```

```
pr_list.append(r_bar)
```

```
eq *= (1.0 + r_bar)
```

```
eq_path.append(eq)
```

```
to_list.append(float(turnover))
```

```
ret_s = pd.Series(pr_list, index=rets.index, name="ret")
```

```
eq_s = pd.Series(eq_path, index=rets.index, name="equity")
```

```
to_s = pd.Series(to_list, index=rets.index, name="turnover")
```

```
return ret_s, eq_s, to_s
```

```
def save_baseline_weight_bars(vol, cov, assets, outdir: Path, cap=0.30):
```

```
    w_ew = cap_project(weights_equal(len(assets)), cap)
```

```
    w_iv = cap_project(weights_invvol(vol), cap)
```

```
    w_mv = cap_project(weights_minvar(cov), cap)
```

```
    combos = {
```

```
        "EW": w_ew,
```

```

    "InvVol": w_iv,

    "MinVar": w_mv,

}

for name, w in combos.items():

    fig = plt.figure(figsize=(7,3))

    plt.bar(assets, w); plt.xticks(rotation=45, ha="right"); plt.ylabel("Weight")

    plt.title(f"Baseline Weights {name}"); plt.tight_layout()

    fig.savefig(outdir / f"wgt_{name.replace(' ','_')}.png"); plt.close(fig)

return combos


def run_baselines(panel_close: pd.DataFrame, assets, cost_bps=5.0, cap=0.30, outdir: Path = RUN_DIR):
    """
    - EW: static buy-and-hold (entry cost only)
    - MinVar: static buy-and-hold (entry cost only)
    - InvVol: monthly rebalanced (costs at month boundaries)

    Saves: equity plots, metrics CSV, baseline turnover CSV for InvVol.
    """

    rets = panel_close.pct_change(fill_method=None).replace([np.inf, -np.inf], np.nan).dropna()

    train = rets.loc[rets.index < TRAIN_TEST_SPLIT[0], assets].dropna()

    test = rets.loc[(rets.index >= TRAIN_TEST_SPLIT[0]) & (rets.index <= TRAIN_TEST_SPLIT[1]),
assets].dropna()


    vol = train.std(); cov = train.cov()

    save_baseline_weight_bars(vol, cov, assets, outdir, cap=cap)


    base_defs = {

        "EW": cap_project(weights_equal(len(assets)), cap),

```

```

"InvVol": cap_project(weights_invvol(vol), cap),

"MinVar": cap_project(weights_minvar(cov), cap),

}

cost_per_unit = cost_bps / 10000.0

rows = []

for name, w in base_defs.items():

    if name == "InvVol":

        pr, eq, to = simulate_monthly_rebalanced(test, weights_invvol, vol=vol, cap=cap,

                                                cost_per_unit=cost_per_unit, method="invvol")

        # save turnover for appendix

        to.to_csv(outdir / f"baseline_turnover_{name}.csv")

    else:

        pr, eq = simulate_static(test, w, cost_per_unit)

        # one-shot entry turnover series for static strategies

        to = pd.Series([float(np.sum(np.abs(w)))] + [0.0]*(len(eq)-1), index=eq.index, name="turnover")

        to.to_csv(outdir / f"baseline_turnover_{name}.csv")


# plot equity

fig = plt.figure(figsize=(10,4)); eq.plot()

plt.title(f"Equity {name}"); plt.tight_layout()

fig.savefig(outdir / f"eq_{name.replace(' ','_')}.png"); plt.close(fig)


# metrics

rows.append({

    "Strategy": name,

    "Sharpe": sharpe_ratio(pr.values),

```

```

    "Sortino": sortino_ratio(pr.values),

    "MaxDD": max_drawdown(eq.values)

    })

```

```

res = pd.DataFrame(rows).set_index("Strategy").sort_index()

res_path = outdir / f"baselines_metrics_cost{int(cost_bps)}bps.csv"

res.to_csv(res_path)

print(f"[BASELINES] Metrics -> {res_path}")

return res

```

```

baseline_main      =      run_baselines(panel_close,      asset_list,      cost_bps=MAIN_COST_BPS,
cap=CAP_PER_ASSET, outdir=RUN_DIR)

baseline_sens      =      run_baselines(panel_close,      asset_list,      cost_bps=SENS_COST_BPS,
cap=CAP_PER_ASSET, outdir=RUN_DIR)

baseline_main, baseline_sens

```

4) RL imports (Torch) + PPO building blocks

```

# %%

```

```

import torch

```

```

import torch.nn as nn

```

```

import torch.optim as optim

```

```

def mlp(sizes, act=nn.Tanh, out_act=None):

```

```

    layers=[]

```

```

    for i in range(len(sizes)-1):

```

```

        layers += [nn.Linear(sizes[i], sizes[i+1])]

```

```

    if i < len(sizes)-2:

        layers += [act()]

    elif out_act is not None:

        layers += [out_act()]

    return nn.Sequential(*layers)

```

```

class ActorContinuous(nn.Module):

```

```

    def __init__(self, obs_dim, act_dim):

        super().__init__()

        # per dissertation: (256,128)

        self.net = mlp([obs_dim, 256, 128, act_dim])

        self.log_std = nn.Parameter(torch.zeros(act_dim))

    def forward(self, obs):

        mu = self.net(obs); std = torch.exp(self.log_std)

        return mu, std

    def dist(self, obs):

        mu, std = self.forward(obs)

        return torch.distributions.Normal(mu, std)

```

```

class Critic(nn.Module):

```

```

    def __init__(self, obs_dim):

        super().__init__()

        # per dissertation: (256,256,128)

        self.v = mlp([obs_dim, 256, 256, 128, 1])

    def forward(self, obs):

        return self.v(obs).squeeze(-1)

```


@dataclass

class PPOConfig:

gamma: float = 0.99

lam: float = 0.95

clip_ratio: float = 0.2

pi_lr: float = 3e-4

vf_lr: float = 3e-4

train_iters: int = 10 # epochs/update

target_kl: float = 0.02

minibatch: int = 1024

entropy_coef: float = 0.01

device: str = "cpu"

def compute_gae(rews, vals, dones, cfg: PPOConfig):

adv = np.zeros_like(rews); lastgaelam = 0.0

for t in reversed(range(len(rews))):

nonterminal = 1.0 - dones[t]

delta = rews[t] + cfg.gamma * vals[t+1] * nonterminal - vals[t]

lastgaelam = delta + cfg.gamma * cfg.lam * nonterminal * lastgaelam

adv[t] = lastgaelam

ret = adv + vals[-1]

return adv, ret

def ppo_update_actor(actor, data, cfg: PPOConfig):

obs = torch.as_tensor(data["obs"], dtype=torch.float32, device=cfg.device)

act = torch.as_tensor(data["act"], dtype=torch.float32, device=cfg.device)

adv = torch.as_tensor(data["adv"], dtype=torch.float32, device=cfg.device)

```

logp_old = torch.as_tensor(data["logp"], dtype=torch.float32, device=cfg.device)

opt = optim.Adam(actor.parameters(), lr=cfg.pi_lr)

for _ in range(cfg.train_iters):

    idx = torch.randperm(len(obs), device=cfg.device)

    for start in range(0, len(obs), cfg.minibatch):

        mb = idx[start:start+cfg.minibatch]

        d = actor.dist(obs[mb]); logp = d.log_prob(act[mb]).sum(-1)

        ratio = torch.exp(logp - logp_old[mb])

        surr1 = ratio * adv[mb]

        surr2 = torch.clamp(ratio, 1-cfg.clip_ratio, 1+cfg.clip_ratio) * adv[mb]

        pi_loss = -torch.min(surr1, surr2).mean()

        if cfg.entropy_coef:

            pi_loss -= cfg.entropy_coef * d.entropy().sum(-1).mean()

        opt.zero_grad(); pi_loss.backward(); opt.step()

        with torch.no_grad():

            kl = (logp_old[mb] - logp).mean().item()

            if kl > 1.5 * cfg.target_kl:

                break

```

```

def ppo_update_critic(critic, data, cfg: PPOConfig):

    obs = torch.as_tensor(data["obs"], dtype=torch.float32, device=cfg.device)

    ret = torch.as_tensor(data["ret"], dtype=torch.float32, device=cfg.device)

    opt = optim.Adam(critic.parameters(), lr=cfg.vf_lr)

    for _ in range(cfg.train_iters):

        idx = torch.randperm(len(obs), device=cfg.device)

        for start in range(0, len(obs), cfg.minibatch):

            mb = idx[start:start+cfg.minibatch]

```

```
v = critic(obs[mb]); loss = ((v - ret[mb])**2).mean()
```

```
opt.zero_grad(); loss.backward(); opt.step()
```

5) Environments + MCP (CTDE for MARL) with HOLD-N, smoothing, turnover penalty

```
HOLD_N = 3      # rebalance every 3 bars (15 minutes)
```

```
LAMBDA_TURN = 0.5 # extra penalty on turnover (beyond cost_bps)
```

```
ALPHA_ACT = 0.2  # action smoothing EMA (0=no smoothing; 0.2 is gentle)
```

```
LAMBDA_L1 = 0.002 # promotes sparse allocations
```

```
TURNOVER_EPS = 1e-6 # snap tiny TO to zero
```

```
ROUND_W = 6      # round weights to 6 decimals to kill float jitter
```

```
class MCPContextStore:
```

```
    def __init__(self, ttl_bars: int = 6*1): # TTL  $\approx$  30 minutes at 5-min bars
```

```
        self.ttl = ttl_bars; self.t = 0; self._store = {}
```

```
    def put(self, key: str, value, ttl: int | None = None):
```

```
        exp = self.t + (ttl if ttl is not None else self.ttl); self._store[key] = (value, exp)
```

```
    def get(self, key: str, default=None):
```

```
        v = self._store.get(key, (default, -1)); return default if v[1] < self.t else v[0]
```

```
    def step(self):
```

```
        self.t += 1
```

```
        for k in list(self._store.keys()):
```

```
            if self._store[k][1] < self.t: del self._store[k]
```

```
class SingleAgentEnv:
```

```
    def __init__(self, rets: pd.DataFrame, assets, lookback=12, w_max=0.30, cost_per_unit=0.0005,
daily_reset=False, use_ctx=True):
```

```

self.rets = rets.copy(); self.assets = list(assets); self.n = len(assets)

self.lookback = lookback; self.w_max = w_max; self.cost_per_unit = cost_per_unit

self.daily_reset = daily_reset; self.use_ctx = use_ctx

self.reset()

```

```

def _obs(self):

    start = max(0, self.ptr - self.lookback)

    win = self.rets.iloc[start:self.ptr][self.assets]; arr = win.values

    if len(win) < self.lookback:

        pad = np.zeros((self.lookback - len(win), self.n)); arr = np.vstack([pad, arr])

    ctxv = np.array([

        self.ctx.get("turnover", 0.0) if self.use_ctx else 0.0,

        self.ctx.get("dd_alert", 0.0) if self.use_ctx else 0.0

    ])

    return np.concatenate([arr.flatten(), ctxv])

```

```

def reset(self, start_index=None):

    self.ptr = 0 if start_index is None else self.rets.index.get_loc(start_index)

    self.step_no = 0

    self.ctx = MCPContextStore(ttlBars=6)

    self.prev_w = None

    self.w = np.zeros(self.n) # start with cash

    self.equity = 1.0

    # internal trackers for MCP

    from collections import deque

    self._to_roll = deque(maxlen=12) # prior hour at 5-min bars

    self._equity_peak = 1.0

```

```
return self._obs()
```

```
def step(self, action: np.ndarray):
```

```
    proposed = np.clip(np.array(action, dtype=float), 0.0, self.w_max)
```

```
    if proposed.sum() > 1.0:
```

```
        proposed = proposed / proposed.sum()
```

```
# Always initialize turnover for this bar (avoids UnboundLocalError)
```

```
    turnover = 0.0
```

```
    if (self.step_no % HOLD_N) == 0:
```

```
        if self.prev_w is not None:
```

```
            proposed = ALPHA_ACT * proposed + (1.0 - ALPHA_ACT) * self.prev_w
```

```
        w_new = cap_project(proposed, cap=self.w_max)
```

```
        w_new = np.round(w_new, ROUND_W)
```

```
        turnover = np.abs(w_new - self.w).sum()
```

```
        if turnover < TURNOVER_EPS:
```

```
            turnover = 0.0
```

```
        self.prev_w = w_new.copy()
```

```
        self.w = w_new
```

```
r_vec = self.rets.iloc[self.ptr][self.assets].values
```

```
port_ret = float(self.w @ r_vec)
```

```
cost = self.cost_per_unit * turnover
```

```
realized_ret = port_ret - cost
```

```
reward = realized_ret - LAMBDA_TURN * turnover - LAMBDA_L1 * np.sum(np.abs(self.w))
```

```

self.equity *= (1.0 + realized_ret)

# --- MCP binary alerts (prior-hour turnover >50%; running DD >5%) ---

self._to_roll.append(float(turnover))

to_hour = sum(self._to_roll)

turnover_alert = 1.0 if (to_hour > 0.50) else 0.0

self._equity_peak = max(self._equity_peak, self.equity)

curr_dd = (self._equity_peak - self.equity) / self._equity_peak

dd_alert = 1.0 if (curr_dd > 0.05) else 0.0

self.ctx.put("turnover", turnover_alert, ttl=6)

self.ctx.put("dd_alert", dd_alert, ttl=6)

done = False

prev_day = self.rets.index[self.ptr].date()

self.ptr += 1; self.step_no += 1; self.ctx.step()

if self.ptr >= len(self.rets):

    done = True

elif self.daily_reset and self.rets.index[self.ptr].date() != prev_day:

    done = True

obs = None if done else self._obs()

info = {

"equity": self.equity,

"turnover": float(turnover),

"cost": float(cost),

```

```

"weights": self.w.copy(),

"realized_ret": float(realized_ret) # <-- add this

}

return obs, reward, done, info

class MultiAgentTeamEnv:

    def __init__(self, rets: pd.DataFrame, assets, lookback=12, w_max=0.30, cost_per_unit=0.0005,
daily_reset=False, use_ctx=True):

        self.rets = rets.copy(); self.assets = list(assets); self.n = len(assets)

        self.lookback = lookback; self.w_max = w_max; self.cost_per_unit = cost_per_unit

        self.daily_reset = daily_reset; self.use_ctx = use_ctx

        self.reset()

    def _obs_agent(self, i: int):

        start = max(0, self.ptr - self.lookback); s = self.assets[i]

        win = self.rets.iloc[start:self.ptr][s]; arr = win.values.reshape(-1, 1)

        if len(win) < self.lookback:

            pad = np.zeros((self.lookback - len(win), 1)); arr = np.vstack([pad, arr])

        ctxv = np.array([

            self.ctx.get("turnover", 0.0) if self.use_ctx else 0.0,

            self.ctx.get("dd_alert", 0.0) if self.use_ctx else 0.0

        ])

        return np.concatenate([arr.flatten(), ctxv])

    def critic_joint_obs(self):

        return np.concatenate([self._obs_agent(i) for i in range(self.n)], axis=0)

```

```

def reset(self, start_index=None):

    self.ptr = 0 if start_index is None else self.rets.index.get_loc(start_index)

    self.step_no = 0

    self.ctx = MCPContextStore(ttlBars=6)

    self.prev_w = None

    self.w = np.zeros(self.n)

    self.equity = 1.0

    from collections import deque

    self._to_roll = deque(maxlen=12)

    self._equity_peak = 1.0

    return {a: self._obs_agent(i) for i, a in enumerate(self.assets)}


def step(self, action_dict):

    a = np.zeros(self.n)

    for i, s in enumerate(self.assets):

        a[i] = float(np.clip(action_dict.get(s, 0.0), 0.0, self.w_max))

    if a.sum() > 1.0:

        a = a / a.sum()

    # Always initialise turnover for this bar (avoids UnboundLocalError)

    turnover = 0.0

    if (self.step_no % HOLD_N) == 0:

        if self.prev_w is not None:

            a = ALPHA_ACT * a + (1.0 - ALPHA_ACT) * self.prev_w

        w_new = cap_project(a, cap=self.w_max)

```



```

w_new = np.round(w_new, ROUND_W)

turnover = np.abs(w_new - self.w).sum()

if turnover < TURNOVER_EPS:

    turnover = 0.0

self.prev_w = w_new.copy()

self.w = w_new


r_vec = self.rets.iloc[self.ptr][self.assets].values

port_ret = float(self.w @ r_vec)

cost = self.cost_per_unit * turnover

realized_ret = port_ret - cost

reward = realized_ret - LAMBDA_TURN * turnover - LAMBDA_L1 * np.sum(np.abs(self.w))


self.equity *= (1.0 + realized_ret)


# MCP (binary)

self._to_roll.append(float(turnover))

to_hour = sum(self._to_roll)

turnover_alert = 1.0 if (to_hour > 0.50) else 0.0

self._equity_peak = max(self._equity_peak, self.equity)

curr_dd = (self._equity_peak - self.equity) / self._equity_peak

dd_alert = 1.0 if (curr_dd > 0.05) else 0.0


self.ctx.put("turnover", turnover_alert, ttl=6)

self.ctx.put("dd_alert", dd_alert, ttl=6)


done = False

```

```

prev_day = self.rets.index[self.ptr].date()

self.ptr += 1; self.step_no += 1; self.ctx.step()

if self.ptr >= len(self.rets):

    done = True

elif self.daily_reset and self.rets.index[self.ptr].date() != prev_day:

    done = True


obs = None if done else {a: self._obs_agent(i) for i, a in enumerate(self.assets)}

info = {

"equity": self.equity,

"turnover": float(turnover),

"cost": float(cost),

"weights": self.w.copy(),

"realized_ret": float(realized_ret) # <-- add this

}

return obs, reward, done, info

```

6) Training functions (Single PPO & MARL CTDE-PPO) + training curves

```

# %%

def _weights_from_actor(a, cap=0.30):

    x = np.asarray(a, dtype=float)

    x = np.exp(x - np.max(x))      # stable

    w = x / x.sum()                # simplex

    w = np.minimum(w, cap)         # enforce cap at target

    s = w.sum()

```

```

if s == 0:

    return np.ones_like(w)/len(w)

if s > 1.0:

    w = w / s                # keep sum  $\leq 1$ 

    return w

def _scalar_to_weight(z, cap=0.30, bias=0.5, temp=0.75, floor=1e-4):

    # softplus((x-bias)*temp) encourages many near-zero weights

    x = float(np.ravel(z)[0])

    y = temp * (x - bias)

    sp = math.log1p(math.exp(-abs(y))) + max(y, 0.0) # stable softplus

    w = min(max(sp, 0.0), cap)

    return 0.0 if w < floor else w


def team_softmax(z_vec, cap=0.30):

    """

    Map per-asset logits z (one from each actor) to a single, competing weight vector.

    - Softmax enforces competition across assets.

    - Per-asset cap applied; if sum>1 after capping, scale back to keep  $\Sigma w \leq 1$ .

    """

    z = np.asarray(z_vec, dtype=float).ravel()

    w = np.exp(z - np.max(z))

    w = w / w.sum()

    w = np.minimum(w, cap)

    s = w.sum()

    if s > 1.0:

        w = w / s

    return w

```

```

def train_single(panel_close: pd.DataFrame, assets, outdir: Path,

                cap=0.30, cost_bps=5.0, epochs=30, lookback=12,

                daily_reset=False, device="cpu", use_ctx=True,

                n_trials=10, rho=0.5, boot_B=1000, boot_block=20, seed=42):

    set_global_seed(seed)

    rets = panel_close.pct_change(fill_method=None).replace([np.inf,-np.inf], np.nan).dropna()

    train = rets.loc[rets.index < TRAIN_TEST_SPLIT[0], assets].dropna()

    test = rets.loc[(rets.index >= TRAIN_TEST_SPLIT[0]) & (rets.index <= TRAIN_TEST_SPLIT[1]),
assets].dropna()

    env = SingleAgentEnv(train, assets, lookback=lookback, w_max=cap,

                        cost_per_unit=cost_bps/10000.0, daily_reset=daily_reset, use_ctx=use_ctx)

    obs_dim = env._obs().shape[0]; act_dim = len(assets)

    cfg = PPOConfig(device=device)

    actor = ActorContinuous(obs_dim, act_dim).to(device)

    critic = Critic(obs_dim).to(device)

    train_rewards = []; train_entropies = []

    for _ in trange(epochs, desc="Single-Agent PPO"):

        obs_buf=[]; act_buf=[]; logp_buf=[]; rew_buf=[]; val_buf=[]; done_buf=[]

        obs = env.reset()

        ep_rew_sum = 0.0; ent_sum = 0.0; ent_cnt = 0

```

```

while True:

    ot = torch.as_tensor(obs, dtype=torch.float32, device=device).unsqueeze(0)

    dist = actor.dist(ot)

    a = dist.sample()[0].detach().cpu().numpy()

    w = _weights_from_actor(a, cap=cap)

    logp = dist.log_prob(torch.as_tensor(a, dtype=torch.float32, device=device)).sum(-1).detach().cpu().item()

    v = critic(ot).detach().cpu().item()

    next_obs, r, done, _ = env.step(w)

    # buffers

    obs_buf.append(obs); act_buf.append(a); logp_buf.append(logp); rew_buf.append(r);
    val_buf.append(v); done_buf.append(float(done))

    # stats

    ep_rew_sum += r

    ent_sum += dist.entropy().sum(-1).item(); ent_cnt += 1

    obs = next_obs

    if done: break

    last_v = 0.0 if obs is None else critic(torch.as_tensor(obs, dtype=torch.float32,
device=device).unsqueeze(0)).detach().cpu().item()

    val_buf.append(last_v)

    adv, ret = compute_gae(np.array(rew_buf, dtype=np.float32), np.array(val_buf, dtype=np.float32),
np.array(done_buf, dtype=np.float32), cfg)

    ppo_update_actor(actor, {"obs":np.array(obs_buf), "act":np.array(act_buf), "adv":adv,
"logp":np.array(logp_buf)}, cfg)

```

```

ppo_update_critic(critic, {"obs":np.array(obs_buf), "ret":ret}, cfg)

train_rewards.append(ep_rew_sum)

train_entropies.append(ent_sum / max(1, ent_cnt))

# save training curves

pd.Series(train_rewards).to_csv(outdir/"train_return_curve.csv", index=False)

pd.Series(train_entropies).to_csv(outdir/"train_entropy_curve.csv", index=False)

fig = plt.figure(figsize=(10,4)); plt.plot(train_rewards); plt.title("Training: episodic return");
plt.tight_layout()

fig.savefig(outdir/"train_return_curve.png"); plt.close(fig)

fig = plt.figure(figsize=(10,4)); plt.plot(train_entropies); plt.title("Training: policy entropy");
plt.tight_layout()

fig.savefig(outdir/"train_entropy_curve.png"); plt.close(fig)

# OOS

env_t = SingleAgentEnv(test, assets, lookback=lookback, w_max=cap,

                        cost_per_unit=cost_bps/10000.0, daily_reset=daily_reset, use_ctx=use_ctx)

obs = env_t.reset(); eq=[]; pr=[]; W=[]

while True:

    ot = torch.as_tensor(obs, dtype=torch.float32, device=device).unsqueeze(0)

    mu,_ = actor.forward(ot)

    a = mu[0].detach().cpu().numpy()

    w = _weights_from_actor(a, cap=cap)

    obs, r, done, info = env_t.step(w)

    eq.append(info["equity"])

    pr.append(info["realized_ret"])

```

```

#eq.append(info["equity"]); pr.append(r);

W.append(info["weights"].copy())

if done: break


eq = pd.Series(eq, index=test.index[:len(eq)], name="Equity")

W = pd.DataFrame(W, index=test.index[:len(W)], columns=assets)

to = turnover_from_weights(W)


# save OOS series

pd.Series(pr, index=test.index[:len(pr)], name="ret").to_csv(outdir / "returns_oos.csv")

eq.to_csv(outdir / "equity_oos.csv")


# plots

fig = plt.figure(figsize=(10,4)); eq.plot(); plt.title("Single-Agent PPO Equity (OOS 2025H1)")

plt.tight_layout(); fig.savefig(outdir/"single_equity.png"); plt.close(fig)

dd = drawdown_curve(eq); fig = plt.figure(figsize=(10,4)); dd.plot(); plt.title("Single-Agent PPO
Drawdown (OOS)")

plt.tight_layout(); fig.savefig(outdir/"single_drawdown.png"); plt.close(fig)

fig = plt.figure(figsize=(10,4)); to.plot(); plt.title("Single-Agent PPO Turnover per 5-min bar (OOS)")

plt.tight_layout(); fig.savefig(outdir/"single_turnover.png"); plt.close(fig)

fig = plt.figure(figsize=(10,5)); plt.imshow(W.T.values, aspect="auto", vmin=0.0, vmax=cap);
plt.colorbar()

plt.yticks(range(len(assets)), assets); plt.xlabel("Time"); plt.title("Single-Agent PPO Exposure
Heatmap (OOS)")

plt.tight_layout(); fig.savefig(outdir/"single_exposure_heatmap.png"); plt.close(fig)


# save diagnostics

W.to_csv(outdir/"single_exposures.csv")

```

```

to.to_csv(outdir/"single_turnover_series.csv")

W.to_csv(outdir/"weights.csv") # alias for diagnostics


r = np.array(pr, dtype=float)

sr = sharpe_ratio(r); so = sortino_ratio(r); mdd = max_drawdown(eq.values)

psr = probabilistic_sharpe_ratio(r, sr_benchmark=0.0)

dsr = deflated_sharpe_ratio(r, n_trials=n_trials, rho=rho)

p_boot = sharpe_block_bootstrap_pvalue(r, B=boot_B, block=boot_block, sr_benchmark=0.0)

_write_single_metrics(outdir, sr, so, mdd, psr, dsr, p_boot, n_trials, rho)

return {"Sharpe":sr,"Sortino":so,"MaxDD":mdd,"PSR":psr,"DSR":dsr,"p_boot":p_boot}


def train_marl(panel_close: pd.DataFrame, assets, outdir: Path,

               cap=0.30, cost_bps=5.0, epochs=30, lookback=12,

               daily_reset=False, device="cpu", use_ctx=True,

               n_trials=10, rho=0.5, boot_B=1000, boot_block=20, seed=42):

    set_global_seed(seed)

    rets = panel_close.pct_change(fill_method=None).replace([np.inf,-np.inf], np.nan).dropna()

    train = rets.loc[rets.index < TRAIN_TEST_SPLIT[0], assets].dropna()

    test = rets.loc[(rets.index >= TRAIN_TEST_SPLIT[0]) & (rets.index <= TRAIN_TEST_SPLIT[1]),

assets].dropna()


    env = MultiAgentTeamEnv(train, assets, lookback=lookback, w_max=cap,

                             cost_per_unit=cost_bps/10000.0, daily_reset=daily_reset, use_ctx=use_ctx)

    obs_dim = len(env._obs_agent(0)); joint_dim = obs_dim * len(assets)

    actors = {a: ActorContinuous(obs_dim, 1).to(device) for a in assets}

    critic = Critic(joint_dim).to(device)

```



```

cfg = PPOConfig(device=device)

train_rewards = []; train_entropies = []

for epoch in trange(epochs, desc="MARL CTDE-PPO"):
    # gently decay exploration to avoid drifting to uniform actions
    cfg.entropy_coef = 0.01 * (0.95 ** epoch) # floor? e.g., max(0.001, ...)

    obs_dict = env.reset()

    obs_buf = {a: [] for a in assets}; act_buf = {a: [] for a in assets}; logp_buf = {a: [] for a in assets}
    joint_obs_buf = []; rew_buf=[]; done_buf=[]; val_buf=[]

    ep_rew_sum = 0.0; ent_sum = 0.0; ent_cnt = 0

    while True:
        jo = env.critic_joint_obs()

        v = critic(torch.as_tensor(jo, dtype=torch.float32,
device=device).unsqueeze(0)).detach().cpu().item()

        joint_obs_buf.append(jo); val_buf.append(v)

        # collect per-asset logits, then map jointly to weights
        z_list = []

        for a in assets:
            ot = torch.as_tensor(obs_dict[a], dtype=torch.float32, device=device).unsqueeze(0)
            d = actors[a].dist(ot)

            z = d.sample()[0].detach().cpu().numpy()

            z_scalar = float(np.asarray(z).reshape(-1)[0]) # or: z.item() if it's 0-d

```

```

        z_list.append(z_scalar)

    obs_buf[a].append(obs_dict[a])
    act_buf[a].append(z)
    logp_buf[a].append(
        d.log_prob(torch.as_tensor(z, dtype=torch.float32, device=device)).sum(-
1).detach().cpu().item()
    )

    ent_sum += d.entropy().sum(-1).item(); ent_cnt += 1

w_vec = team_softmax(z_list, cap=cap)
action_dict = {assets[i]: float(w_vec[i]) for i in range(len(assets))}
obs_dict, r, done, _ = env.step(action_dict)

rew_buf.append(r); done_buf.append(float(done))
ep_rew_sum += r
if done: break

last_v = 0.0 if obs_dict is None else critic(torch.as_tensor(env.critic_joint_obs(), dtype=torch.float32,
device=device).unsqueeze(0)).detach().cpu().item()
val_buf.append(last_v)

adv, ret = compute_gae(np.array(rew_buf, dtype=np.float32), np.array(val_buf, dtype=np.float32),
np.array(done_buf, dtype=np.float32), cfg)

for a in assets:

```

```

ppo_update_actor(actors[a], {"obs":np.array(obs_buf[a]), "act":np.array(act_buf[a]).reshape(-1,1),
"adv":adv, "logp":np.array(logp_buf[a])}, cfg)

ppo_update_critic(critic, {"obs":np.array(joint_obs_buf), "ret":ret}, cfg)

train_rewards.append(ep_rew_sum)

train_entropies.append(ent_sum / max(1, ent_cnt))

# save training curves

pd.Series(train_rewards).to_csv(outdir/"train_return_curve.csv", index=False)

pd.Series(train_entropies).to_csv(outdir/"train_entropy_curve.csv", index=False)

fig = plt.figure(figsize=(10,4)); plt.plot(train_rewards); plt.title("Training: episodic return (team)");
plt.tight_layout()

fig.savefig(outdir/"train_return_curve.png"); plt.close(fig)

fig = plt.figure(figsize=(10,4)); plt.plot(train_entropies); plt.title("Training: policy entropy (avg)");
plt.tight_layout()

fig.savefig(outdir/"train_entropy_curve.png"); plt.close(fig)

# OOS

env_t = MultiAgentTeamEnv(test, assets, lookback=lookback, w_max=cap,

                        cost_per_unit=cost_bps/10000.0, daily_reset=daily_reset, use_ctx=use_ctx)

obs = env_t.reset(); eq=[]; pr=[]; W=[]

while True:

    z_list = []

    for a in assets:

        ot = torch.as_tensor(obs[a], dtype=torch.float32, device=device).unsqueeze(0)

        mu,_ = actors[a].forward(ot)

        z_scalar = float(mu.detach().cpu().numpy().reshape(-1)[0]) # <- extract from mu

        z_list.append(z_scalar)

```

```

w_vec = team_softmax(z_list, cap=cap)

action_dict = {assets[i]: float(w_vec[i]) for i in range(len(assets))}

obs, r, done, info = env_t.step(action_dict)

eq.append(info["equity"])

pr.append(info["realized_ret"])

#eq.append(info["equity"]); pr.append(r);

W.append(info["weights"].copy())

if done: break


eq = pd.Series(eq, index=test.index[:len(eq)], name="Equity")

W = pd.DataFrame(W, index=test.index[:len(W)], columns=assets)

to = turnover_from_weights(W)


# save OOS series

pd.Series(pr, index=test.index[:len(pr)], name="ret").to_csv(outdir / "returns_oos.csv")

eq.to_csv(outdir / "equity_oos.csv")


# plots

fig = plt.figure(figsize=(10,4)); eq.plot(); plt.title(f'MARL CTDE-PPO Equity (OOS) MCP {\'ON\' if
use_ctx else \'OFF\'}')

plt.tight_layout(); fig.savefig(outdir/"marl_equity.png"); plt.close(fig)

dd = drawdown_curve(eq); fig = plt.figure(figsize=(10,4)); dd.plot(); plt.title("MARL CTDE-PPO
Drawdown (OOS)")

plt.tight_layout(); fig.savefig(outdir/"marl_drawdown.png"); plt.close(fig)

fig = plt.figure(figsize=(10,4)); to.plot(); plt.title("MARL CTDE-PPO Turnover per 5-min bar (OOS)")

plt.tight_layout(); fig.savefig(outdir/"marl_turnover.png"); plt.close(fig)

fig = plt.figure(figsize=(10,5)); plt.imshow(W.T.values, aspect="auto", vmin=0.0, vmax=cap);
plt.colorbar()

```

```

plt.yticks(range(len(assets)), assets); plt.xlabel("Time"); plt.title("MARL CTDE-PPO Exposure
Heatmap (OOS)")

plt.tight_layout(); fig.savefig(outdir/"marl_exposure_heatmap.png"); plt.close(fig)

corrW = W.diff().corr(min_periods=5).fillna(0.0)

fig = plt.figure(figsize=(6,5))

im = plt.imshow(corrW.values, vmin=-1, vmax=1); plt.colorbar()

plt.xticks(range(len(assets)), assets, rotation=45, ha="right"); plt.yticks(range(len(assets)), assets)

plt.title("MARL Exposure Correlation (OOS)"); plt.tight_layout()

fig.savefig(outdir/"marl_exposure_corr.png"); plt.close(fig)

corrW.to_csv(outdir/"marl_exposure_corr.csv")

# save diagnostics

W.to_csv(outdir/"marl_exposures.csv")

to.to_csv(outdir/"marl_turnover_series.csv")

W.to_csv(outdir/"weights.csv")

r = np.array(pr, dtype=float)

sr = sharpe_ratio(r); so = sortino_ratio(r); mdd = max_drawdown(eq.values)

psr = probabilistic_sharpe_ratio(r, sr_benchmark=0.0)

dsr = deflated_sharpe_ratio(r, n_trials=n_trials, rho=rho)

p_boot = sharpe_block_bootstrap_pvalue(r, B=boot_B, block=boot_block, sr_benchmark=0.0)

_write_marl_metrics(outdir, sr, so, mdd, psr, dsr, p_boot, n_trials, rho, use_ctx)

return {"Sharpe":sr,"Sortino":so,"MaxDD":mdd,"PSR":psr,"DSR":dsr,"p_boot":p_boot}

# 7) Run experiments (Single + MARL; 5 & 3 bps)

```

```

assets = list(asset_list) # from EDA

# ---- Single-Agent PPO (5 bps) ----

single_dir = RUN_DIR / "single_main_5bps"; single_dir.mkdir(exist_ok=True)

single_main = train_single(panel_close, assets, single_dir, cap=CAP_PER_ASSET,
                           cost_bps=MAIN_COST_BPS, epochs=EPOCHS, lookback=LOOKBACK_BARS,
                           daily_reset=False, use_ctx=True, seed=RANDOM_SEED)

# ---- Single-Agent PPO (3 bps sensitivity) ----

single_dir3 = RUN_DIR / "single_sens_3bps"; single_dir3.mkdir(exist_ok=True)

single_sens = train_single(panel_close, assets, single_dir3, cap=CAP_PER_ASSET,
                           cost_bps=SENS_COST_BPS, epochs=EPOCHS, lookback=LOOKBACK_BARS,
                           daily_reset=False, use_ctx=True, seed=RANDOM_SEED+1)

# ---- MARL CTDE-PPO (MCP OFF, 5 bps) ----

marl_off_dir = RUN_DIR / "marl_mcpOFF_5bps"; marl_off_dir.mkdir(exist_ok=True)

marl_off = train_marl(panel_close, assets, marl_off_dir, cap=CAP_PER_ASSET,
                     cost_bps=MAIN_COST_BPS, epochs=EPOCHS, lookback=LOOKBACK_BARS,
                     daily_reset=False, use_ctx=False, seed=RANDOM_SEED)

# ---- MARL CTDE-PPO (MCP ON, 5 bps) ----

marl_on_dir = RUN_DIR / "marl_mcpON_5bps"; marl_on_dir.mkdir(exist_ok=True)

marl_on = train_marl(panel_close, assets, marl_on_dir, cap=CAP_PER_ASSET,
                    cost_bps=MAIN_COST_BPS, epochs=EPOCHS, lookback=LOOKBACK_BARS,
                    daily_reset=False, use_ctx=True, seed=RANDOM_SEED)

```

```

def plot_equity_all(panel_close, assets, outdir: Path):

    rets = panel_close.pct_change(fill_method=None).dropna()

    train = rets.loc[rets.index < TRAIN_TEST_SPLIT[0], assets].dropna()

    test = rets.loc[(rets.index >= TRAIN_TEST_SPLIT[0]) & (rets.index <= TRAIN_TEST_SPLIT[1]),
assets].dropna()

    vol = train.std(); cov = train.cov()

    cost = MAIN_COST_BPS/10000.0

    # baselines

    ew_eq = simulate_static(test, weights_equal(len(assets)), cost)[1]

    iv_eq = simulate_monthly_rebalanced(test, weights_invvol, vol=vol, cap=CAP_PER_ASSET,
cost_per_unit=cost, method="invvol")[1]

    mv_eq = simulate_static(test, weights_minvar(cov), cost)[1]

    # RL equity (saved)

    #eq_single = pd.read_csv(RUN_DIR / "single_main_5bps" / "equity_oos.csv", index_col=0).iloc[:,0]

    #eq_mcp_on = pd.read_csv(RUN_DIR / "marl_mcpON_5bps" / "equity_oos.csv",
index_col=0).iloc[:,0]

    # eq_mcp_off = pd.read_csv(RUN_DIR / "marl_mcpOFF_5bps" / "equity_oos.csv",
index_col=0).iloc[:,0]

    eq_single = pd.read_csv(RUN_DIR/"single_main_5bps"/"equity_oos.csv", index_col=0,
parse_dates=[0]).iloc[:,0]

    eq_mcp_on = pd.read_csv(RUN_DIR/"marl_mcpON_5bps"/"equity_oos.csv", index_col=0,
parse_dates=[0]).iloc[:,0]

    eq_mcp_off = pd.read_csv(RUN_DIR/"marl_mcpOFF_5bps"/"equity_oos.csv", index_col=0,
parse_dates=[0]).iloc[:,0]

    plt.figure(figsize=(11,5))

```

```

ew_eq.plot(label="EW (static)")

iv_eq.plot(label="InvVol (monthly)")

mv_eq.plot(label="MinVar (static)")

eq_single.plot(label="Single-Agent PPO (5bps)")

eq_mcp_on.plot(label="MARL+MCP (5bps)")

eq_mcp_off.plot(label="MARL (no MCP, 5bps)")

plt.title("OOS Equity  Baselines vs RL")

plt.legend(); plt.tight_layout()

plt.savefig(outdir / "fig_4_1_equity_all.png"); plt.close()


# ---- Consolidated summary ----

summary = pd.DataFrame({

    "Single 5bps": single_main,

    "Single 3bps": single_sens,

    "MARL MCP OFF 5bps": marl_off,

    "MARL MCP ON 5bps": marl_on

}).T

summary_path = RUN_DIR / "summary_metrics.csv"


# --- Consolidated Table 4.1 (all strategies, full metric set) ---

def ann_vol(r):

    r = np.asarray(r, float)

    sd = np.nanstd(r, ddof=1)

    return sd * math.sqrt(BARS_PER_YEAR) if sd==sd else np.nan


def cum_return_from_equity(eq):

```



```

return float(eq[-1] - 1.0)

def aggregate_turnover_cost(turnover_series, cost_per_unit):

    tot_to = float(np.nansum(turnover_series))

    tot_cost = tot_to * cost_per_unit

    return tot_to, tot_cost

table_rows = []

# Baselines (recompute with correct methods so metrics/plots are consistent)

rets = panel_close.pct_change(fill_method=None).dropna()

train = rets.loc[rets.index < TRAIN_TEST_SPLIT[0], assets].dropna()

test  = rets.loc[(rets.index >= TRAIN_TEST_SPLIT[0]) & (rets.index <= TRAIN_TEST_SPLIT[1]),
assets].dropna()

vol = train.std(); cov = train.cov()

base_defs = {

    "EW": cap_project(weights_equal(len(assets)), CAP_PER_ASSET),

    "InvVol": cap_project(weights_invvol(vol), CAP_PER_ASSET),

    "MinVar": cap_project(weights_minvar(cov), CAP_PER_ASSET),

}

for name, w in base_defs.items():

    if name.startswith("InvVol"):

        pr, eq, to = simulate_monthly_rebalanced(

            test, weights_invvol, vol=vol, cap=CAP_PER_ASSET,

```

```

        cost_per_unit=MAIN_COST_BPS/10000.0, method="invvol"
    )

elif name.startswith("MinVar"):

    # static min-var; only initial entry turnover

    pr, eq = simulate_static(test, w, MAIN_COST_BPS/10000.0)

    to = pd.Series(0.0, index=eq.index, name="turnover")

    if len(to) > 0:

        to.iloc[0] = float(np.sum(np.abs(w)))

else: # EW static; only initial entry turnover

    pr, eq = simulate_static(test, w, MAIN_COST_BPS/10000.0)

    to = pd.Series(0.0, index=eq.index, name="turnover")

    if len(to) > 0:

        to.iloc[0] = float(np.sum(np.abs(w)))


# save series for appendix

to.to_csv(RUN_DIR / f"baseline_turnover_{name.replace(' ','_')}.csv")

pr.to_csv(RUN_DIR / f"baseline_returns_{name.replace(' ','_')}.csv")

eq.to_csv(RUN_DIR / f"baseline_equity_{name.replace(' ','_')}.csv")


avol = ann_vol(pr.values)

tot_to, tot_cost = aggregate_turnover_cost(to.values, MAIN_COST_BPS/10000.0)

crow = {

    "Strategy": name,

    "CumReturn": cum_return_from_equity(eq.values),

    "AnnVol": avol,

    "Sharpe": sharpe_ratio(pr.values),

    "Sortino": sortino_ratio(pr.values),

```

```

"MaxDD": max_drawdown(eq.values),

"PSR": probabilistic_sharpe_ratio(pr.values, 0.0),

"DSR": deflated_sharpe_ratio(pr.values, n_trials=5, rho=0.5),

"p_boot": sharpe_block_bootstrap_pvalue(pr.values, B=BOOT_B, block=BOOT_BLOCK,
sr_benchmark=0.0),

"TurnoverCum": tot_to,

"CostCum": tot_cost,

}

table_rows.append(crow)

```

RL strategies

```

rl_defs = [

("Single-Agent PPO (5bps)", RUN_DIR / "single_main_5bps", "single_turnover_series.csv",
"single_metrics.txt"),

("MARL+MCP (5bps)", RUN_DIR / "marl_mcpON_5bps", "marl_turnover_series.csv",
"marl_metrics.txt"),

("MARL (no MCP, 5bps)", RUN_DIR / "marl_mcpOFF_5bps", "marl_turnover_series.csv",
"marl_metrics.txt"),

]

```

for name, d, to_csv, metrics_txt in rl_defs:

```

mtxt = (d / metrics_txt).read_text(encoding="utf-8")

```

```

def _grab(tag):

```

```

    m = re.search(rf"{tag}=[-\\d\\.]+)", mtxt)

```

```

    return float(m.group(1)) if m else np.nan

```

```

sr = _grab("Sharpe"); so = _grab("Sortino"); mdd = _grab("MaxDD")

```

```
psr = _grab("PSR"); dsr = _grab("DSR"); pbt = _grab("p_boot")
```

```
to = pd.read_csv(d / to_csv).iloc[:,1].values
```

```
tot_to, tot_cost = aggregate_turnover_cost(to, MAIN_COST_BPS/10000.0)
```

```
# r = pd.read_csv(d / "returns_oos.csv", index_col=0).iloc[:,0].values
```

```
# eq = pd.read_csv(d / "equity_oos.csv", index_col=0).iloc[:,0].values
```

```
r = pd.read_csv(d / "returns_oos.csv", index_col=0).iloc[:,0].values
```

```
eq = pd.read_csv(d / "equity_oos.csv", index_col=0).iloc[:,0].values
```

```
table_rows.append({  
    "Strategy": name,  
    "CumReturn": cum_return_from_equity(eq),  
    "AnnVol": ann_vol(r),  
    "Sharpe": sr,  
    "Sortino": so,  
    "MaxDD": mdd,  
    "PSR": psr,  
    "DSR": dsr,  
    "p_boot": pbt,  
    "TurnoverCum": tot_to,  
    "CostCum": tot_cost,  
})
```

```
table41 = pd.DataFrame(table_rows).set_index("Strategy")
```

```

table41_path = RUN_DIR / "table_4_1_full_metrics.csv"

table41.to_csv(table41_path)

print(f"[TABLE 4.1] -> {table41_path}")


# Save summary

summary.to_csv(summary_path)

print(f"[SUMMARY] -> {summary_path}")


# Combined equity overlay (Figure 4.1)

fig = plt.figure(figsize=(11,5))

curves = []


# Baselines (same logic as table)

for name, w in base_defs.items():

    if name == "InvVol":

        pr, eq, _ = simulate_monthly_rebalanced(test, weights_invvol, vol=vol, cap=CAP_PER_ASSET,

                                                cost_per_unit=MAIN_COST_BPS/10000.0, method="invvol")

    elif name == "MinVar":

        pr, eq = simulate_static(test, w, MAIN_COST_BPS/10000.0)

    else:

        pr, eq = simulate_static(test, w, MAIN_COST_BPS/10000.0)

    curves.append((name, eq))


# RL curves

for label, d in [("Single-Agent PPO (5bps)", RUN_DIR/"single_main_5bps"),

                ("MARL+MCP (5bps)",    RUN_DIR/"marl_mcpON_5bps"),

                ("MARL (no MCP, 5bps)", RUN_DIR/"marl_mcpOFF_5bps")]:

```

```

#eq = pd.read_csv(d/"equity_oos.csv", index_col=0).iloc[:,0]

eq = pd.read_csv(d/"equity_oos.csv", index_col=0, parse_dates=[0]).iloc[:,0]

curves.append((label, eq))

for name, eq in curves:

    eq.plot(label=name)

plt.legend(); plt.title("OOS Equity Curves  Baselines vs RL")

plt.tight_layout(); fig.savefig(RUN_DIR/"fig_4_1_equity_all.png"); plt.close(fig)

plot_equity_all(panel_close, assets, RUN_DIR)

# ---- Write-up notes ----

rationale = f"""

Design notes:

- Data: Bloomberg EDU, 5-min OHLCV, Jan-2024 to Jun-2025, assets per provided sheets.

- Split: Train < {TRAIN_TEST_SPLIT[0]}, Test = {TRAIN_TEST_SPLIT[0]}..{TRAIN_TEST_SPLIT[1]}.

- Constraints: per-asset cap = {CAP_PER_ASSET:.2f};  $\Sigma w \leq 1$  with cash.

- Costs: main {MAIN_COST_BPS:.1f} bps (sens {SENS_COST_BPS:.1f} bps). Bootstrap block={BOOT_BLOCK} bars (~1 day).

- Baselines: EW (static), MinVar (static from train cov), InvVol (monthly rebalanced, targets from train vol). All capped at rebalance targets; weights drift between rebalances.

- RL: PPO with GAE; Actor (256,128), Critic (256,256,128), clip=0.2, entropy=0.01, minibatch=1024, 10 epochs/update.

- CTDE MARL: central critic over joint obs; per-asset actors.

```

- MCP: binary context alerts with $TTL \approx 30$ min Turnover Alert=1 if prior-hour turnover>50%; Drawdown Alert=1 if running drawdown>5%.

- Trading frictions mitigation: HOLD_N={HOLD_N} (15 min), action smoothing $\alpha=\{\text{ALPHA_ACT}\}$, turnover penalty $\lambda=\{\text{LAMBDA_TURN}\}$.

- Diagnostics saved: equity, drawdown, exposure heatmaps, MARL exposure-corr, turnover, training curves; summary metrics; Table 4.1 with CumReturn/AnnVol/Sharpe/Sortino/MaxDD/PSR/DSR/bootstrap-p/Turnover/CumulativeCost.

"""

with open(RUN_DIR / "READ_ME_FOR_WRITEUP.txt", "w", encoding="utf-8") as f:

 f.write(rationale)

print(rationale)

8) FINAL INLINE VIEWER

%%

from IPython.display import display, Markdown, Image

def show_img(path, width=980):

 p = Path(path)

 if not p.exists():

 print(f"[SKIP] {p.relative_to(RUN_DIR)} (not created yet)")

 return

 display(Markdown(f"#### {p.relative_to(RUN_DIR)}"))

 display(Image(filename=str(p), width=width))

def show_table(path, n=10):

 p = Path(path)

 if not p.exists():

```

    print(f"[SKIP] {p.relative_to(RUN_DIR)} (not created yet)")

    return

display(Markdown(f"#### {p.relative_to(RUN_DIR)}"))

display(pd.read_csv(p).head(n))


display(Markdown("## Run Outputs"))

print(f"[RUN DIR] {RUN_DIR.resolve()}")


# 1) EDA

display(Markdown("### EDA"))

show_table(RUN_DIR / "audit_summary.csv")

for name in ["eda_normalized.png", "eda_return_hist.png", "eda_intraday_profile.png",
            "eda_corr_train.png", "eda_corr_test.png"]:

    show_img(RUN_DIR / name)


# 2) Baselines

display(Markdown("### Baselines"))

for cost in (int(MAIN_COST_BPS), int(SENS_COST_BPS)):

    show_table(RUN_DIR / f"baselines_metrics_cost{cost}bps.csv")

# baseline weight bars

for p in sorted(RUN_DIR.glob("wgt_*.png")):

    show_img(p)

# baseline equities + turnover csvs

for p in sorted(RUN_DIR.glob("eq_*.png")):

    show_img(p)

for p in sorted(RUN_DIR.glob("baseline_turnover_*.csv")):

    show_table(p, n=15)

```


3) Single-Agent PPO (5 bps)

```
display(Markdown("### Single-Agent PPO (5 bps)"))

sd = RUN_DIR / "single_main_5bps"

for name in ["train_return_curve.png", "train_entropy_curve.png",

"single_equity.png", "single_drawdown.png", "single_turnover.png", "single_exposure_heatmap.png"]:

    show_img(sd / name)

for name in
["single_metrics.txt", "returns_oos.csv", "equity_oos.csv", "single_exposures.csv", "single_turnover_series.
csv", "weights.csv"]:

    p = sd / name

    if p.suffix == ".txt":

        if p.exists():

            display(Markdown(f"#### {p.relative_to(RUN_DIR)}"))

            print(p.read_text(encoding="utf-8"))

        else:

            show_table(p, n=10)
```

4) Single-Agent PPO (3 bps)

```
display(Markdown("### Single-Agent PPO (3 bps)"))

sd3 = RUN_DIR / "single_sens_3bps"

for name in ["train_return_curve.png", "train_entropy_curve.png",

"single_equity.png", "single_drawdown.png", "single_turnover.png", "single_exposure_heatmap.png"]:

    show_img(sd3 / name)

for name in
["single_metrics.txt", "returns_oos.csv", "equity_oos.csv", "single_exposures.csv", "single_turnover_series.
csv", "weights.csv"]:
```

```

p = sd3 / name

if p.suffix == ".txt":

    if p.exists():

        display(Markdown(f"#### {p.relative_to(RUN_DIR)}"))

        print(p.read_text(encoding="utf-8"))

    else:

        show_table(p, n=10)

# 5) MARL (MCP OFF / ON)

for label, d in [("#### MARL CTDE-PPO (MCP OFF, 5 bps)", RUN_DIR / "marl_mcpOFF_5bps"),

                ("#### MARL CTDE-PPO (MCP ON, 5 bps)", RUN_DIR / "marl_mcpON_5bps")]:

    display(Markdown(label))

    for name in ["train_return_curve.png", "train_entropy_curve.png",

                "marl_equity.png", "marl_drawdown.png", "marl_turnover.png", "marl_exposure_heatmap.png", "marl_exp

osure_corr.png"]:

        show_img(d / name)

    for name in

["marl_metrics.txt", "returns_oos.csv", "equity_oos.csv", "marl_exposures.csv", "marl_turnover_series.csv",

"marl_exposure_corr.csv", "weights.csv"]:

        p = d / name

        if p.suffix == ".txt":

            if p.exists():

                display(Markdown(f"#### {p.relative_to(RUN_DIR)}"))

                print(p.read_text(encoding="utf-8"))

            else:

                show_table(p, n=10)

```

6) Summary

```
display(Markdown("### Summary metrics"))
```

```
show_table(RUN_DIR / "summary_metrics.csv", n=20)
```

```
display(Markdown("### Table 4.1 (full metrics)"))
```

```
show_table(RUN_DIR / "table_4_1_full_metrics.csv", n=20)
```

```
display(Markdown("### Combined equity (Figure 4.1)"))
```

```
show_img(RUN_DIR / "fig_4_1_equity_all.png")
```

Append write-up notes

```
display(Markdown("### Write-up notes"))
```

```
p = RUN_DIR / "READ_ME_FOR_WRITEUP.txt"
```

```
if p.exists():
```

```
    print(p.read_text(encoding="utf-8"))
```

9)

%%

```
RUN_ROBUSTNESS = False
```

```
if RUN_ROBUSTNESS:
```

```
    SEEDS = [41, 42, 43]
```

```
    ROBUST_DIR = RUN_DIR / "robustness"; ROBUST_DIR.mkdir(exist_ok=True)
```

```
    configs = [
```

```

("Single 5bps", lambda s, d: train_single(panel_close, assets, d, cap=CAP_PER_ASSET,
cost_bps=MAIN_COST_BPS, epochs=EPOCHS, lookback=LOOKBACK_BARS, daily_reset=False,
use_ctx=True, seed=s)),

("MARL MCP OFF 5bps", lambda s, d: train_marl (panel_close, assets, d, cap=CAP_PER_ASSET,
cost_bps=MAIN_COST_BPS, epochs=EPOCHS, lookback=LOOKBACK_BARS, daily_reset=False,
use_ctx=False, seed=s)),

("MARL MCP ON 5bps", lambda s, d: train_marl (panel_close, assets, d, cap=CAP_PER_ASSET,
cost_bps=MAIN_COST_BPS, epochs=EPOCHS, lookback=LOOKBACK_BARS, daily_reset=False,
use_ctx=True, seed=s)),

]

rows=[]

for cfg_name, runner in configs:

    for s in SEEDS:

        outdir = ROBUST_DIR / f'{cfg_name.replace(' ','_')} / f'seed_{s}'

        outdir.mkdir(parents=True, exist_ok=True)

        metrics = runner(s, outdir)

        row = {"Config": cfg_name, "Seed": s}; row.update(metrics); rows.append(row)

df = pd.DataFrame(rows); df.to_csv(ROBUST_DIR/"metrics_by_seed.csv", index=False)

summary =
df.groupby("Config").agg({"Sharpe":["mean","std"],"Sortino":["mean","std"],"MaxDD":["mean","std"],"
PSR":["mean"],"DSR":["mean"],"p_boot":["mean"]}).round(3)

summary.to_csv(ROBUST_DIR/"summary_mean_std.csv")

display(summary)

fig = plt.figure(figsize=(8,4))

means = summary['Sharpe']['mean']; stds = summary['Sharpe']['std']

means.plot(kind='bar', yerr=stds, capsize=3)

plt.title("Sharpe (mean  $\pm$  std) across seeds"); plt.tight_layout()

fig.savefig(ROBUST_DIR / "sharpe_seed_bar.png"); plt.close(fig)

print(f"[ROBUSTNESS] Saved to {ROBUST_DIR}")

```

